

Arba Minch University

Arba Minch Institute of Technology



Faculty of computing and software Engineering (FCSE)

Exit Exam Material for Design and Analysis of Algorithm

(CoSec3094)

B.Sc. Computer science

Prepared by Yitayish Lema (MSC.)

Email: Yitulema@gmail.com

March April 10, 2023 G.C.

Arba Minch, Ethiopia

Contents

CHAPTER 1: INTRODUCTION	1
1.1 INTRODUCTION TO ANALYSIS	1
1.1.1 Algorithm Analysis:	1
1.1.2 Asymptotic Notations:	7
1.1.3 Basic Efficiency classes	8
1.2 Plan for Analysis of Recursive Algorithms	10
1.2.1 SUBSTITUTION METHOD:	11
1.2.2 ITERATIVE METHOD:	14
1.2.3 RECURSSION TREE METHOD:	15
1.2.4 BY MASTER METHOD	17
1.3 Analysis of Linear search:	18
1.4 Heaps and Heap sort	19
1.5 Lower Bounds for Sorting	25
1.6 Lower Bounds for Comparison-Based Sorting:	27
1.7 Hashing	29
8.2.2 Linear Probing	30
CHAPTER 2: DIVIDE AND CONQUER	31
2.1 General Method:	31
2.2 Binary search:	33
2.4 Finding Maximum and minimum	40
CHAPTER 3: GREEDY METHOD	45
3.1 Introduction	45
3.2 Fractional Knapsack Problem	46
3.3 Job Sequencing	47
3.4 Optimal Merge Pattern	50
3.5 Huffman Code	51

Design and Analysis of Algorithm

3.6 Minimum Spanning tree	58
3.6.1 Prim's Algorithm	59
3.6.2 Kruskal's Algorithm	63
3.7 Shortest Path Problem	67
3.7.1 Dijkstra's Algorithm	68
Chapter 4: Dynamic Programming	69
4.1 Introduction	69
4.2 Multi stage Graphs.....	73
4.3 ALL PAIRS SHORTEST PATH PROBLEM:.....	78
Analysis:	79
4.4 0/1 KNAPSACK PROBLEM.....	80
4.5 Representation of Graphs:	84
4.5.1 Adjacency matrix:	84
4.5.2 Adjacency list:	84
4.6 Graph Traversals.....	85
4.6.1 Breadth First search (BFS)	85
4.6.2 Depth First Search (DFS)	85
CHAPTER 5: BACKTRACKING.....	88
5.1 N-Queen's Problem	89
5.2 Sum of Subsets	95
5.3 Graph coloring.....	96
CHAPTER 6: Introduction to Probabilistic Algorithms	101
REFERENCES	104
Summary Questions	105

CHAPTER 1: INTRODUCTION

1.1 INTRODUCTION TO ANALYSIS

Introduction:

- An algorithm is a well-defined procedure or a finite number of steps to be followed for solving a problem.
- This is given in pseudo code format and is independent and implement in any programming language and solved using any type of computers.
- The efficiency of the algorithm plays a major role in determining the efficiency of the program when its implemented.

The main **characteristics of the algorithm** are

- ◆ Must take an **input**—zero or more quantities or externally supplied.
- ◆ Must give some **output** (yes/no, value etc.)- at least one quantity must be produced.
- ◆ **Definiteness** –each instruction is clear and unambiguous.
- ◆ **Finiteness** – if we trace out the instructions of the algorithm , then for all the cases, algorithm terminates after a finite number of steps.
- ◆ **Effectiveness** –every instruction must be basic i.e. simple instruction.

Before writing code (program) for solving a problem, a good programmer first design and writes the concerned algorithm. Analysis it and refines it as many times as required and arrival at the final efficient from that works well for all valid input data and solves the problem in the shortest possible time, utilizing minimum memory space.

Factors in designing and analyzing algorithms:

Followings are some of the basic factors in designing and analyzing algorithms:

- **Designing Algorithm:** this part consists of designing and writing algorithm for solving problems.
- **Validation of Algorithm:** First, we design the required algorithm and check whether it computes the correct answer for all possible valid inputs. This part of checking is called **validation of algorithm**. It has to be proved that the program is correct. This proof involves establishing that each statement in itself is well defined as well as correctness of all basic operation.
- **Analyzing of algorithm:** it involves the estimating the computing time and storage space requirements.
- **Testing a Program:** write a program and execute it with some sample data and check whether you get an output. This part is called *debugging*.

1.1.1 Algorithm Analysis:

- An algorithm is mainly analyzed to determine the execution time of the program and how much memory space required.
- Algorithm analysis is required to measure the efficiency of the algorithm and to see how the execution time varies with size of the input.

Design and Analysis of Algorithm

Resource usage:

Here, the time is considered to be the primary measure of efficiency. We are also concerned with how much the respective algorithm involves the computer memory. But mostly time is the resource that is dealt with. And the actual running time depends on a variety of backgrounds: like the speed of the Computer, the language in which the algorithm is implemented, the compiler/interpreter, skill of the programmers etc.

So, mainly the resource usage can be divided into: 1.Memory (space) 2.Time

Time taken by an algorithm?

- Performance measurement or Aposteriori Analysis: Implementing the algorithm in a machine and then calculating the time taken by the system to execute the program successfully.
- Performance Evaluation or Apriori Analysis. Before implementing the algorithm in a system.

Analysis of algorithm is important for the following reasons:

- ◆ **Analysis is more reliable than the experimental testing---** analysis gives the performance of the algorithm for all cases of inputs, whereas testing is done only on specific cases of input
- ◆ **Analysis helps to select better algorithm----** when there are different algorithms for the same task, analysis helps to pick out the most efficient algorithm.
- ◆ **Analysis predicts performance---** analysis can be used to predict the run time of the algorithm. If the algorithm is analyzed to be very slow, then it need not be implemented at all.
- ◆ **Analysis identifies the scope of improvement of algorithm---** analysis of algorithm can find out which portion of the algorithm is faster and which is slower part may then be modified, if possible to increase execution time.

Performance Analysis:

The efficiency of the algorithm can be decided by measuring the performance of an algorithm. we can measure the performance of an algorithm by computing two factors:

1. Amount of time required by an algorithm to execute.
2. Amount of storage required by an algorithm.

Design and Analysis of Algorithm

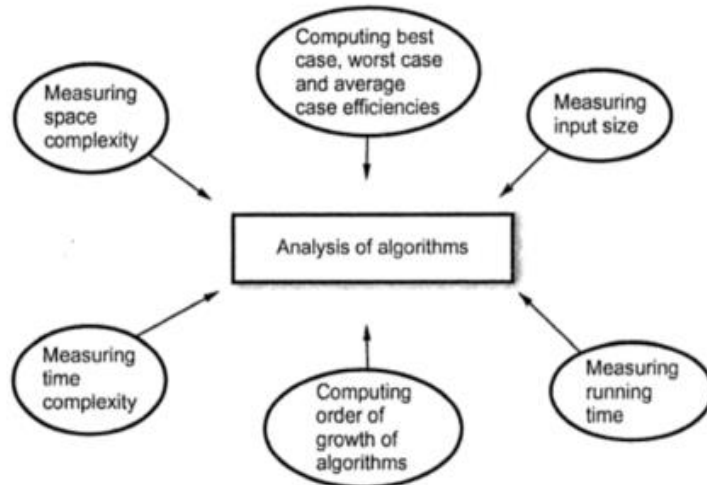


Fig. 1.4 Analysis of algorithms

Space Complexity:

- The space complexity of an algorithm P is the amount of memory it requires for running the algorithm.
- To compute the space complexity we use two factors: constant and instance characteristics.
- It has two parts, i.e. a fixed and variable part.

Thus,

$$S(P) = c + S_p(\text{instance characteristics})$$

Where c is a constant.

- The fixed part component c is independent of the characteristics of inputs and outputs and depends on the space for the code and space for certain type of variables.
- The variable part component S_p depends upon the space needed by the component variables, referenced variables and the recursion stack space.
- This recursion stack space includes the space for the formal parameters, local variables and the return address.
- We estimate S_p for the given program P and this method is illustrated by the following two simple algorithms.

An iterative algorithm for the sum and product of elements of an array.

Algorithm Totprod(a,n)

```
{
Tot:=0.0;
Prod:=1.0;
For k:=1 to n do
{
Tot:=Tot +a[k];
Prod:=Prod+a[k];
}
returnTot;
return Prod;
}
```

Design and Analysis of Algorithm

In this algorithm the space needed by n, k, Tot and Prod is one word for each and for the array a[] is at least n words.

Thus,

$$S_p = S_{\text{ArrayTot}}(n) \geq n+4$$

Array elements product-A recursive algorithm

Algorithm RProd(a,n)

```
{  
If(n=1) then  
return a[1];  
else  
return RProd(a,n-1)*a[n];  
}
```

- ◆ In this algorithm ,the instances are characterized by n.
- ◆ The internal stack used for recursion includes space for formal parameters, local variables and return address.the space required by each cell to function.
- ◆ Let us suppose that the return address requires one word of memory. Atleast three words(space for n,space for the return address and a pointer to a[]) are required for each call to RProd.
- ◆ Thus,the recursion stack space is greater than or equal to $3(n+1)$,as depth of recursion is $(n+1)$ (n times call to function and one return call).

TIME COMPLEXITY:

- The time complexity of an algorithm is the amount of computer time required by an algorithm to run the completion.
- It is difficult to compute the time complexity in terms of physically clocked time. For instance in multi user system, executing time depends on many factors such as—
 - ◆ System load,
 - ◆ Number of other programs running,
 - ◆ Instruction set used,
 - ◆ Speed of underlying hardware.
- The time complexity is therefore given in terms of frequency count.
- Frequency count is a count denoting number of times of execution of statement.

Each execution of the program is associated with two types of times:

1. The complete time which does not depend upon the instance characteristics?
 2. The run time of the program
- The time T_p taken by the program is sum of the compile time and run time.

Design and Analysis of Algorithm

Analysis Rules:

1. We assume an arbitrary time unit.
2. Execution of one of the following operations takes time 1:
 - Assignment Operation
 - Single Input/Output Operation
 - Single Boolean Operations
 - Single Arithmetic Operations
 - Function Return
3. Running time of a selection statement (if, switch) is the time for the condition evaluation + the maximum of the running times for the individual clauses in the selection.
4. Loops: Running time for a loop is equal to the running time for the statements inside the loop * number of iterations.
The total running time of a statement inside a group of nested loops is the running time of the statements multiplied by the product of the sizes of all the loops.
For nested loops, analyze inside out.
 - Always assume that the loop executes the maximum number of iterations possible.
5. Running time of a function call is 1 for setup + the time for any parameter calculations + the time required for the execution of the function body.

Examples:

```
1. int count(){
int k=0;
cout<< "Enter an integer";
cin>>n;
for (i=0;i<n;i++)
    k=k+1;
return 0;}
```

Time Units to Compute

1 for the function
1 for the assignment statement: int k=0
1 for the output statement.
1 for the input statement.
In the for loop:
 1 assignment, $n+1$ tests, and n increments.
 n loops of 2 units for an assignment, and an addition.
 1 for the return statement.

$$T(n) = 1 + 1 + 1 + 1 + (1 + n + 1 + n) + 2n + 1 = 4n + 7$$

```
2. int total(int n)
{
```

Design and Analysis of Algorithm

```
int sum=0;
for (int i=1;i<=n;i++)
sum = sum+1;
return sum;
}
```

Time Units to Compute

1 for the function

1 for the assignment statement: `int sum=0`

In the for loop:

1 assignment, $n+1$ tests, and n increments.

n loops of 2 units for an assignment, and an addition.

1 for the return statement.

$$T(n) = 1 + 1 + (1 + n + 1 + n) + 2n + 1 = 4n + 5$$

3. voidfunc()

```
{
int x=0;
int i=0;
int j=1;
cout<< "Enter an Integer value";
cin>>n;
while (i<n){
x++;
i++;
}
while (j<n)
{
j++;
}
}
```

Time Units to Compute

1 for the function

1 for the first assignment statement: `x=0;`

1 for the second assignment statement: `i=0;`

1 for the third assignment statement: `j=1;`

1 for the output statement.

1 for the input statement.

In the first while loop:

$n+1$ tests

n loops of 2 units for the two increment (addition) operations

In the second while loop:

Design and Analysis of Algorithm

n tests
n-1 increments

$$T(n) = 1 + 1 + 1 + 1 + 1 + 1 + n + 1 + 2n + n + n - 1 = 5n + 6$$

```
4. int sum (int n)
{
    int partial_sum = 0;
    for (int i = 1; i <= n; i++)
        partial_sum = partial_sum + (i * i * i);
    return partial_sum;
}
```

Time Units to Compute

1 for the function
1 for the assignment.
1 assignment, $n+1$ tests, and n increments.
 n loops of 4 units for an assignment, an addition, and two multiplications.
1 for the return statement.

$$T(n) = 1 + 1 + (1 + n + 1 + n) + 4n + 1 = 6n + 5$$

1.1.2 Asymptotic Notations:

- To choose the best algorithm, we need to check efficiency of each algorithm.
- The efficiency can be measured by computing time complexity of each algorithm.
- Asymptotic notation is a shorthand way to represent the time complexity.

Analysis of algorithm is concerned on 3 cases:

- **Best case** complexity- it is the function defined by the minimum number of steps taken on any instance of size n .
- **Average case** complexity- it is the function defined by the average number of steps taken on any instance of size n .
- **Worst case** complexity- it is the function defined by the maximum number of steps taken on any instance of size n .

Various notations such as Ω , Θ and O used are called asymptotic notations.

Big oh Notation: (Worst case):

The Big oh notation is denoted by 'O'. It is a method of representing the upper bound of algorithm's running time. Using Big oh notation we can give longest amount of time taken by the algorithm to complete.

Definition:

Let $F(n)$ and $g(n)$ be two non negative functions.

Let n^0 and constant c are two integers such that n^0 denotes some value of input and $n > n^0$. Similarly c is some constant such that $c > 0$. We can write

Design and Analysis of Algorithm

$F(n) \leq c * g(n)$

Then $F(n)$ is big oh of $g(n)$. it is also denoted as $F(n) \in O(g(n))$. in other way $F(n)$ is less than $g(n)$ if $g(n)$ is multiple of some constant c .

Thus always upper bound of existing time is obtained by Big oh notation.

Omega notation(Best case)

It is denoted by Ω . this notation is used to represent the lower bound of algorithm's running time. using omega notation we can denote shortest amount of time taken by algorithm.

Definition:

A function $F(n)$ is said to be in $\Omega(g(n))$ if $F(n)$ is bounded below by some positive constant multiple of $g(n)$ such that

$$F(n) \geq c * g(n).$$

It is denoted as $F(n) \in \Omega(g(n))$.

Theta notation:(average case)

It is denoted by Θ . By this method, the running time is between upper bound and lower bound.

Definition: let $F(n)$ and $g(n)$ be two non negative functions. There are two positive constants namely c_1 and c_2 such that

$$c_1 \leq g(n) \leq c_2 g(n)$$

Then we can say that

$$F(n) \in \Theta(g(n))$$

Properties of order of growth:

1. if $F_1(n)$ is order of $g_1(n)$ and $f_2(n)$ is order of $g_2(n)$, then $F_1(n) + f_2(n) \in O(\max(g_1(n), g_2(n)))$.

2. polynomials of degree $m \in \Theta(n^m)$ maximum degree is considered from the polynomial.

3. for eg: $a_1n^3 + a_2n^2 + a_3n + c$ has the order of growth $\Theta(n^3)$.

4. $O(1) < O(\log n) < O(n) < O(n^2) < O(2n)$

4. Exponential function a^n have different orders of growth for different values of a .

Key points:

1. $O(g(n))$ is a class of functions $F(n)$ that grows less fast than $g(n)$, that means $F(n)$ possesses the time complexity which is always lesser than the time complexities of $g(n)$.
2. $\Theta(g(n))$ is a class of functions $F(n)$ that grows at same rate as $g(n)$.
3. $\Omega(g(n))$ is a class of functions $F(n)$ that grows faster than $g(n)$, that means $F(n)$ possesses the time complexity which is always greater than the time complexities of $g(n)$.

1.1.3 Basic Efficiency classes

Classification of different order of growth

Name of efficiency class	Order of growth	Description	Example
Constant	1	As input size grows that we get larger running time	Scanning array of elements
Logarithmic	$\log n$	When we get logarithmic running time then it is sure that the algorithm	Performing binary search operation

Design and Analysis of Algorithm

		does not consider all its input rather the problem is divided into smaller parts on each iteration.	
Linear	N	The running time of the algorithm depends on the input size n.	Performing sequential search operation
Nlogn	Nlogn	Some instance of input is considered for the list of size n.	Sorting the elements search operation
Quadratic	n^2	When the algorithm has two nested loops then this type of efficiency occurs.	Scanning matrix elements.
Cubic	n^3	When the algorithm has three nested loops then this type of efficiency occurs.	Performing matrix multiplication
Exponential	2^n	When the algorithm has very faster rate of growth then this type of efficiency occurs.	Generating all subsets of elements.
Factorial	$n!$	When an algorithm is computing all the permutations then this type of efficiency occurs.	Generating all permutations

Properties of Big-oh:

Design and Analysis of Algorithm

Following are some important properties of big oh notations -

1. If there are two functions $f_1(n)$ and $f_2(n)$ such that $f_1(n) = O(g_1(n))$ and $f_2(n) = O(g_2(n))$ then $f_1(n) + f_2(n) = \max(O(g_1(n)), O(g_2(n)))$.
2. If there are two functions $f_1(n)$ and $f_2(n)$ such that $f_1(n) = O(g_1(n))$ and $f_2(n) = O(g_2(n))$ then $f_1(n) * f_2(n) = O(g_1(n) * g_2(n))$.
3. If there exists a function f_1 such that $f_1 = f_2 * c$ where c is the constant then, f_1 and f_2 are equivalent. That means $O(f_1 + f_2) = O(f_1) = O(f_2)$.
4. If $f(n) = O(g(n))$ and $g(n) = O(h(n))$ then $f(n) = O(h(n))$.
5. In a polynomial the highest power term dominates other terms.
For example if we obtain $3n^3 + 2n^2 + 10$ then its time complexity is $O(n^3)$.
6. Any constant value leads to $O(1)$ time complexity. That is if $f(n) = c$ then it $\in O(1)$ time complexity.
7. If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$ then $f(n) \in O(g(n))$ but $f(n) \notin \theta(g(n))$.

Recurrences, Solution of Recurrences by substitution, Recursion Tree and Master Method

Recursion is a particularly powerful kind of reduction, which can be described loosely as follows:

- If the given instance of the problem is small or simple enough, just solve it.
- Otherwise, reduce the problem to one or more simpler instances of the same problem.

1.2 Plan for Analysis of Recursive Algorithms

- Decide on a parameter indicating an input's size.
- Identify the algorithm's basic operation.
- Check whether the number of times the basic op. is executed may vary on different inputs of the same size. (If it may, the worst, average, and best cases must be investigated separately.)
- Set up a recurrence relation with an appropriate initial condition expressing the number of times the basic op. is executed.
- Solve the recurrence by substitutions or another method.

Recursion is generally expressed in terms of recurrences. In other words, when an algorithm calls to itself, we can often describe its running time by a **recurrence equation** which describes the overall running time of a problem of size n in terms of the running time on smaller inputs.

E.g. the worst case running time $T(n)$ of the merge sort procedure by recurrence can be expressed as

$$T(n) = \Theta(1) ; \text{ if } n=1$$

$$2T(n/2) + \Theta(n) ; \text{ if } n>1 \text{ whose solution can be found as } T(n) = \Theta(n \log n)$$

Design and Analysis of Algorithm

There are various techniques to solve recurrences.

1.2.1 SUBSTITUTION METHOD:

It is a kind of method in which a **guess** for the solution is made.

There are two types of substitution

1. Forward substitution method
2. Backward substitution method

Forward substitution method:

This method makes use of an initial condition in the initial term and the value for the next term is generated. This process is continued until some formula is guessed. Thus, in this kind of substitution method, we use recurrence equation to generate few terms.

For example:

Eg. Consider a recursive relation

$$T(n) = T(n-1) + n \text{ -----1} \quad \text{with initial condition } T(0) = 0$$

$$\text{Let } T(n) = T(n-1) + n$$

If $n=1$ then

$$T(1) = T(0) + 1 \text{ (as per initial condition } T(0) = 0)$$

$$T(1) = 0.$$

If $n=2$ then

$$T(2) = T(1) + 2 = 1 + 2 = 3$$

If $n=3$ then

$$T(3) = T(2) + 3 = 3 + 3 = 6$$

By observing above generated equations we can derive a formula

$$T(n) = \frac{n(n+1)}{2} = \frac{n^2}{2} + \frac{n}{2}$$

We can also denote $T(n)$ in terms of big Oh notation as: $T(n) = O(n^2)$.

But in practice, it is difficult to guess the pattern from forward substitution method. Hence this method is not used very often.

Backward Substitution Method:

Design and Analysis of Algorithm

In this method, backward values are substituted recursively in order to derive some formula.

Eg. Consider a recursive relation

$$T(n) = T(n-1) + n \text{ -----1} \quad \text{with initial condition } T(0) = 0$$

Substitute $n=n-1$ in 1

$$T(n-1) = T(n-1-1) + (n-1) = T(n-2) + (n-1) \text{ -----2}$$

Put 2 in 1

$$T(n) = T(n-2) + (n-1) + n \text{ -----3}$$

Put $n=n-2$ in 1

$$T(n-2) = T(n-2-1) + (n-2) \text{ -----4}$$

Put 4 in 3

$$T(n) = T(n-3) + (n-2) + (n-1) + n$$

Generally

$$T(n) = T(n-k) + (n-k+1) + (n-k+2) + \dots + n$$

If $k=n$

$$T(n) = T(0) + 1 + 2 + \dots + n \text{ since } T(0) = 0$$

$$T(n) = 0 + 1 + 2 + 3 + \dots + n$$

$$T(n) = n(n+1) / 2$$

In terms of big oh notation

$$T(n) \in O(n^2)$$

Examples:

Solve the recurrence relation $T(n) = T(n-1) + 1$ with initial condition $T(0) = 0$.also find big oh notation .

Solution:

$$T(n) = T(n-1) + 1 \text{ -----1}$$

By substitution method

Put $n= n-1$

$$T(n-1) = T(n-1-1) + 1 = (n-2) + 1 \text{ -----2}$$

Substitute 2 in 1

Design and Analysis of Algorithm

$$T(n) = T(n-2) + 1 + 1 = T(n-2) + 2 \text{ -----3}$$

Again $n = n-2$ in 1

$$T(n-2) = T(n-2) + 1 \text{ -----4}$$

Sub 4 in 3

$$T(n) = T(n-3) + 1 + 2 = T(n-3) + 3$$

Generally

$$T(n) = T(n-k) + K$$

If $k=n$

$$T(n) = T(n-n) + n$$

$$T(n) = T(0) + n$$

$$T(n) = n$$

In big oh notation $T(n) \in O(n)$

Example:

Solve the recurrence relation $T(n) = 2T(n/2) + n$ with $T(1) = 1$ as initial condition

The algorithm having two $n/2$, calls to itself

Solution:

$$T(n) = 2T(n/2) + n \text{ -----1}$$

By substitution method,

Put $n = n/2$

$$T(n/2) = 2T\{(n/2)/2\} + n/2 = 2T(n/4) + n/2 \text{ -----2}$$

Sub 2 in 1

$$T(n) = 2\{2T(n/4) + n/2\} + n = 4T(n/4) + 2n \text{ -----3}$$

Put $n = n/4$ in 1

$$T(n/4) = 2T\{(n/4)/2\} + n/4 = 2T(n/8) + n/4 \text{ -----4}$$

Sub 4 in 3

$$T(n) = 4\{2T(n/8) + n/4\} + 2n$$

$$T(n) = 8T(n/8) + 3n$$

Design and Analysis of Algorithm

Generally

$$T(n) = 2^k T(n/2^k) + kn$$

Consider $2^k = n$ and $k = \log_2 n$

$$T(n) = nT(n/n) + \log_2 n \cdot n$$

$$= n \cdot 1 + n \log n \text{ since } T(1)$$

$$= n + n \log n$$

$$T(n) = n \log n$$

In big oh notation

$$T(n) \in O(n \log n)$$

1.2.2 ITERATIVE METHOD:

$$\text{e.g. } T(n) = 2T(n/2) + n$$

$$\Rightarrow 2[2T(n/4) + n/2] + n$$

$$\Rightarrow 2^2 T(n/4) + n + n$$

$$\Rightarrow 2^2 [2T(n/8) + n/4] + 2n$$

$$\Rightarrow 2^3 T(n/2^3) + 3n$$

$$\text{After } k \text{ iterations, } T(n) = 2^k T(n/2^k) + kn \text{ ----- (1)}$$

Sub problem size is 1 after $n/2^k = 1 \Rightarrow k = \log n$

So, after $\log n$ iterations, the sub-problem size will be

1. So, when $k = \log n$ is put in equation 1

$$T(n) = nT(1) + n \log n$$

$$\Rightarrow nc + n \log n \quad (\text{say } c = T(1))$$

$$\Rightarrow O(n \log n)$$

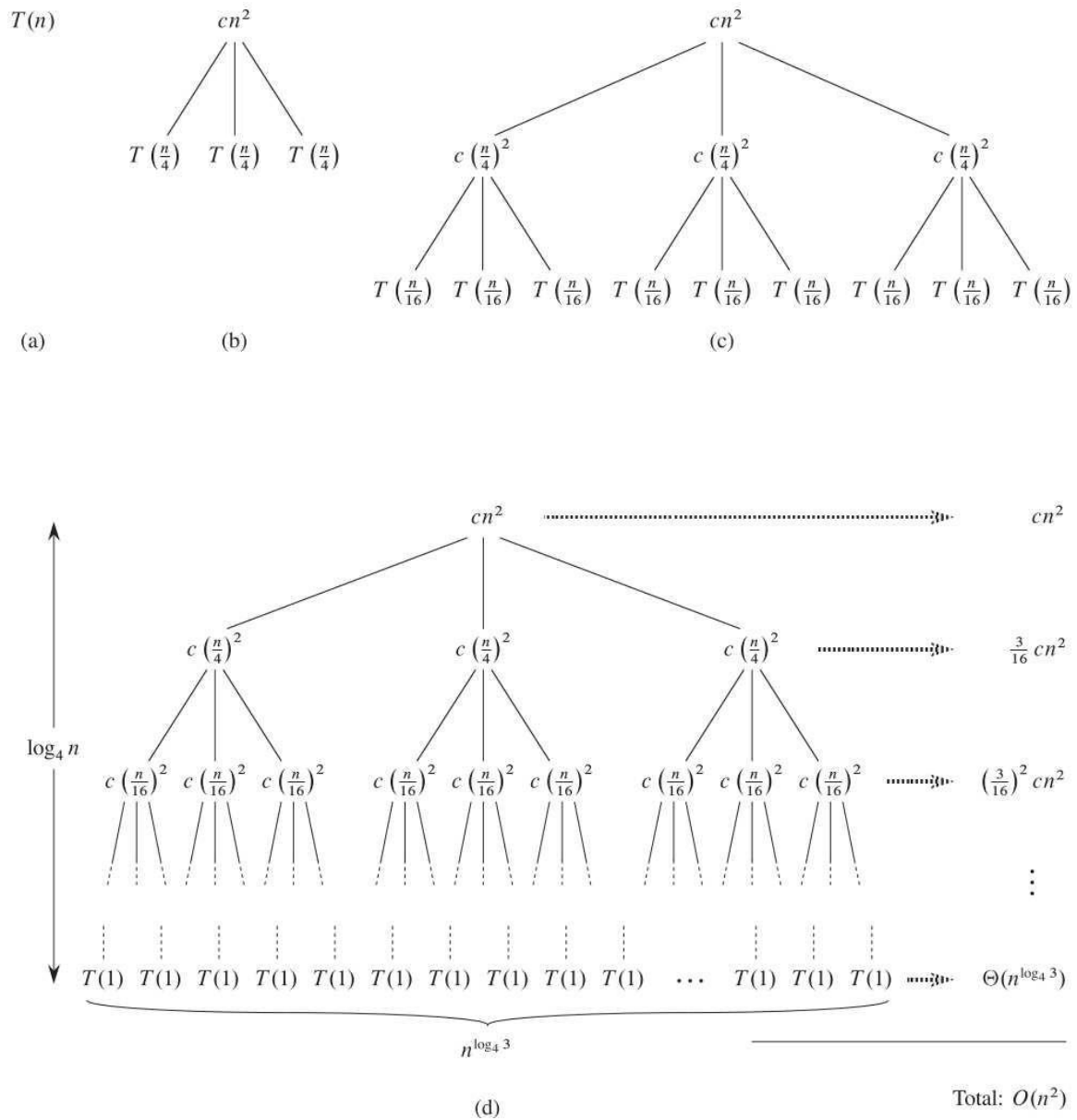
Design and Analysis of Algorithm

1.2.3 RECURSSION TREE METHOD:

In a recursion tree ,each node represents the cost of a single sub-problem somewhere in the set of recursive problems invocations .we sum the cost within each level of the tree to obtain a set of per level cost,and then we sum all the per level cost to determine the total cost of all levels of recursion .

Constructing a recursion tree for the recurrence $T(n)=3T(n/4)+cn^2$

Design and Analysis of Algorithm



Constructing a recursion tree for the recurrence $T(n) = 3T(n/4) + cn^2$. Part (a) shows $T(n)$, which progressively expands in (b)–(d) to form the recursion tree. The fully expanded tree in part (d) has height $\log_4 n$ (it has $\log_4 n + 1$ levels).

Sub problem size at depth $i = n/4^i$

Sub problem size is 1 when $n/4^i = 1 \Rightarrow i = \log_4 n$

So, no. of levels $= 1 + \log_4 n$

Cost of each level = (no. of nodes) $\times$ (cost of each node)

Design and Analysis of Algorithm

No. Of nodes at depth $i=3^i$

Cost of each node at depth $i=c(n/4^i)^2$

Cost of each level at depth $i=3^i c(n/4^i)^2 = (3/16)^i cn^2 T(n) = \sum_{i=0}^{\log_4 n} cn^2 (3/16)^i$

$T(n) = \sum_{i=0}^{\log_4 n - 1} cn^2 (3/16)^i + \text{cost of last level}$

Cost of nodes in last level $= 3^i T(1)$

$$\Rightarrow c3^{\log_4 n} \text{ (at last level } i=\log_4 n)$$

$$\Rightarrow cn^{\log_4 3}$$

$$T(n) = \sum_{i=0}^{\log_4 n - 1} cn^2 (3/16)^i + cn^{\log_4 3}$$

$$<= cn^2 \sum_{i=0}^{\infty} (3/16)^i + cn^{\log_4 3}$$

$$\Rightarrow <= cn^2 * (16/13) + cn^{\log_4 3} \Rightarrow T(n) = O(n^2)$$

1.2.4 BY MASTER METHOD

The master method solves recurrences of the form

$$T(n) = aT(n/b) + f(n)$$

where $a \geq 1$ and $b > 1$ are constants and $f(n)$ is a asymptotically positive function .To use the master method, we have to remember 3 cases:

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constants $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$
2. If $f(n) = \Theta(n^{\log_b a})$ then $T(n) = \Theta(n^{\log_b a} \log n)$
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$,and if $a*f(n/b) \leq c*f(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$

Design and Analysis of Algorithm

e.g. $T(n) = 2T(n/2) + n \log n$

ans: $a=2$ $b=2$

$f(n) = n \log n$

using 2nd formula $f(n) = \Theta(n^{\log 2} \log^k n)$

$\Rightarrow \Theta(n^1 \log^k n) = n \log n \Rightarrow K=1$

$T(n) = \Theta(n^{\log 2} \log^1 n) \Rightarrow \Theta(n \log^2 n)$

1.3 Analysis of Linear search:

For analysis of linear search we will consider 3 cases—best case, worst case and average case. In linear search algorithm, if we get the element at the first place then its **best case** complexity because in this case algorithm takes minimum amount of time to run to its completion. But if the element is present at the end of the list or not at all present in the list, then it is called **worst time** complexity because this case algorithm takes maximum amount of time to run to its completion.

But the time complexity that we get for certain set of input is an average the same. Then for corresponding input, the time complexity is called **average case time** complexity.

Consider the following algorithm

Algorithm Linear search($x\{0, \dots, n-1\}$, key)

//Problem description: This algorithm is for searching the key element from an array $x[0, \dots, n-1]$ and search key

//input: An array $x[0, \dots, n-1]$ and search key

//output: Returns the index of x where key value is present.

For $i=0$ to $n-1$ do

If ($x[i] = \text{key}$) then

Return i ;

Best case time complexity:

It is a time complexity when an algorithm runs for short time. In the above searching at first location in the list $x[0, \dots, n-1]$ then the algorithm runs for a very short time and thereby we will get the best case time complexity. We can denote the best case time complexity as

$C_{\text{best}} = 1$

Worst case Time complexity:

IT is time complexity when algorithm runs for longest time. In the above searching algorithm the element key is searched from the list of n elements. if the key element is present at n th location then clearly the algorithm will run for longest time and thereby we will get the worst time complexity. We can denote as

Design and Analysis of Algorithm

$$C_{\text{worst}} = n$$

Average case time complexity:

This type of complexity gives information about the behavior of an algorithm on a specific random input. Let us understand some terminologies that are required for computing average case time complexity.

Let the algorithm is for sequential search and P be a probability of getting successful search.

N is the total number of elements in the list.

The first match of the element will occur at i_{th} location .hence probability of occurring first match is P/n for every i_{th} element.

The probability of getting unsuccessful search is $(1-P)$

Now, we can find average case time complexity $C_{\text{avg}}(n)$ as

$C_{\text{avg}}(n)$ =Probability of successful search (for elements 1 to n in the list)+ probability of unsuccessful search

$$\begin{aligned} C_{\text{avg}}(n) &= [1.P/n + 2.P/n + \dots + i.P/n] + n.(1-P) \\ &= P/n[1+2+\dots+i\dots n] + n(1-p) \\ &= P/n.n(n+1)/n + n(1-P) \end{aligned}$$

$$C_{\text{avg}}(n) = P(n+1)/2 + n(1-P)$$

Thus we can obtain the general formula for computing average case time complexity.

Suppose if $p=0$ that means there is no successful search. i.e we have scanned the entire list of n elements and still we don't found the desired element in the list then in such a situation,

$$C_{\text{avg}}(n) = 0(n+1)/2 + n(1-0)$$

$$C_{\text{avg}}(n) = n$$

Thus the average case running time complexity becomes equal to n .

Suppose if $p=1$ i.e we get a successful search then

$$C_{\text{avg}}(n) = 1(n+1)/2 + n(1-1)$$

$$C_{\text{avg}}(n) = (n+1)/2.$$

That means the algorithm scans about half of the elements from the list.

For calculating average case time complexity we have to consider probability of getting success of the operation.

And any operation in the algorithm is heavily dependent on input elements. Thus, computing average case time complexity is difficult than computing worst case and best case complexities.

Time complexity for linear search

Best case	Worst case	Average case
$O(1)$	$O(n)$	$O(n)$

1.4 Heaps and Heap sort

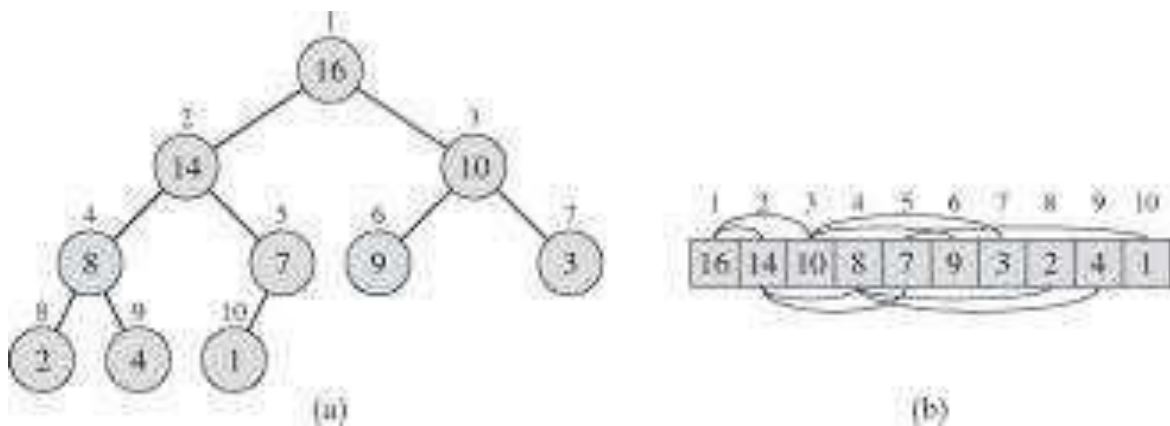
HEAPSORT

Design and Analysis of Algorithm

- Inplace algorithm
- Running Time: $O(n \log n)$
- Complete Binary Tree

The **(binary) heap** data structure is an array object that we can view as a nearly complete binary tree. Each node of the tree corresponds to an element of the array. The tree is completely filled on all levels except possibly the lowest, which is filled from the left up to a point.

The root of the tree is $A[1]$, and given the index i of a node, we can easily compute the indices of its parent, left child, and right child:



- $\text{PARENT}(i) \Rightarrow \text{return } [i/2]$
- $\text{LEFT}(i) \Rightarrow \text{return } 2i$
- $\text{RIGHT}(i) \Rightarrow \text{return } 2i + 1$

On most computers, the LEFT procedure can compute $2i$ in one instruction by simply shifting the binary representation of i left by one bit position.

Similarly, the RIGHT procedure can quickly compute $2i + 1$ by shifting the binary representation of i left by one bit position and then adding in a 1 as the low-order bit.

Design and Analysis of Algorithm

The PARENT procedure can compute $\lfloor i/2 \rfloor$ by shifting i right one bit position. Good implementations of heapsort often implement these procedures as "macros" or "inline" procedures.

There are two kinds of binary heaps: **max-heaps** and **min-heaps**.

- In a **max-heap**, the **max-heap property** is that for every node i other than the root, $A[\text{PARENT}(i)] \geq A[i]$, that is, the value of a node is at most the value of its parent. Thus, the largest element in a max-heap is stored at the root, and the subtree rooted at a node contains values no larger than that contained at the node itself.
- A **min-heap** is organized in the opposite way; the **min-heap property** is that for every node i other than the root, $A[\text{PARENT}(i)] \leq A[i]$,

The smallest element in a min-heap is at the root.

- ✓ The height of a node in a heap is the number of edges on the longest simple downward path from the node to a leaf and
- ✓ The height of the heap is the height of its root.
- ✓ Height of a heap of n elements which is based on a complete binary tree is $O(\log n)$.

Maintaining the heap property

MAX-HEAPIFY lets the value at $A[i]$ "float down" in the max-heap so that the subtree rooted at index i obeys the max-heap property.

MAX-HEAPIFY(A, i)

l ← LEFT(i)

r ← RIGHT(i)

if $A[l] > A[i]$
largest ← l
if $A[r] > A[\text{largest}]$
largest ← r
if largest ≠ i

Then exchange $A[i] \leftrightarrow A[\text{largest}]$

Design and Analysis of Algorithm

MAX-HEAPIFY (A,largest)

At each step, the largest of the elements $A[i]$, $A[\text{LEFT}(i)]$, and $A[\text{RIGHT}(i)]$ is determined, and its index is stored in *largest*. If $A[i]$ is largest, then the subtree rooted at node i is already a max-heap and the procedure terminates. Otherwise, one of the two children has the largest element, and $A[i]$ is swapped with $A[\text{largest}]$, which causes node i and its children to satisfy the max-heap property. The node indexed by *largest*, however, now has the original value $A[i]$, and thus the subtree rooted at *largest* might violate the max-heap property. Consequently, we call MAX-HEAPIFY recursively on that subtree.

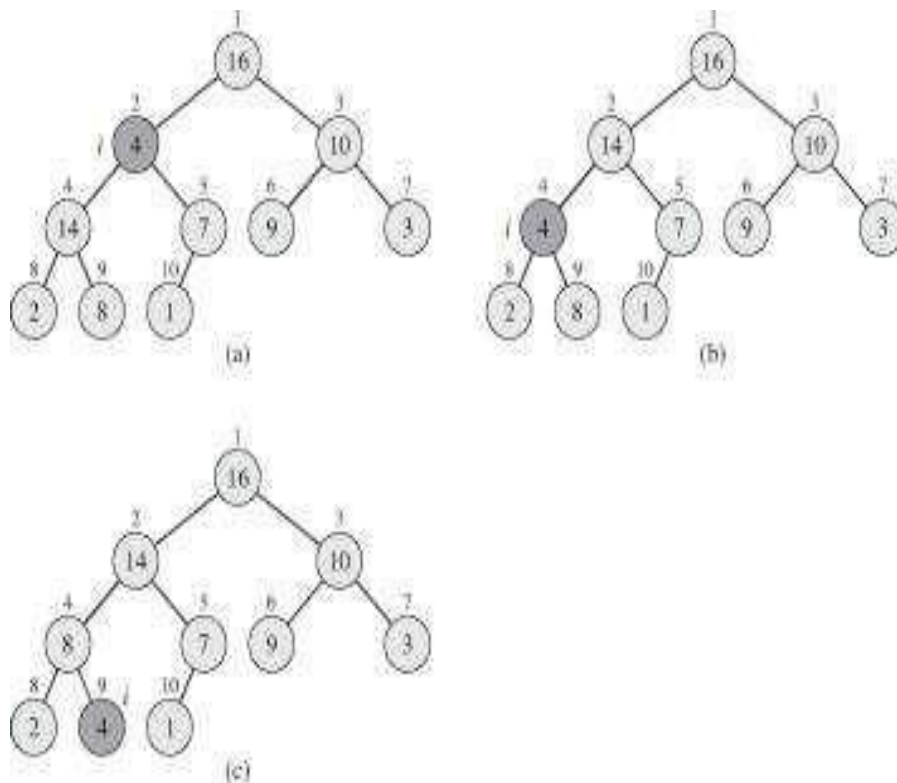


Figure: The action of MAX-HEAPIFY (A, 2), where *heap-size* = 10. **(a)** The initial configuration, with $A[2]$ at node $i = 2$ violating the max-heap property since it is not larger than both children. The max-heap property is restored for node 2 in **(b)** by exchanging $A[2]$ with $A[4]$, which destroys the max-heap property for node 4. The recursive call MAX-HEAPIFY (A,4)

Design and Analysis of Algorithm

now has $i = 4$. After swapping $A[4]$ with $A[9]$, as shown in **(c)**, node 4 is fixed up, and the recursive call $\text{MAX-HEAPIFY}(A, 9)$ yields no further change to the data structure.

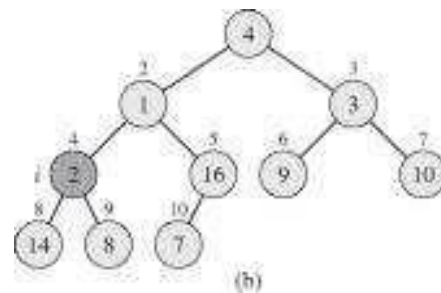
The running time of MAX-HEAPIFY by the recurrence can be described as $T(n) \leq T(2n/3) + O(1)$

The solution to this recurrence is $T(n) = O(\log n)$

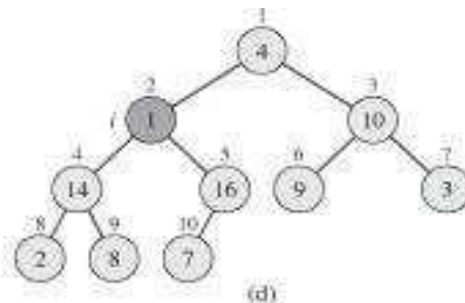
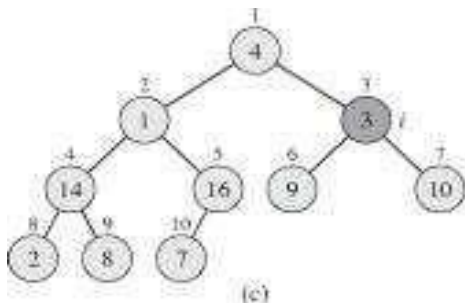
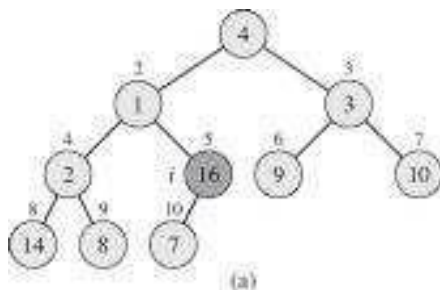
Building a heap

$\text{Build-Max-Heap}(A)$

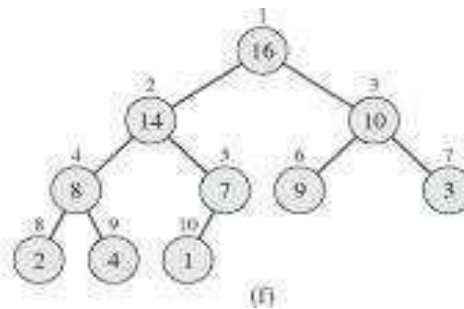
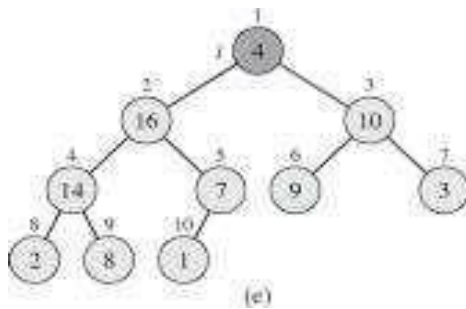
1. for $i \leftarrow \lfloor n/2 \rfloor$ to 1
2. do $\text{MAX-HEAPIFY}(A, i)$



4	1	3	2	1	9	1	1	8	7
---	---	---	---	---	---	---	---	---	---



Design and Analysis of Algorithm



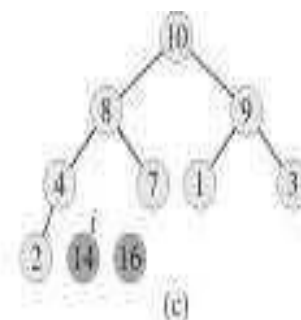
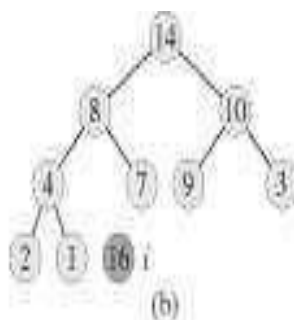
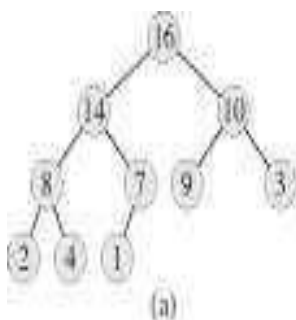
We can derive a tighter bound by observing that the time for MAX-HEAPIFY to run at a node varies with the height of the node in the tree, and the heights of most nodes are small. Our tighter analysis relies on the properties that an n -element heap has height $\lceil \log n \rceil$ and at most $\lceil n/2^{h+1} \rceil$ nodes of any height h .

The total cost of BUILD-MAX-HEAP as being bounded is $T(n)=O(n)$

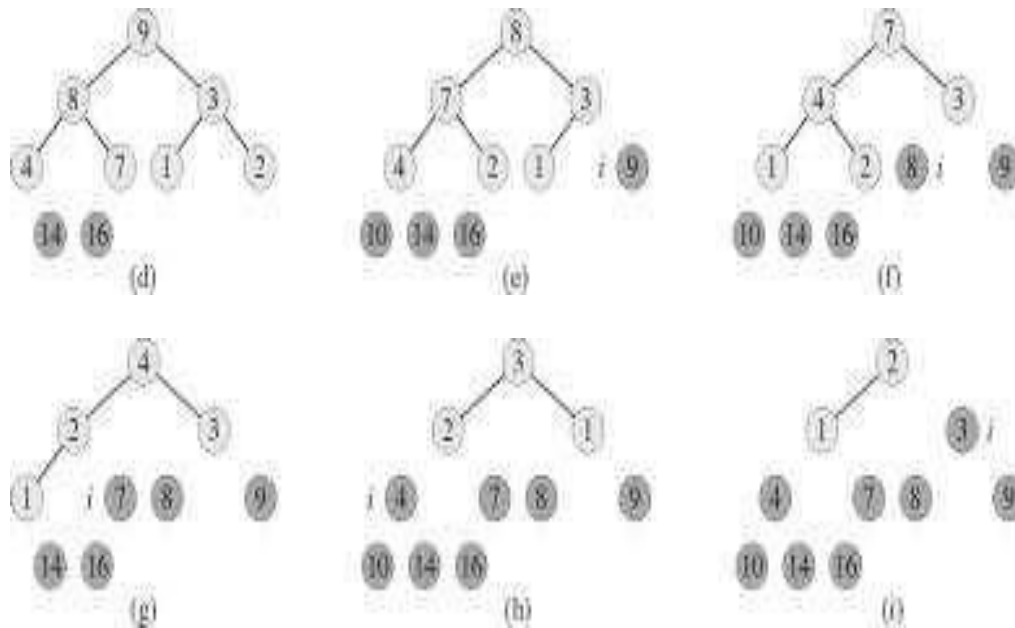
The HEAPSORT Algorithm

HEAPSORT(A)

1. BUILD MAX-HEAP(A)
2. for $i=n$ to 2
3. exchange $A[1]$ with $A[i]$
4. MAX-HEAPIFY(A,1)



Design and Analysis of Algorithm



1	2	3	4	7	8	9	10	14	16
---	---	---	---	---	---	---	----	----	----

The HEAPSORT procedure takes time $O(n \log n)$, since the call to BUILD-MAX-HEAP takes time $O(n)$ and each of the $n - 1$ calls to MAX-HEAPIFY takes time $O(\log n)$.

1.5 Lower Bounds for Sorting

Review of Sorting: So far we have seen a number of algorithms for sorting a list of numbers in ascending order. Recall that an *in-place* sorting algorithm is one that uses no additional array storage (however, we allow Quicksort to be called in-place even though they need a stack of size $O(\log n)$ for keeping track of the recursion). A sorting algorithm is *stable* if duplicate elements remain in the same relative position after sorting.

Slow Algorithms: Include Bubble Sort, Insertion Sort, and Selection Sort. These are all simple

$\Theta(n^2)$ in-place sorting algorithms. Bubble Sort and Insertion Sort can be implemented as stable algorithms, but Selection Sort cannot (without significant modifications).

Design and Analysis of Algorithm

Merge sort: Merge sort is a stable $\Theta(n \log n)$ sorting algorithm. The downside is that Merge Sort is the only algorithm of the three that requires additional array storage, implying that it is not an **in-place** algorithm.

Quicksort: Widely regarded as the *fastest* of the fast algorithms. This algorithm is $O(n \log n)$ in the *expected case*, and $O(n^2)$ in the worst case. The probability that the algorithm takes asymptotically longer (assuming that the pivot is chosen randomly) is extremely small for large n . It is an (almost) **in-place sorting** algorithm but is not stable.

Heap sort: Heap sort is based on a nice data structure, called a *heap*, which is a fast priority queue. Elements can be inserted into a heap in $O(\log n)$ time, and the largest item can be extracted in $O(\log n)$ time. (It is also easy to set up a heap for extracting the smallest item.) If you only want to extract the k largest values, a heap can allow you to do this in $O(n + k \log n)$ time. It is an **in-place algorithm**, but it is not stable.

Theorem: Any comparison-based sorting algorithm has worst-case running time $(n \log n)$.

This can be generalized to show that the *average-case* time to sort is also $(n \log n)$ (by arguing about the average height of a leaf in a tree with at least $n!$ leaves). The lower bound on sorting can be generalized to provide lower bounds to a number of other problems as we

Design and Analysis of Algorithm

1.6 Lower Bounds for Comparison-Based Sorting:

Can we sort faster than $O(n \log n)$ time?

We will give an argument that if the sorting algorithm is based solely on making comparisons between the keys in the array, then it is impossible to sort more efficiently than $(n \log n)$ time. Such an algorithm is called a **comparison-based** sorting algorithm, and includes all of the algorithms given above. Virtually all known general purpose sorting algorithms are based on making comparisons, so this is not a very restrictive assumption. This does not preclude the possibility of a sorting algorithm whose actions are determined by other types of operations, for example, consulting the individual bits of numbers, performing arithmetic operations, indexing into an array based on arithmetic operations on keys.

Design and Analysis of Algorithm

Decision Tree Argument: In order to prove lower bounds, we need an abstract way of modeling “any possible” comparison-based sorting algorithm, we model such algorithms in terms of an abstract model called a *decision tree*. In a *comparison-based* sorting algorithm only comparisons between the keys are used to determine the action of the algorithm. Let $a_1; a_2; \dots; a_n$ be the input sequence. Given two elements, a_i and a_j , their relative order can only be determined by the results of comparisons like $a_i < a_j$, $a_i \leq a_j$, $a_i = a_j$, $a_i \geq a_j$, and $a_i > a_j$. A decision tree is a mathematical representation of a sorting algorithm (for a fixed value of n). Each node of the decision tree represents a comparison made in the algorithm (e.g., $a_4 \leq a_7$), and the two branches represent the possible results, for example, the left sub-tree consists of the remaining comparisons made under the assumption that $a_4 \leq a_7$ and the right sub-tree for $a_4 > a_7$. (Alternatively, one might be labeled with $a_4 < a_7$ and the other with $a_4 \geq a_7$.) Observe that once we know the value of n , then the “action” of the sorting algorithm is completely determined by the results of its comparisons. This action may involve moving elements around in the array, copying them to other locations in memory, performing various arithmetic operations on non-key data. But the bottom-line is that at the end of the algorithm, the keys will be permuted in the final array in some way. Each leaf in the decision tree is labeled with the final permutation that the algorithm generates after making all of its comparisons. To make this more concrete, let us assume that $n = 3$, and let’s build a decision tree for Selection Sort. Recall that the algorithm consists of two phases. The first finds the smallest element of the entire list, and swaps it with the first element. The second finds the smaller of the remaining two items, and swaps it with the second element. Here is the decision tree (in outline form). The first comparison is between a_1 and a_2 . The possible results are:

$a_1 \leq a_2$: Then a_1 is the current minimum. Next we compare a_1 with a_3 whose results might be either:

$a_1 \leq a_3$: Then we know that a_1 is the minimum overall, and the elements remain in their original positions. Then we pass to phase 2 and compare a_2 with a_3 . The possible results are:

$a_2 \leq a_3$: Final output is $a_1; a_2; a_3$.

$a_2 > a_3$: These two are swapped and the final output is $a_1; a_3; a_2$.

Design and Analysis of Algorithm

1.7 Hashing

- ✓ Hashing is the process of mapping large amount of data item to smaller table with the help of hashing function.
- ✓ Hashing is also known as **Hashing Algorithm** or **Message Digest Function**.
- ✓ It is a technique to convert a range of key values into a range of indexes of an array.
- ✓ It is used to facilitate the next level searching method when compared with the linear or binary search.
- ✓ Hashing allows to update and retrieve any data entry in a constant time $O(1)$.
- ✓ Constant time $O(1)$ means the operation does not depend on the size of the data.
- ✓ Hashing is used with a database to enable items to be retrieved more quickly.
- ✓ It is used in the encryption and decryption of digital signatures.

What is Hash Function?

- ✓ A fixed process converts a key to a hash key is known as a **Hash Function**.
- ✓ This function takes a key and maps it to a value of a certain length which is called a **Hash value** or **Hash**.
- ✓ Hash value represents the original string of characters, but it is normally smaller than the original.
- ✓ It transfers the digital signature and then both hash value and signature are sent to the receiver. Receiver uses the same hash function to generate the hash value and then compares it to that received with the message.
- ✓ If the hash values are same, the message is transmitted without errors.

What is Hash Table?

- ✓ Hash table or hash map is a data structure used to store key-value pairs.
- ✓ It is a collection of items stored to make it easy to find them later.
- ✓ It uses a hash function to compute an index into an array of buckets or slots from which the desired value can be found.
- ✓ It is an array of list where each list is known as bucket.
- ✓ It contains value based on the key.
- ✓ Hash table is used to implement the map interface and extends Dictionary class.
- ✓ Hash table is synchronized and contains only unique elements.



Fig. Hash Table

- ✓ The above figure shows the hash table with the size of $n = 10$. Each position of the hash table is called as **Slot**. In the above hash table, there are n slots in the table, names = $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$. Slot 0, slot 1, slot 2 and so on. Hash table contains no items, so every slot is empty.
- ✓ As we know the mapping between an item and the slot where item belongs in the hash table is called the hash function. The hash function takes any item in the collection and returns an integer in the range of slot names between 0 to $n-1$.

Design and Analysis of Algorithm

- ✓ Suppose we have integer items {26, 70, 18, 31, 54, 93}. One common method of determining a hash key is the division method of hashing and the formula is :

$$\text{Hash Key} = \text{Key Value} \% \text{Number of Slots in the Table}$$

- ✓ Division method or remainder method takes an item and divides it by the table size and returns the remainder as its hash value.

Data Item	Value % No. of Slots	Hash Value
26	$26 \% 10 = 6$	6
70	$70 \% 10 = 0$	0
18	$18 \% 10 = 8$	8
31	$31 \% 10 = 1$	1
54	$54 \% 10 = 4$	4
93	$93 \% 10 = 3$	3

0	1	2	3	4	5	6	7	8	9
70	31		93	54		26		18	

Fig. Hash Table

- ✓ After computing the hash values, we can insert each item into the hash table at the designated position as shown in the above figure. In the hash table, 6 of the 10 slots are occupied, it is referred to as the load factor and denoted by, $\lambda = \text{No. of items} / \text{table size}$. For example, $\lambda = 6/10$.
- ✓ It is easy to search for an item using hash function where it computes the slot name for the item and then checks the hash table to see if it is present.
- ✓ Constant amount of time $O(1)$ is required to compute the hash value and index of the hash table at that location.

8.2.2 Linear Probing

- ✓ Take the above example, if we insert next item 40 in our collection, it would have a hash value of 0 ($40 \% 10 = 0$). But 70 also had a hash value of 0, it becomes a problem. This problem is called as **Collision** or **Clash**. Collision creates a problem for hashing technique.
- ✓ **Linear probing is used for resolving the collisions in hash table**, data structures for maintaining a collection of key-value pairs.
- ✓ Linear probing was invented by Gene Amdahl, Elaine M. McGraw and Arthur Samuel in 1954 and analyzed by Donald Knuth in 1963.

Design and Analysis of Algorithm

- ✓ It is a component of open addressing scheme for using a hash table to solve the dictionary problem.
- ✓ The simplest method is called Linear Probing. Formula to compute linear probing is:

$$P = (1 + P) \% (\text{MOD}) \text{ Table_size}$$

For example,

0	1	2	3	4	5	6	7	8	9
70	31		93	54		26		18	

Fig. Hash Table

If we insert next item 40 in our collection, it would have a hash value of 0 ($40 \% 10 = 0$). But 70 also had a hash value of 0, it becomes a problem.

Linear probing solves this problem:

$$P = H(40)$$

$$44 \% 10 = 0$$

Position 0 is occupied by 70. so, we look elsewhere for a position to store 40.

Using Linear Probing:

$$P = (P + 1) \% \text{table-size}$$

$$0 + 1 \% 10 = 1$$

But, position 1 is occupied by 31, so we look elsewhere for a position to store 40.

Using linear probing, we try next position: $1 + 1 \% 10 = 2$

Position 2 is empty, so 40 is inserted there.

0	1	2	3	4	5	6	7	8	9
70	31	40	93	54		26		18	

Fig. Hash Table

CHAPTER 2: DIVIDE AND CONQUER

- ✓ It's a very interesting algorithmic strategy.

2.1 General Method:

The method has following activities, For a given problem is

1. **Divide:** Divided into smaller sub problems,
2. **Conquer:** These sub problems are solved independently,
3. **Combine:** Combining all the solutions of sub problems into a solution of the whole
 - If the sub problems are large enough then divide and conquer is reapplied.
 - The general sub problems are usually of the same type as original problem. So, Recursive algorithms are used in divide and conquer strategy.

Design and Analysis of Algorithm

- A control abstraction for divide and conquer is as given below- using control abstraction a flow of control of a procedure is given.

The base case for the recursion is sub-problem of constant size.

Advantages of Divide & Conquer technique:

- For solving conceptually difficult problems like Tower Of Hanoi, divide & conquer is a powerful tool
- Results in efficient algorithms
- Divide & Conquer algorithms are adapted for execution in multi-processor machines
- Results in algorithms that use memory cache efficiently.

Limitations of divide & conquer technique:

- Recursion is slow
- Very simple problem may be more complicated than an iterative approach.

Example: adding n numbers etc

Algorithm DC(P)

```
{  
If  
P is too small then  
Return solution of P  
Else  
{  
Divide (P) and obtain  $P_1, P_2, \dots, P_n$   
where  $n \geq 1$   
Apply DC to each sub problem  
Return combines  $(DC(P_1), DC(P_2), \dots, DC(P_n))$ ;  
}  
}
```

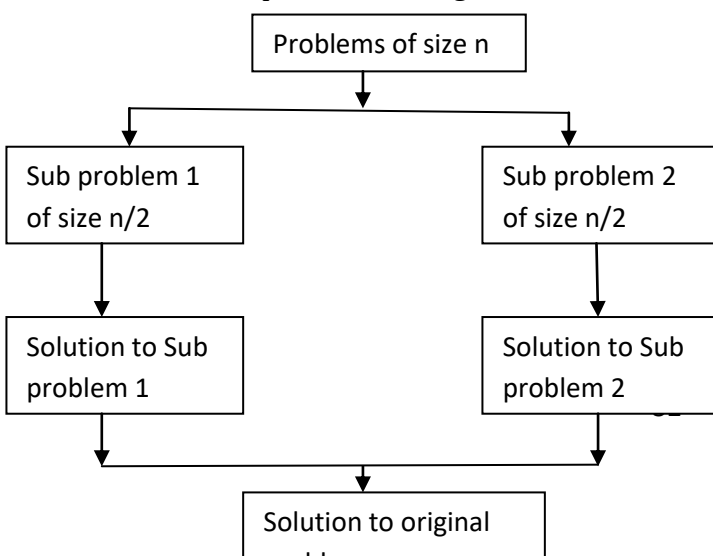
The computing time of the above procedure of divide and conquer is given by recurrence relation

$T(n) = \{g(n) \text{ if } n \text{ is small}$

$T(n) = T(n_1) + T(n_2) + \dots + T(n_r) + F(n) \text{ when } n \text{ is sufficient.}$

Where $T(n)$ is the time for divide and conquer of size n . the $g(n)$ is the computation time required to solve small inputs. The $F(n)$ is the time required for dividing problem P and combining the solution to sub problems.

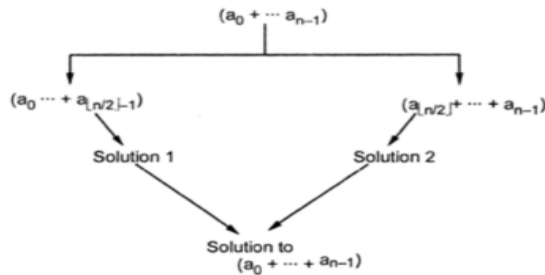
The divide and conquer method is given in the below figure.



Design and Analysis of Algorithm

The generated sub problems are usually of same type as the original problem. Hence sometimes recursive algorithms are used in divide and conquer method.

For example, if we want to compute sum of n numbers then by divide and conquer we can solve the problem as



If we want to divide a problem of size n into a size of n/b taking $f(n)$ time to divide and combine, An instance of size n can be divided into b instances of size n/b , with “ a ” of them needing to be solved. [$a \geq 1, b > 1$]. Assume size n is a power of b .

The general divide and conquer recurrence relation is

$$T(n) = aT(n/b) + f(n)$$

$T(n)$ - time for size of input n ,

a = no.of instances,

$T(n/b)$ = time for size n/b ,

$f(n)$ = time required for dividing the problem into sub problem. The order of growth of $T(n)$ depends upon the constants a, b and order of growth function $f(n)$.

Applications of divide and conquer method:

1. Binary search
2. Merge sort
3. Finding Maximum and Minimum

2.2 Binary search:

Binary Search is an efficient searching method. The most essential thing is that the elements in the array should be sorted one.

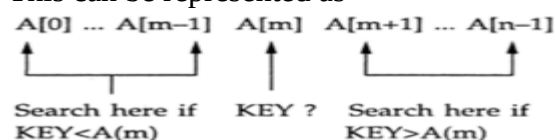
An element which is to be searched from the list of elements sorted in an array $A[0, \dots, n-1]$ is called Key element.

Let $A[m]$ be the mid element of the array A .

Then there are three conditions

1. If $\text{key} = A[m]$ then desired element is present in the list.
2. Otherwise if $\text{key} < A[m]$ then search left sublist
3. Otherwise if $\text{key} > A[m]$ then search the right sublist.

This can be represented as



Algorithm:

Alg binsearch($A[0, \dots, n-1], \text{key}$)

Design and Analysis of Algorithm

//Problem description: this algorithm is for searching the

// element using binary search method.

//Input:an array A from which the KEY element to be searched

// Output:it returns the index of the array element if it is equal to KEY otherwise it returns -1.

low $\leftarrow$ 0

high $\leftarrow$ n-1

while(low<high) do

{

m=(low+high)/2

if(key=A[m]) then

return m

else if(key < A[m]) then

high=m-1 // search left sublist

else

low=m+1 //search right sublist

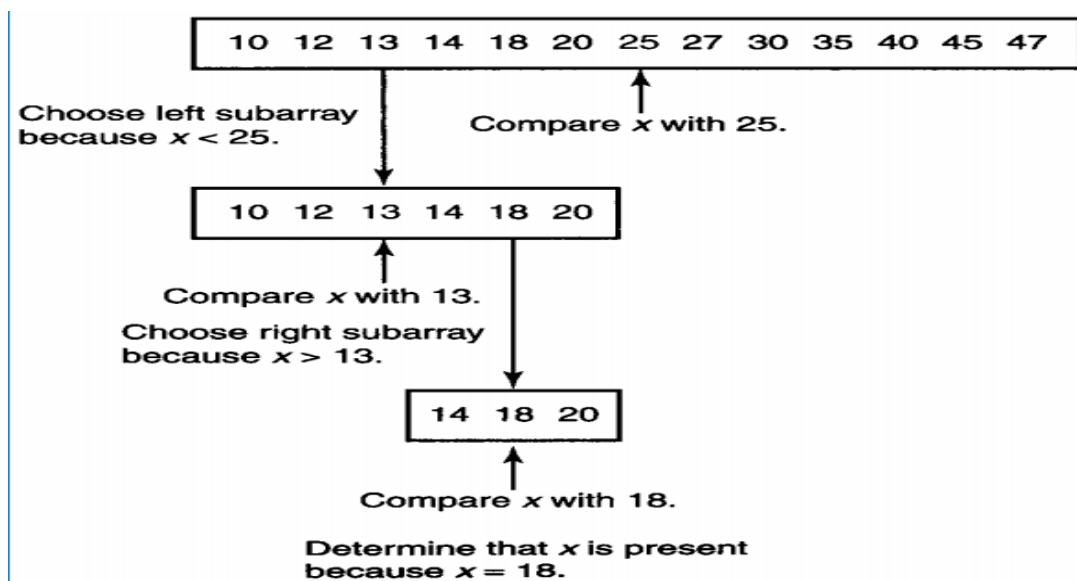
}return -1 // if element is not present in the list.

Example:

Suppose the search element x=18 and the input array is 10,12,13,14,18,20,25,27,30,35,40,45,47

Here middle element A[m]=20

The middle element= $0+12/2 = 6$ th position of array is the middle element.



Analysis:

The basic operation in binary search is comparison of search key with the other array elements.

Depend on

Best – key matched with mid element

Worst – key not found or key sometimes in the list

To analyze efficiency of binary search, we must count the number of times the search key gets compared with the array elements. The comparison is also called a three way comparison because algorithm makes the comparison to determine whether KEY is smaller, equal to or greater than A{m}. In this algorithm, after one comparison the list of n elements is divided into n/2 sub list.

Design and Analysis of Algorithm

The worst case efficiency is that the algorithm compares all the array elements for searching the desired element. In this method, one comparison is made and based on this comparison array is divided each time to $n/2$ sublist. Since after each comparison the algorithm divides the problem into half the size, hence the worst case complexity as

$$C_{\text{worst}}(n) = C_{\text{worst}}(n/2) + 1 \text{ for } n > 1$$

$$\text{Also } C_{\text{worst}}(1) = 1.$$

Here $C_{\text{worst}}(n/2)$ ----- time required to compare left or right sublist.

1----- one comparison made with middle element.

But as we consider the rounded down value when array gets divided the above equation can be written as

$$C_{\text{worst}}(n) = C_{\text{worst}}(n/2) + 1 \quad \text{for } n > 1 \text{ ----- (1)}$$

$$C_{\text{worst}}(1) = 1 \quad \text{----- (2)}$$

The above recurrence equation can be solved further. Assume $n = 2^K$ the equation 1 becomes

$$C_{\text{worst}}(2^K) = C_{\text{worst}}(2^K/2) + 1 \quad \text{----- (3)}$$

$$C_{\text{worst}}(2^K) = C_{\text{worst}}(2^{K-1}) + 1$$

Using backward substitution method, we can substitute

$$C_{\text{worst}}(2^{K-1}) = C_{\text{worst}}(2^{K-2}) + 1$$

Then equation(3) becomes

$$\begin{aligned} C_{\text{worst}}(2^K) &= [C_{\text{worst}}(2^{K-2}) + 1] + 1 \\ &= C_{\text{worst}}(2^{K-2}) + 2 \end{aligned}$$

$$\begin{aligned} \text{Then } C_{\text{worst}}(2^K) &= [C_{\text{worst}}(2^{K-3}) + 1] + 2 \\ &= [C_{\text{worst}}(2^{K-3}) + 3 \end{aligned}$$

.....

.....

.....

.....

$$\begin{aligned} C_{\text{worst}}(2^K) &= [C_{\text{worst}}(2^{K-k}) + K] \\ &= C_{\text{worst}}(2^0) + k \end{aligned}$$

$$C_{\text{worst}}(2^K) = [C_{\text{worst}}(1) + K]$$

But as per equation(2),

$C_{\text{worst}}(1) = 1$ then we get equation(4),

$$C_{\text{worst}}(2^K) = [1 + K]$$

As we have assumed,

$$n = 2^K$$

taking logarithm(base 2) on both sides

$$\log_2 n = \log_2 2^K$$

$$\log_2 n = K \cdot \log_2 2$$

$$\log_2 n = K(1)$$

therefore $\log_2 2 = 1$

$$\therefore C_{\text{worst}}(n) = 1 + \log_2 n$$

$$\therefore C_{\text{worst}}(n) = \log_2 n \text{ for } n > 1$$

The worst case of complexity of binary search is $\Theta(\log n)$

As $C_{\text{worst}}(n) = 1 + \log_2 n$ we can verify equation(1) with this value

Considering equation (2),

$$C_{\text{worst}}(n) = C_{\text{worst}}(n/2) + 1$$

Design and Analysis of Algorithm

For L.H.S

Assume $n = 2^i$

$\therefore$ substitute

$$\begin{aligned} C_{\text{worst}}(n) &= \log_2 n + 1 \\ &= \log_2 (2^i) + 1 \\ &= \log_2 2 + \log_2 i + 1 \\ &= \log_2 i + 2 \end{aligned}$$

For R.H.S

Assume $n = 2^i$

$\therefore$ substitute

$$\begin{aligned} C_{\text{worst}}(n/2) + 1 &= C_{\text{worst}}(2^i/2) + 1 \\ &= C_{\text{worst}}(2^{i-1}) + 1 \end{aligned}$$

As $C_{\text{worst}}(n) = \log_2 n + 1$

In the same manner

$$\begin{aligned} C_{\text{worst}}(i) + 1 &= (\log_2 i + 1) + 1 \\ &= \log_2 i + 2 \end{aligned}$$

Average case complexity:

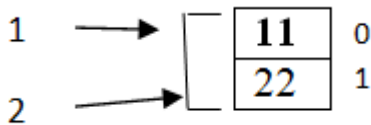
To obtain average case efficiency of binary search we will consider some samples of input n

If $n=1$

i.e there is only one element 11 is there

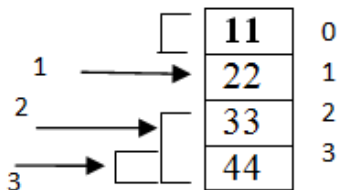
$1 \rightarrow [11]$ only one search is required to search some key

If $n=2$ and search key is 22



Two comparisons are made to search key 22

If $n=4$ and search key is 44



$$m = (0+3)/2 = 1$$

As $A[1] = 22$ and $22 < 44$

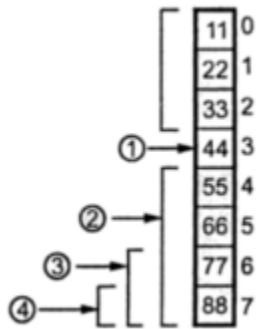
Search right sublist again,

$$M = (2+3)/2 = 2$$

Again $A[2] = 33$ and $33 < 44$ we divide list in right sublist $A[4] = 44$ and key is 44. Thus, total 3 comparisons are made to search 44.

If $n=8$ and search key = 88

Design and Analysis of Algorithm



$$m = (0+7)/2 = 3$$

As $A[3] = 44$ and $44 < 88$ search right sublist, again

$$M = (4+7)/2 = 5$$

Again $A[5] = 66$ and $66 < 88$ search right sublist again

$$M = (6+7)/2 = 6$$

But $A[6] = 77$ and $77 < 88$

Then search right sublist

$$M = (7+7)/2 = 7$$

$A[m] = A[7] = 88$. Yes, the desired element is present .

Thus, total 4 comparisons are made to search 88

To summarize the above operations

N	Total comparison
1	1
2	2
4	3
8	4
16	5
...	

Observing the above given table we can write as

$$\log_2 n + 1 = c$$

For instance, if $n=2$ then

$$\log_2 2 = 1$$

$$\text{then } c = 1 + 1 = 2$$

similarly $n=8$ then

$$c = \log_2 n + 1 = \log_2 8 + 1 = 3 + 1 = 4$$

thus, we can write as

average case complexity is $\Theta(\log n)$

The time complexity of binary search is

- **Worst case:** $\Theta(\log n)$
- **Average case:** $\Theta(\log n)$
- **Best case:** $\Theta(1)$

Applications of binary search:

- Number guessing game

Design and Analysis of Algorithm

- Word lists/search dictionary etc

Advantages:

- Efficient on very big list
- Can be implemented iteratively/recursively

Limitations:

- Interacts poorly with the memory hierarchy
- Requires given list to be sorted
- Due to random access of list element, needs arrays instead of linked list.

2.3 MERGE SORT

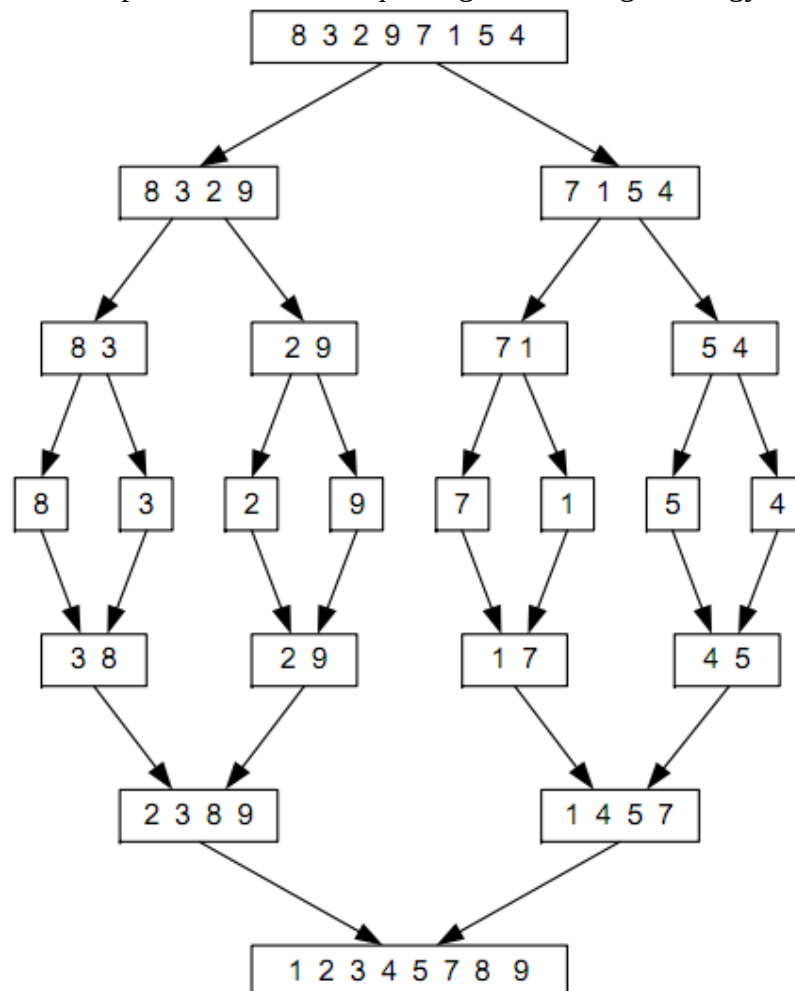
Merge sort is a sort algorithm that splits the items to be sorted into two groups, recursively sorts each group, and merges them into a final sorted sequence.

It has 3 steps

- ✓ **Divide:** partition the array into two sublist s_1 and s_2 with $n/2$ element each
- ✓ **Conquer:** then sort sublist s_1 & s_2
- ✓ **Combine :** Merge s_1 and s_2 into a unique group.

Features:

- ◆ It is a comparison based algorithm
- ◆ It is a stable algorithm
- ◆ It is a perfect example of divide & conquer algorithm design strategy



Algorithm Mergesort $A[0, \dots, n-1], \text{low}, \text{high}$)

Design and Analysis of Algorithm

//Problem description: this algorithm is for sorting the elements using merge sort.

//Input: Array A of unsorted elements, low as beginning pointer of array A and high as end //pointer of array A

//Output: Sorted array A[0,...n-1]

If (low < high) then

{

mid ← (low+high)/2 //split the list at mid

Mergesort(A,low,mid) //first sublist

Mergesort(A,mid+1,high) //second sublist

Combine(A,low,mid,high) //Merging of two sublist

Algorithm Combine(low,mid,high)

{

K=low; // k as index for array temporary

i=low; // I as index for left sublist

j=mid+1; // j as index for right sublist.

While(i ≤ mid and j ≤ high) do

{

If(A[i] ≤ A[j]) then // if small element present in left sublist

{ // copy that smaller elements to temp array

Temp[k]=A[i];

i=i+1;

k=k+1;

}

Else //smaller elements is present in right sublist

{

// copy that smaller elements to temp array

Temp[k]=A[j]; // if small element present in right sublist

j=j+1;

k=k+1;

}

// copy remaining elements of left sublist to temp

While(i ≤ mid) do

{

Temp[k]=A[i];

i=i+1;

k=k+1;

}

Copy remaining element right to temp

While(j ≤ high) do

{

Temp[K]=A[j];

j=j+1;

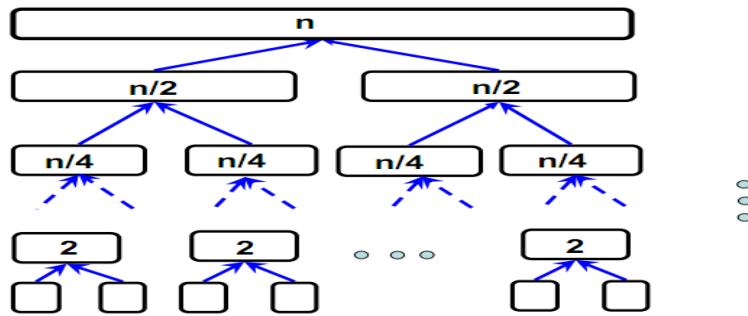
k=k+1;

}

Analysis:

Design and Analysis of Algorithm

In merge sort, two recursive calls are made. Each recursive call focuses on $n/2$ elements of the list. After two recursive calls, one call is made to combine two sub lists.



The recurrence relation is

$$T(n) = T(n/2) + T(n/2) + n$$

Where $T(n/2)$ = time taken for the left sub list to be sorted.

$T(n/2)$ = time taken for the right sub list to be sorted.

n = time taken for combining right and left sub list. Where $n > 1$ i.e. $T(1) = 0$

Using Master's theorem,

As per theorem $T(n) = \Theta(n^d \log n)$ if $a = b$

Here $T(n) = 2T(n/2) + n$, Here $a = 2, b = 2$ with $d = 1$

So, $T(n) = \Theta(n \log n)$

Here most of the work done in combining the solutions,

- **Best case: $\Theta(n \log n)$**
- **Worst case: $\Theta(n \log n)$**
- **Average case: $\Theta(n \log n)$**

2.4 Finding Maximum and minimum

Algorithm for finding minimum element from the set of n numbers.

Algorithm Minimum_val($A[1, \dots, n]$)

{

// Problem description: this algorithm is to find minimum value from array A to n elements.

min ← $A[1]$ // Assuming first element as min.

for ($i \leftarrow 2$ to n) do

{

If ($\text{min} > A[i]$) then

min ← $A[i]$ // set new min value.

} return min;

}

Design and Analysis of Algorithm

Thus we require total $(n-1)$ comparison to obtain minimum value . similarly we can obtain the maximum value from an array A in $(n-1)$ comparison.

The algorithm for finding maximum element.

Algorithm **Maximum_val**(A[1,...n])

```
{
//Problem description: This algorithm is to find maximum value from array A of n elements.
max ← A[1]
for(i ← 2 to n) do
{
if(A[i] > max) then
max ← A[i]
}
Return max
}
```

We can obtain maximum and minimum values from an array simultaneously following algorithm.

Algorithm **Max_min**(A[1,...n], Max, Min)

```
{
Problem description: This algorithm obtains min and max value simultaneously
Max ← Min ← A[1]
For( i ← 2 to n) do
{
If(A[i] > Max) then
Max ← A[i] //Obtaining maximum value
If(A[i] < Min) then
Min ← A[i] //Obtaining minimum value
} }
}
```

Thus we can get maximum element in max and minimum element in min variable.\

Example: consider the array

1	2	3	4	5	6	7	8	9
50	40	-5	-9	45	90	65	25	75

Step 1:

1	2	3	4	5	6	7	8	9
50	40	-5	-9	45	90	65	25	75

↑
Min
Max

Step 2: now from index 2 to n we will compare an array element with Min and Max value.

1	2	3	4	5	6	7	8	9	Min = 40
50	40	-5	-9	45	90	65	25	75	Max = 50

Step 3:

Design and Analysis of Algorithm

1	2	3	4	5	6	7	8	9	Min = - 5
50	40	- 5	- 9	45	90	65	25	75	Max = 50

Step 4:

1	2	3	4	5	6	7	8	9	Min = - 9
50	40	- 5	- 9	45	90	65	25	75	Max = 50

Step 5:

1	2	3	4	5	6	7	8	9	Min = - 9
50	40	- 5	- 9	45	90	65	25	75	Max = 50

Step 6:

1	2	3	4	5	6	7	8	9	Min = - 9
50	40	- 5	- 9	45	90	65	25	75	Max = 90

Step 7:

1	2	3	4	5	6	7	8	9	Min = - 9
50	40	- 5	- 9	45	90	65	25	75	Max = 90

Step 8:

1	2	3	4	5	6	7	8	9	Min = - 9
50	40	- 5	- 9	45	90	65	25	75	Max = 90

Step 9:

1	2	3	4	5	6	7	8	9	Min = - 9
50	40	- 5	- 9	45	90	65	25	75	Max = 90

Step 10:

As we have reached at nth location of the array we will terminate the procedure of finding maximum and minimum values and print minimum values as -9 and maximum value as 90.

Analysis:

The above algorithm takes $\Theta(n)$ run n time. This is because the basic operation comparing array element with min or max value is done within a for loop.

The above algorithm is a straightforward algorithm of finding the maximum and minimum but we can obtain them using recursive method. The recursive algorithms makes use of divide and conquer strategy.

In this algorithm, the list is divided at the mid in order to obtain two sub lists. From both the sub list maximum and minimum elements are chosen. Two maxima and minima are compared and from then real maximum and minimum elements are determined. This process is carried out for entire list.

The algorithm is given below:

Algorithm Max_val(i,j,max,min)

//Problem description:Finding min, max elements recursively.

Design and Analysis of Algorithm

//input:I,j are integers used as index to an array A. the max and min will contain maximum and minimum value elements.

//output:None

If(i==j) then

{

max←A[i];

min←A[j];

}

Else if(i= j-1) then

{ if(A[i]< A[j])then

{

max←A[j];

min←A[i]

}else

{max←A[i]

min←A[j]

}

//end of els

} //end of if.

Else

{

mid←(i+j)/2 // divide the list handling two lists separately.

Max_Min_val(i ,mid ,max, min)

Max_Min_val(mid+1,j,max_new.min_new)

If(Max<max_new) then

max←max_new //combine solution

if(min<min_new) then

min←min_new // combine solution

}

Example: consider a list of some elements from which maximum and minimum element can be found out.

1	2	3	4	5	6	7	8	9
50	40	- 5	- 9	45	90	65	25	75

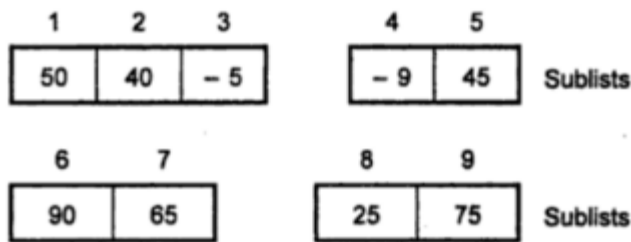
Step 1:

1	2	3	4	5	
50	40	- 5	- 9	45	Sublist 1
6	7	8	9		
90	65	25	75	Sublist 2	

We have divided the original list at mid and two sublists:sublist1 and sublist 2 are created. We will find min and max values respectively from each sublist.

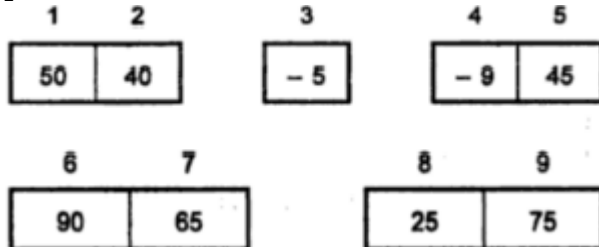
Step 2:

Design and Analysis of Algorithm



Again divide each sublist and create further sublists. then from each sublist obtain.

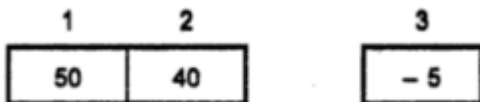
Step 3:



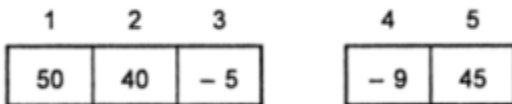
It is possible to divide the list(50,40,-5) further. Hence we have divided the list into sublists. And min , max values are obtained.

Step 4:

Now further division of the list is not possible. Hence we start combining the solutions of min and max values from each sublist.



Combine(1,2) and(3) Min= -5 , Max=50.



Combine(1,..3) and(4,5) Min =-9 , Max =50

Now we will combine (1,..3) and (4,5) and the min and max values among them are obtained.hence,

Min value= -9

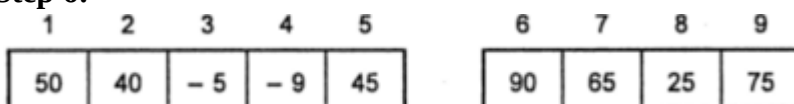
Max value = 50

Step 5:



Combine(6,7) and(8,9) Min=25 ,Max=90

Step 6:



Combine th sublists_1,...5) and (6,..9).find out min mad max values which are min = -9 and max=90.

Thus, the complete list is formed from which the min and max values are obtained. Hence final min and max values min=-9 and max=90

The two recursive calls made in this algorithm for each half divided sublist. Hence time required for computing min nad max will be\

Design and Analysis of Algorithm

$$T(n) = T(n/2) + T(n/2) + 2 \text{ when } n > 2$$

$$T(n) = 1 \text{ when } n = 2$$

If single element is present in the list then $T(n) = 0$

Now, time required for computing min and max will be

$$\begin{aligned} T(n) &= 2T(n/2) + 2 \\ &= 2\{2T(n/4) + 2\} + 2 \\ &= 2(2[2T(n/8) + 2] + 2) + 2 \\ &= 8T(n/8) + 10 \end{aligned}$$

Continuing in this fashion a recursive equation can be obtained. If we put $n = 2^k$ then

$$T(n) = 2^{k-1} T(2) + \sum_{i=1}^{k-1} 2^i$$

$$= 2^{k-1} + 2^k - 2$$

$$= 3n/2 - 2$$

So the time complexity **$O(n)$** .

CHAPTER 3: GREEDY METHOD

3.1 Introduction

Let we are given a problem to sort the array $a = \{5, 3, 2, 9\}$. Someone says the array after sorting is $\{1, 3, 5, 7\}$. Can we consider the answer is correct? The answer is definitely “**no**” because the elements of the output set are not taken from the input set. Let someone says the array after sorting is $\{2, 5, 3, 9\}$. Can we admit the answer? The answer is again “**no**” because the output is not satisfying the objective function that is the first element must be less than the second, the second element must be less than the third and so on. Therefore, the solution is said to be a feasible solution if it satisfies the following constraints.

- (i) **Explicit constraints:** - The elements of the output set must be taken from the input set.
- (ii) **Implicit constraints:** - The objective function defined in the problem.

The best of all possible solutions is called the optimal solution. In other words we need to find the solution which has the optimal (maximum or minimum) value satisfying the given constraints.

The Greedy approach constructs the solution through a sequence of steps. Each step is chosen such that it is the best alternative among all feasible choices that are available. The choice of a step once made cannot be changed in subsequent steps.

Let us consider the problem of coin change. Suppose a greedy person has some 25p, 20p, 10p, 5paise coins. When someone asks him for some change then he wants to give the change with minimum number of coins. Now, let someone requests for a change of 45p then he first selects 25p. Then the remaining amount is 20p. Next, he selects the largest coin that is less than or equal to 20p i.e. 20p.

The remaining 20p is paid by selecting a 20p coin. So the demand for 45p is paid by giving total 3 numbers of coins. This solution is an optimal solution. Now, let someone requests for a change of 40p then the Greedy approach first selects 25p coin, then a 10p coin and finally a 5p coin.

Design and Analysis of Algorithm

However, the some could be paid with two 20p coins. So it is clear from this example that Greedy approach tries to find the optimal solution by selecting the elements one by one that are locally optimal. But Greedy method never gives the guarantee to find the optimal solution.

The choice of each step is a greedy approach is done based in the following:

- It must be feasible
- It must be locally optimal
- It must be unalterable

3.2 Fractional Knapsack Problem

Let there are n number of objects and each object is having a weight and contribution to profit. The knapsack of capacity M is given. The objective is to fill the knapsack in such a way that profit shall be maximum. We allow a fraction of item to be added to the knapsack.

Algorithm:

Greedy(D,n)

// in Greedy approach D is a domain

//from which solution is to be obtained of size n

//Initially assume

Solution=0;

For i=0 to n do

{

S= select(D) //select solution from D

If(feasible(solution,s)) then // check if the selected solution is feasible or not

Solution=union(solution,S);//make feasible choices and select optimum solution

}

Return solution;

}

Applications of greedy method:

- Job Sequencing with deadlines
- Minimum spanning tree

Design and Analysis of Algorithm

3.3 Job Sequencing

Consider that there are n jobs that are to be executed. At any time $t=1,2,3,\dots$ only exactly one job is to be executed. The profits p_i are given. These profits are gained by corresponding jobs. For obtaining feasible solution we should take care that the jobs get completed within their given deadlines.

Let $n=4$

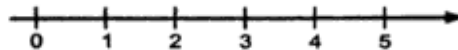
n	p_i	d_i
1	70	2
2	12	1
3	18	2
4	35	1

We will follow following rules to obtain feasible solution

- Each job takes one unit of time.
- If job starts before or at its deadline, profit is obtained, otherwise no profit.
- Goal is to schedule jobs to maximize the total profit.
- Consider all possible schedules and compute the minimum total time in the system

Design and Analysis of Algorithm

Consider the Time line as



The feasible solutions are obtained by various permutations and combinations of jobs.

n	p _i
1	70
2	12
3	18
4	35
1, 3	88
2, 1	82
2, 3	30
3, 1	88
4, 1	105
4, 3	53

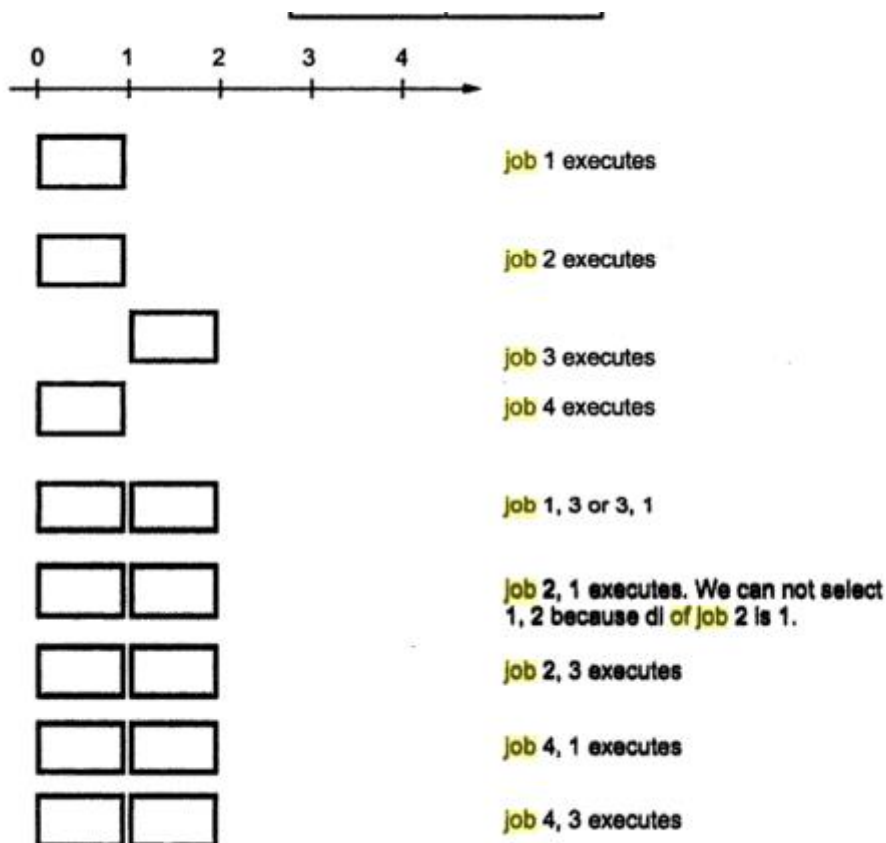


Fig. 3.1 Job sequencing with deadline

Each job takes only one unit of time. Deadline of job means a time on which or before which the job has to be executed. The sequence {2,4} is not allowed because both have deadline 1. If job 2 is started at 0 it will be completed on 1 but we cannot start job 4 on 1 since deadline of job 4 is 1. the feasible

Design and Analysis of Algorithm

sequence is a sequence that allows all jobs in a sequence to be executed within their deadlines and highest profit can be gained.

The optimal solution is a feasible solution with maximum profit.

In above example sequence 3,2 is not considered as $d_3 > d_2$ but we have considered the sequence 2,3 as feasible solution because $d_2 < d_3$

We have chosen job 1 first then we have chosen job 4. The solution 4, 1 is feasible as the order of execution is 4 then 1. Also $d_4 < d_1$. If we try {1,3,4} then it is not a feasible solution, hence reject 3 from the set. Similarly, if we add job 2 in the sequence then the sequence becomes {1,2,4}. This is also not a feasible solution hence reject it. Finally the feasible sequence is 4,1. This sequence is optimum solution as well.

Algorithm:

Algorithm Job_seq(D,J,n)

{

//Problem description: This algorithm is for job sequencing using Greedy method.

//D[i] denotes i^{th} deadline where $1 \leq i \leq n$

// J[i] denotes i^{th} job

// $D[J[i]] \leq D[J[i+1]]$

Initially $D[0] = 0$; $J[0] = 0$; $J[1] = 1$;

Count = 1 ;

For 2 to n do

{t = count;

While $D[J[t]] > D[1]$ AND $D[J[t]] \neq t$) do $t = t-1$;

If $((D[J[t]] \leq D(i)$ AND $(D(i) > t))$ then

{

// insertion of i^{th} feasible sequence into J array

For $s \leftarrow \text{count}$ to $(t+1)$ step -1 do

$J[s+1] \leftarrow J[s]$;

$J[t+1] \leftarrow i$;

Count $\leftarrow \text{count} + 1$;

}// end of if;

Design and Analysis of Algorithm

```
} // end of while
```

```
Return count;
```

```
}
```

The sequence of J will be inserted if and only if $D[J[t]] \neq t$. This also means that the job J will be processed if it is within the deadline.

The computing time taken by above Job_seq algorithm is $O(n^2)$, because the basic operation of computing sequence in array J is within two nested for loops.

3.4 Optimal Merge Pattern

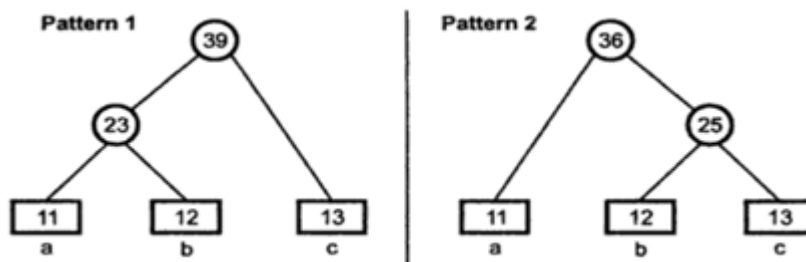
The merge sort is a technique in which two sorted files can be merged together in one sorted file. When two or more files need to be merged in a single file, then the merging of two sorted files must be performed repeatedly. That means merging of sorted files must be done pair-wise.

For instance :if there are a, b, c and d files which are already sorted and has to be merged then-

- We may merge a, b to produce x_1 then x_1 can be merged with file c to produce x_2 , then x_2 can be merged with file d to produce x_3 .
- We may merge a, b to produce x_1 , then merge c and d to produce x_2 finally merge x_1 and x_2 to produce x_3 .

Thus different pairings can be possible to produce final merged file. Clearly these pairings may require different amount of computing time. The pairing of files which require optimum (minimum) computing time to get a single merged file denotes the optimal merge pattern.

For example :consider that there are 3 files a, b, c of lengths 11, 12 and 13 respectively. Then following are two sample merge patterns that can be produced.



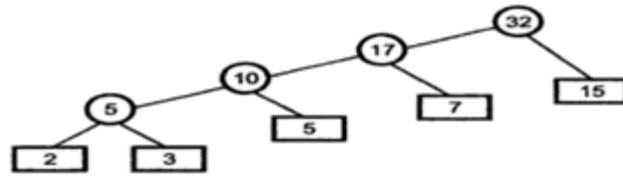
Pattern 2 is faster than first pattern. A Greedy method is used to obtain optimal merge pattern

Representation of Optimal Merge pattern:

The merge pattern is typically represented by a binary tree, it is also called as two way merge pattern.

For example 2,3,5,7,15 are six files with given lengths then we can build a merge pattern as

Design and Analysis of Algorithm



The leaf nodes denoted by squares called external nodes and remaining nodes are called internal nodes. The path length from root to corresponding external node is called external path length. An application of optimal merge pattern is Huffman code

3.5 Huffman Code

Huffman's coding is a technique for encoding data, by which data compression can be done. This technique is used for lossless data compression. this algorithm was developed by David A.Huffman.

Each character is represented in 8 bits when characters are coded using standard codes such as ASCII. It can be seen that the characters coded using standard codes have fixed-length code word representation. In this fixed-length coding system the total code length is more. For example, let we have six characters (a, b, c, d, e, f) and their frequency of occurrence in a message is {45, 13, 12, 16, 9, 5}. In fixed-length coding system we can use three characters to represent each code. Then the total code length of the message is $(45+13+12+16+9+5) \times 3 = 100 \times 3 = 300$.

From the table of frequencies Huffman's tree is constructed using Greedy approach.

The Huffman's tree is further used for encoding each character, so that binary encoding is obtained for given data.

In Huffman coding there is a specific method of representing each symbol. This method produces a code in such a manner that no code word is prefix of some other code word. such codes are called prefix code or prefix-free codes. Thus this method is useful for obtaining optimal data compression.

Example: Obtain the Huffman's encoding for following data

A:39 b:10 c:9 d:25 e:7 f:3

Solution: Basically there are two types of coding-fixed length encoding and variable length coding.If we use fixed length coding we need 3 bits to represent any character from a to h.

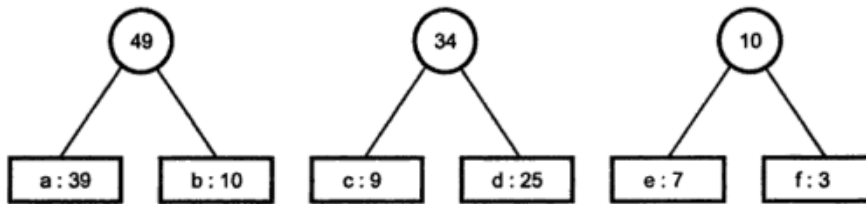
Let us arrange the symbols along with their frequencies as follows:

Design and Analysis of Algorithm

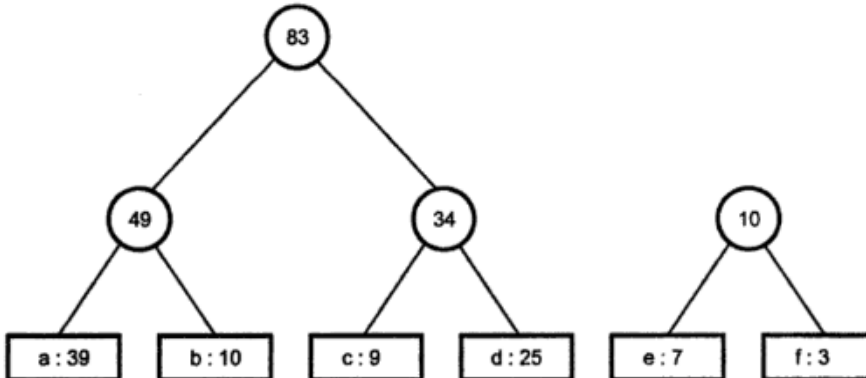
Step 1 :



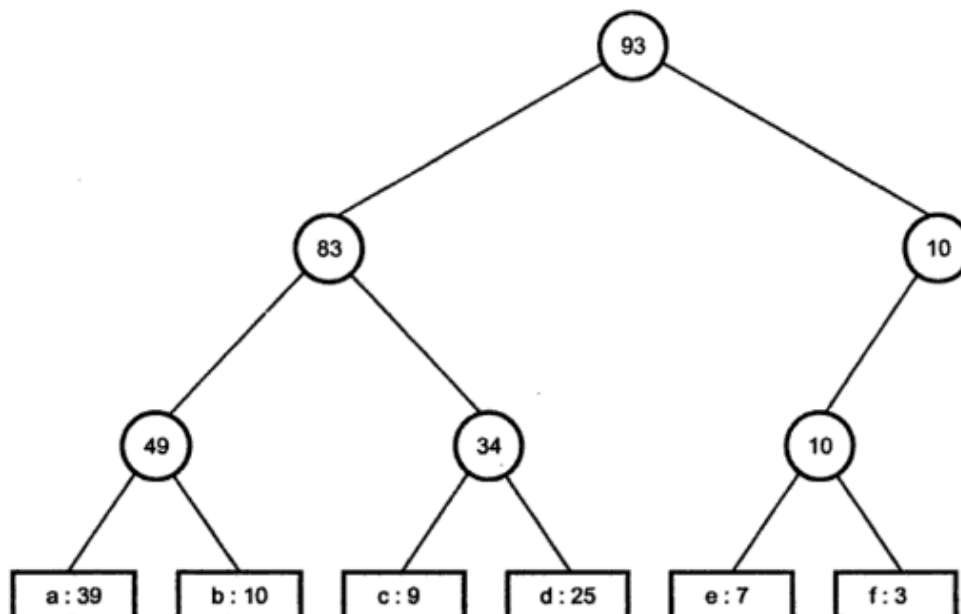
Step 2 :



Step 3 :



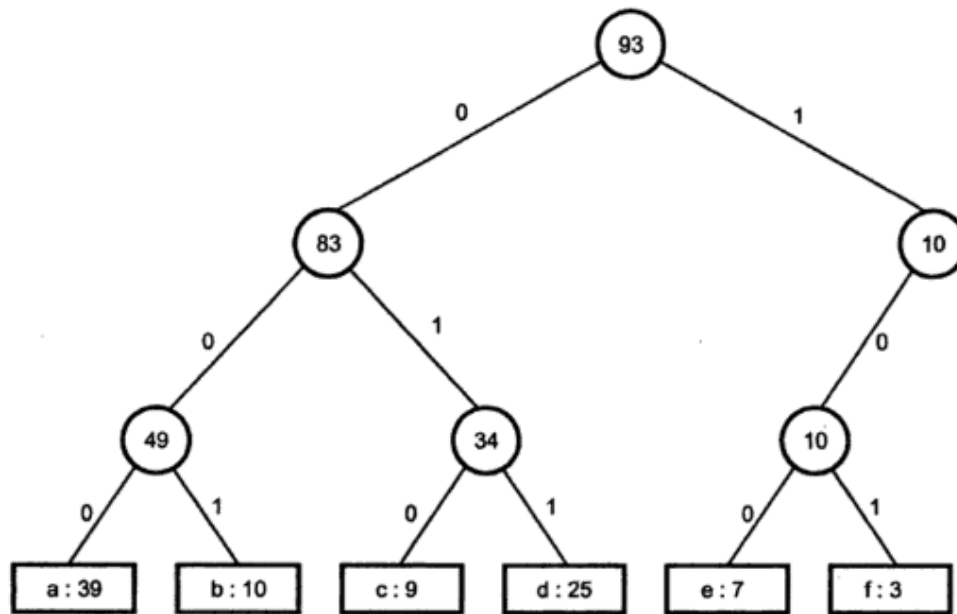
Step 4 :



Now we start encoding this tree. For that purpose we start coding the tree from top to down. If we follow the **left branch** then we should encode it as "0" and if we follow the **right branch** then we should encode it as "1". Hence we get

Design and Analysis of Algorithm

Step 5 :



Step 6 :

Symbol	Code word
a	000
b	001
c	010
d	011
e	100
f	101

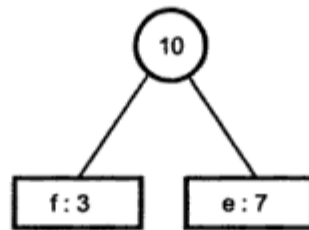
Step 7:

Hence if the string is “aeed” then we get 000100100011 as a code word. But this method is not that much efficient.

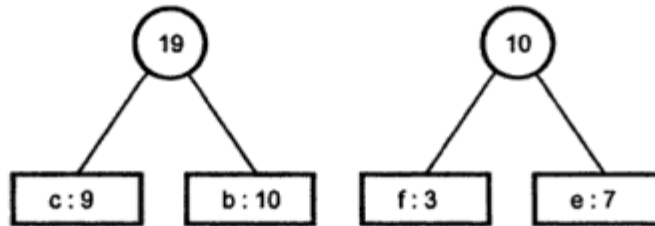
Hence variable length encoding has come up to bring efficiency in Huffman’s algorithm. By this method, we assign short code word for frequently used characters and long code word for infrequent characters.

Design and Analysis of Algorithm

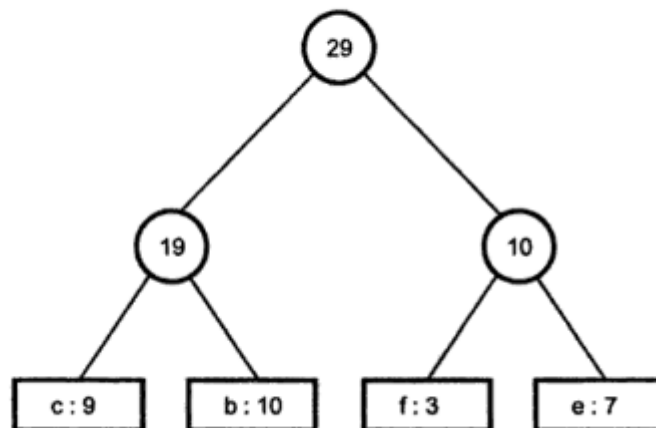
Step 1 :



Step 2 :

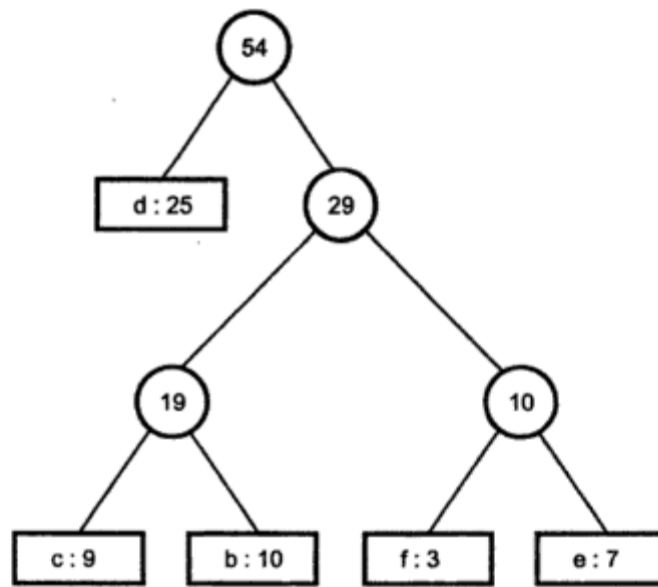


Step 3 :

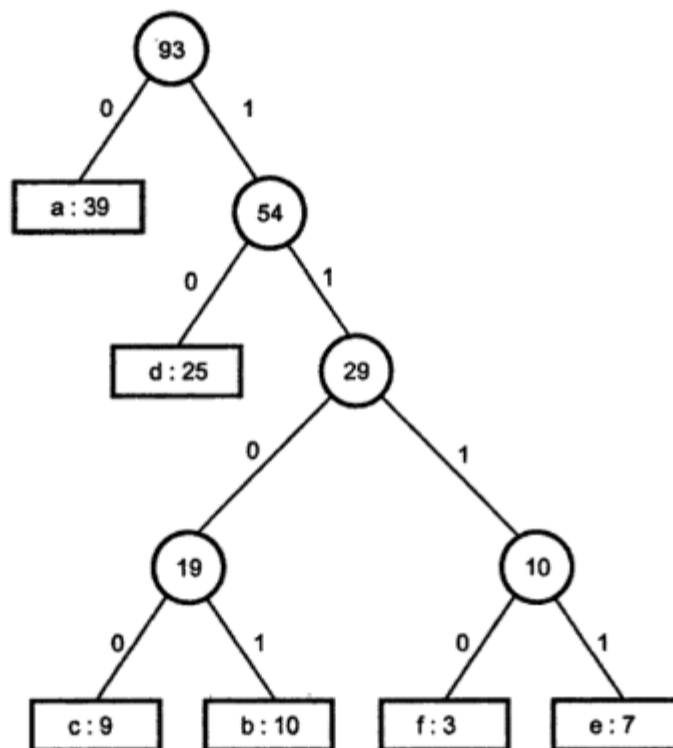


Design and Analysis of Algorithm

Step 4 :



Step 5 :



Design and Analysis of Algorithm

Step 6 :

Symbol	Code word
a	0
b	1101
c	1100
d	10
e	1111
f	1110

Step 7:

The encoding for the word “aeed” will be 01111111110.

The method of using variable length encoding is really efficient than fixed length encoding.

Consider, for each character number of bits required = frequency * number of bits used for representation.

i.e Total bits required in fixed length encoding

$$= 39 * 3 + 10 * 3 + 9 * 3 + 25 * 3 + 7 * 3 + 3 * 3 = 279 \text{ bits}$$

Similarly, Total bits required in Variable length encoding

$$= 39 * 1 + 25 * 2 + 9 * 4 + 10 * 4 + 7 * 4 + 3 * 4 = 205 \text{ bits}$$

Clearly Variable length encoding requires less number of bits than fixed length encoding.

Huffman's Algorithm: The Greedy method is used to construct optimal prefix code called Huffman code. This algorithm builds a tree in bottom up manner. We can denote this tree by T.

Let $|c|$ be number of leaves.

$|c| - 1$ are number of operations required to merge the nodes.

Q be the priority queue which can be used while constructing binary heap

Algorithm Huffman (c)

```
{
n = | c |
Q = c
For i ← 1 to n-1
Do
{
Temp ← get_node ()
Left(temp) = Get_min (Q)
Right (temp) = Get_min (Q)
a = left(temp)
b=right(temp)
```

Design and Analysis of Algorithm

```
f(temp)  $\leftarrow$  f(a) + f(b)
insert (Q,temp)
}
Return Get_min (Q)
```

Analysis:

The Priority queue Q can be implemented as binary Heap. The construction of binary heap can be done in $O(n)$ time. The for loop will be executed in $O(\log n)$ time. Hence for $(n-1)$ nodes $O(n \log n)$ time will be required. therefore total running time of Huffman's algorithm will be $O(n \log n)$

Graphs:

Definition of Graph:

A graph is a collection of two sets V and E where V is a finite non empty set of vertices and E is a finite non empty set of edges.

Vertices are nothing but the nodes in the graph and the two adjacent vertices are joined by edges. The graph is thus a set of two sets.

Any graph G is denoted by $G = \{V, E\}$

Type of Graphs:

Basically there are two graphs:

Directed Graphs

Undirected Graphs

In the directed graph the directions are shown on the edges. The edges between the vertices are ordered. In this type of graph, the edge E_1 is in between the vertices V_1 and V_2 . The V_1 is called head and the V_2 is called the tail. Similarly for V_1 head, the tail is V_3 and so on.

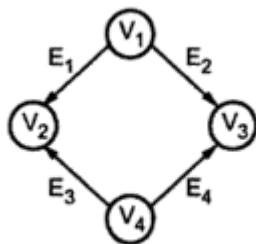


Fig. 3.3 Directed graph

We can say E_1 is the set of (V_1, V_2) and not of (V_2, V_1) . Similarly, in an undirected graph, the edges are not ordered. In this type, the edge E_1 is set of (V_1, V_2) or (V_2, V_1) .

Design and Analysis of Algorithm

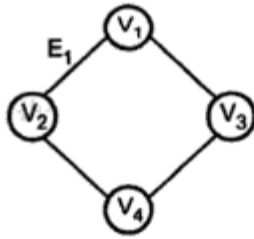


Fig. 3.4 Undirected graph

3.6 Minimum Spanning tree

Spanning tree:

A spanning tree of a graph G is a subgraph which is basically a tree and it contains all the vertices of G containing no circuit.

Minimum Spanning tree:

A minimum spanning tree of a weighted connected graph G is a spanning tree with minimum or smallest weight.

Weight of the tree:

A weight of the tree is defined as the sum of weights of all its edges.

For example: consider a weighted connected graph G because some weights are given along every edge and the graph is a connected graph.

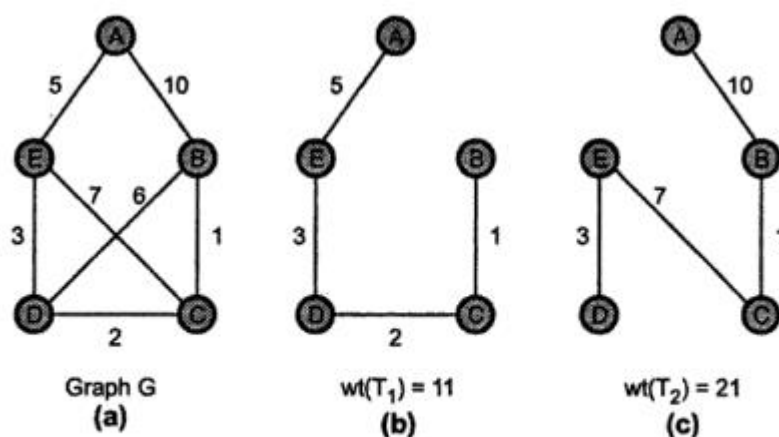


Fig. 3.12 Graph and two spanning trees out of which (b) is a minimum spanning tree

Application of spanning trees:

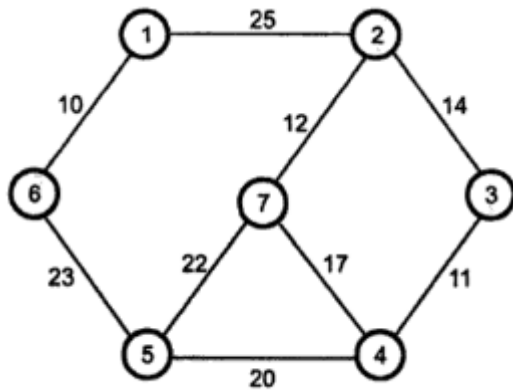
1. Spanning trees are very important in designing efficient routing algorithms

Design and Analysis of Algorithm

2. Spanning trees have wide applications in many areas such as network design.

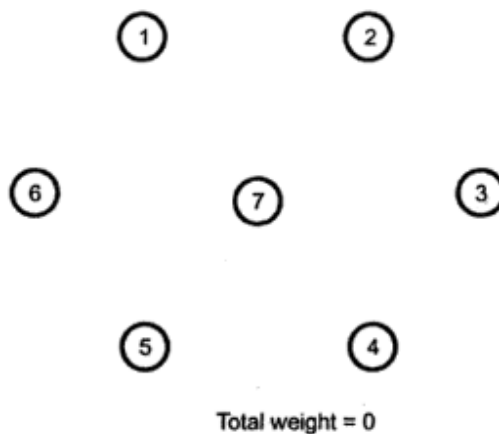
3.6.1 Prim's Algorithm

Example of Prim's algorithm: consider the below graph:

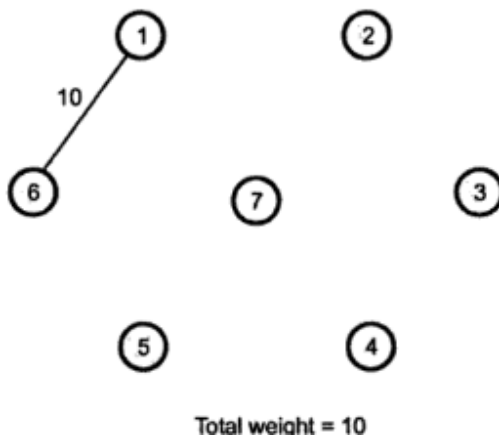


Now, we will consider all the vertices first. Then we will select an edge with minimum weight. The algorithm proceeds by selecting adjacent edges with minimum weight. Care should be taken for not forming circuit

Step 1 :

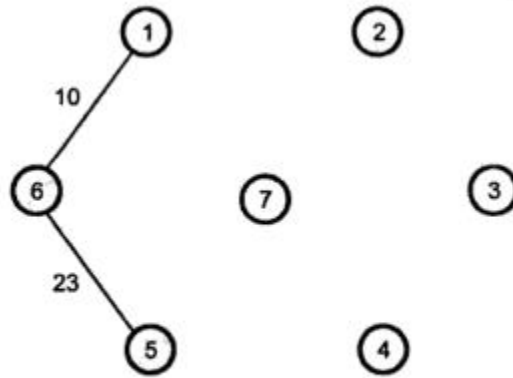


Step 2 :



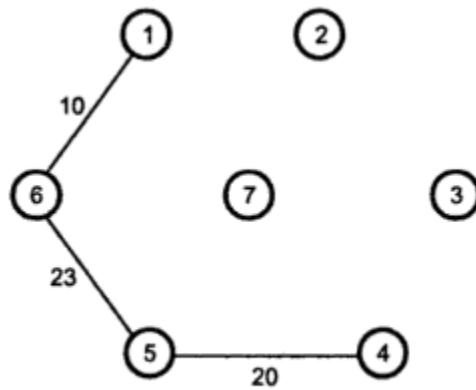
Design and Analysis of Algorithm

Step 3 :



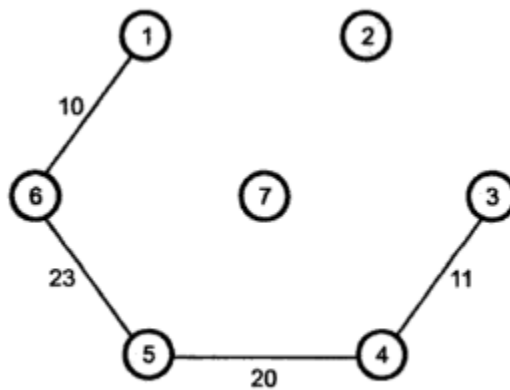
Total weight = 33

Step 4 :



Total weight = 53

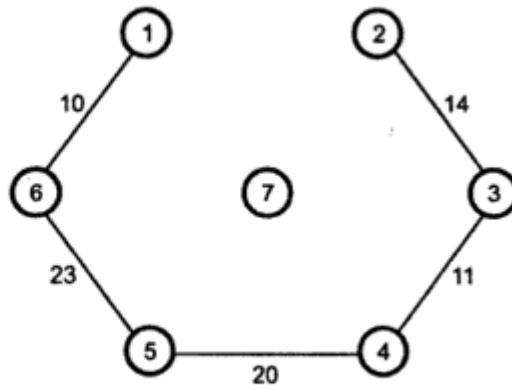
Step 5 :



Total weight = 64

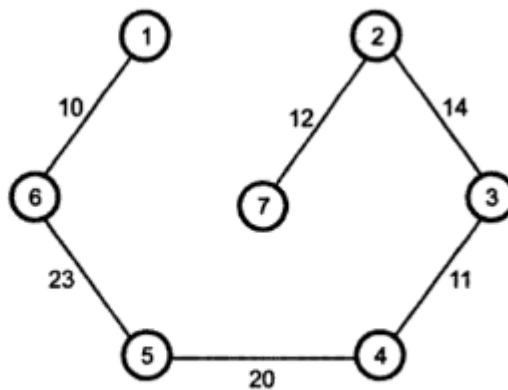
Design and Analysis of Algorithm

Step 6 :



Total weight = 78

Step 7 :



Total weight = 90

Algorithm:

Prim (G[0,...size-1,0....size-1],nodes)

//Problem Description:This algorithm is for implementing Prim's algorithm for finding the spanning tree

//Input:Weighted Graph G and total number of nodes

//Output:spanning tree gets printed with total path length

Total = 0;

//Initialize the selected vertices list

For i ← 0 to nodes -1 do

Tree[i] ← 0

Tree[0] =1 //take initial vertex

Design and Analysis of Algorithm

For K ← 1 to nodes do

{

Min_dist ← ∞

// initially assign minimum dist as infinity

For i ← 0 to nodes-1 do

{

For j ← 0 to nodes-1 do (

If (G[i,j] AND ((tree[i] AND !tree[j]) OR

(! Tree[i] AND tree[j]))) then

{

If (G[i,j] < Min_dist then

//select an edge such that one vertex is selected and the other is not and the edge has the least weight

{

Min_dist ← G[i,j] // Obtained edge with minimum wt.

V1 ← i;

V2 ← j; // Picking up those vertices yielding minimum edge

}}}}

Write (V1,V2,Min_dist);

Tree[V1] ← tree{V2] ← 1

Total ← total +min_dist

}

Write("Total path length is ",Total)

Analysis: The algorithm spends most of the time in selecting the edge with minimum length. Hence the basic operation of this algorithm is to find the edge with minimum path length. This can be given by following formula

Design and Analysis of Algorithm

$$T(n) = \sum_{k=1}^{nodes-1} \left(\sum_{i=0}^{nodes-1} 1 + \sum_{j=0}^{nodes-1} 1 \right)$$

Time taken by
for k=1 to nodes-1 loop
Time taken by
for i=0 to nodes-1 loop
Time taken by
for j=0 to nodes-1 loop

We take variable n for 'nodes' for the sake of simplicity of solving the equation then

$$\begin{aligned}
 T(n) &= \sum_{k=1}^{n-1} \left(\sum_{i=0}^{n-1} 1 + \sum_{j=0}^{n-1} 1 \right) \\
 &= \sum_{k=1}^{n-1} [(n-1) + 0 + 1] + [(n-1) + 0 + 1] \\
 &= \sum_{k=1}^{n-1} 2n \\
 &= 2n \sum_{k=1}^{n-1} 1 \\
 &= 2n ((n-1) - 1 + 1) = 2n(n-1) \\
 &= 2n^2 - 2n \\
 T(n) &= n^2 \\
 T(n) &= \Theta(n^2)
 \end{aligned}$$

N= no.of.vertices or nodes of the tree

The time complexity of the Prim's algorithm $T(n) = \Theta(n^2)$

3.6.2 Kruskal's Algorithm

It is an another algorithm for obtaining minimum spanning tree. In this algorithm always the minimum cost edge has to be selected. But it is not necessary that selected optimum edge is adjacent.

Difference between Prim's and Kruskal's Algorithm:

Prim's Algorithm	Kruskal's Algorithm
This algorithm is for obtaining the minimum spanning tree by selecting the adjacent vertices of already selected vertices	This algorithm is for obtaining the minimum spanning tree but it is not necessary to choose adjacent vertices of already selected vertices

Example:

Consider the graph given below:

Design and Analysis of Algorithm

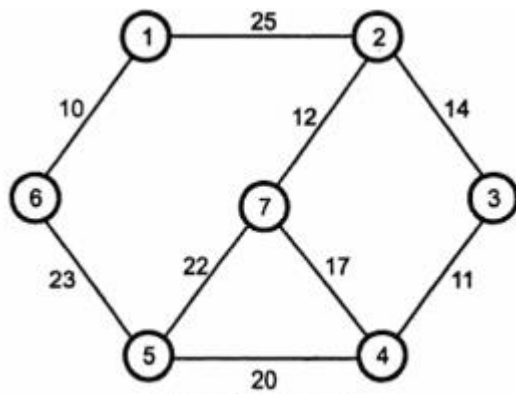
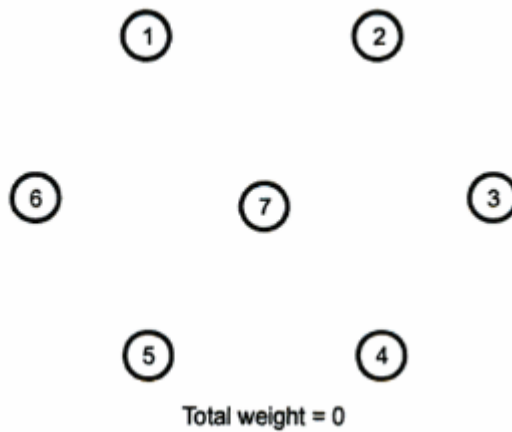


Fig. 3.15 Graph

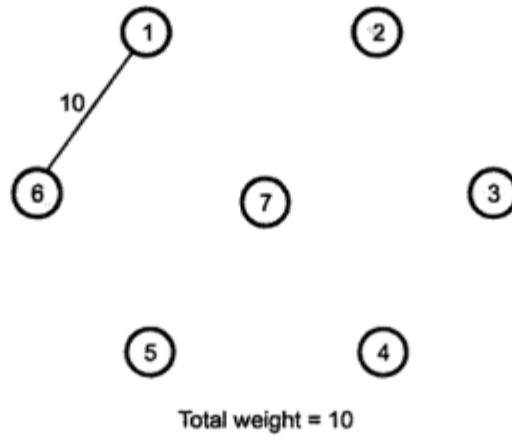
First , we will select all the vertices .Then an edge with optimum weight is selected from heap,even though it is not adjacent to previously seelcted edge.Care should be taken for not forming circuit.

Step 1 :

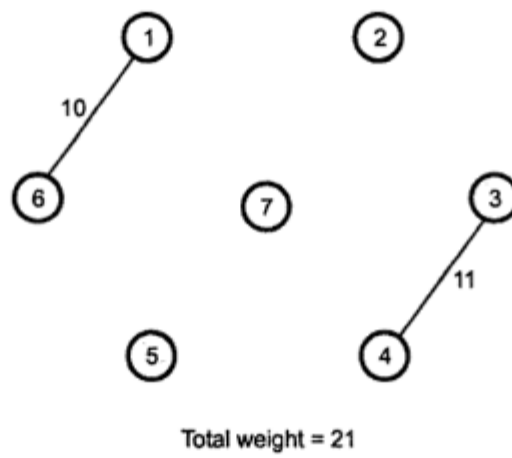


Design and Analysis of Algorithm

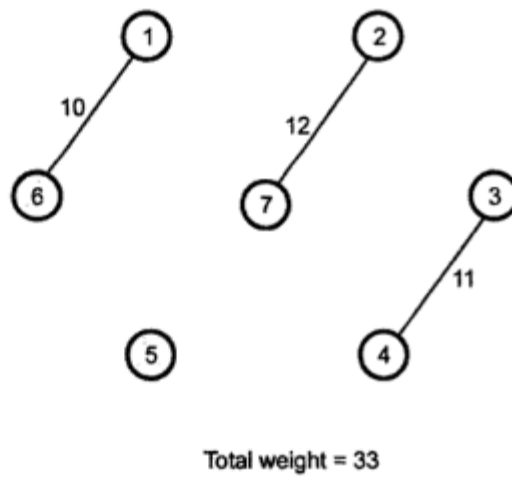
Step 2 :



Step 3 :

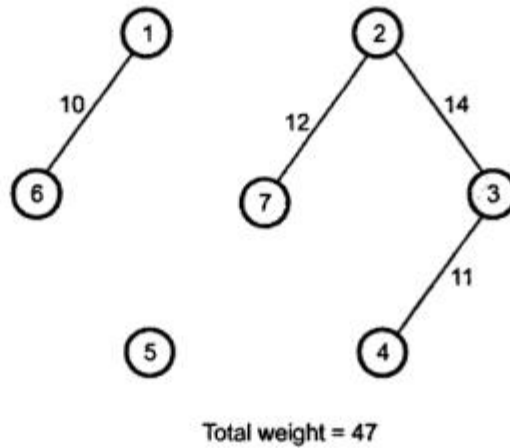


Step 4 :

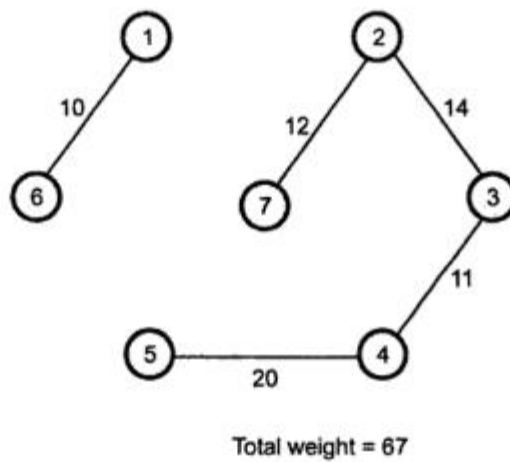


Design and Analysis of Algorithm

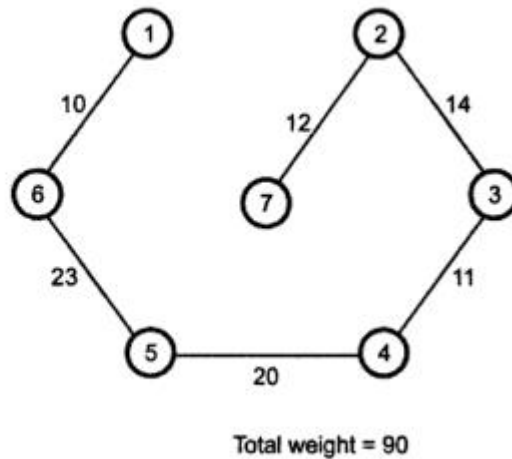
Step 5 :



Step 6 :



Step 7 :



Algorithm: spanning_tree()

//Problem Description: This algorithm finds the minimum spanning tree using Kruskal's Algorithm

// Input: The adjacency matrix graph G containing cost

//Output: Prints the spanning tree with the total cost of spanning tree\

Count $\leftarrow$ 0

K $\leftarrow$ 0

Sum $\leftarrow$ 0

For i $\leftarrow$ 0 to tot_nodes do

Design and Analysis of Algorithm

```
parents[i] ← i
while ( count != tot_nodes -1 ) do
{
Post _Minimum (tot_edges); // finding the minimum cost edge
If(pos = -1 then //Perhaps no node in the graph
Break
V1 ← G[pos].V1
V2 ← G[pos].V2
i ←find(v1,parent)
j ←Find(v2,parent)
if( i !=j) then
{
Tree[k][0] ← V1 //Tree[] is an array in which the spanning tree edges are stored.
Tree[k] [1] ← v2
K++
Count ++;
Sum + ← G[pos].cost //computing the total cost of all the minimum distances
Union( I,j,parent);
}
G[pos].cost INFINITY
}
If(count = tot_nodes -1 ) then
{
For i ← 0 to tot_nodes -1
{
Write (tree[i][0],tree[i] [1] //for each node of I, the minimum distnace edges are collected in the array
tree[[]].The spanning treeis printed here
}
Write (“cost of spanning tree is”,sum)
}
```

Analysis: The efficiency of Kruskal’s algorithm is $\Theta(|E| \log |E|)$ where E is the edges in the graph.

Time complexity of Kruskal’s Algorithm

The Kruskal’s algorithm first creates n trees from n vertices which is done in $O(n)$ time. Then, a heap is created in $O(n)$ time using heapify procedure. The least weight edge is at the root of the heap. Hence, the edges are deleted one by one from the heap and either added to the MST or discarded if it forms a cycle. This deletion process requires $O(n \log_2 n)$. Hence, the time complexity of Kruskal’s algorithm is $O(n \log_2 n)$.

3.7 Shortest Path Problem

Let us consider a number of cities connected with roads and a traveler wants to travel from his home city A to the destination B with a minimum cost. So the traveler will be interested to know the following:

- Is there a path from city A to city B?

Design and Analysis of Algorithm

- If there is more than one path from A to B, which is the shortest or least cost path?

Let us consider the graph $G = (V, E)$, a weighting function $w(e)$ for the edges in E and a source node v_0 . The problem is to determine the shortest path from v_0 to all the remaining nodes of G . The solution to this problem is suggested by E.W. Dijkstra and the algorithm is popularly known as Dijkstra's algorithm.

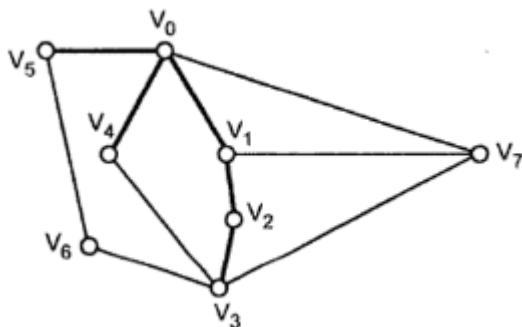
This algorithm finds the shortest paths one by one. If we have already constructed i shortest paths, then the next path to be constructed should be the next shortest path. Let S be the set of vertices to which the shortest paths have already been generated. For z not in S , let $dist[z]$ be the length of the shortest path starting from v_0 , going through only those vertices that are in S and ending at z . Let u be the vertex in S to which the shortest path has already been found. If $dist[z] > dist[u] + w(u,z)$ then $dist[z]$ is updated to $dist[u] + w(u,z)$ and the predecessor of z is set to u .

3.7.1 Dijkstra's Algorithm

It is a popular algorithm for finding the shortest path. This is also called the single source shortest path algorithm. In this algorithm, for a given vertex called source the shortest path to all other vertices is obtained and the main focus is not to find only one single path but to find the shortest paths from any vertex to all other remaining vertices.

This algorithm is applicable to graphs with non-negative weight only.

This algorithm finds shortest paths to graph's vertices in order of their distance from a given source. In this process of finding shortest path, first it finds the shortest path from the source to a vertex nearest to it, then second nearest and so on.



The shortest path from V_0 is obtained. First we find shortest path from V_0 - V_1 then V_1 - V_2 then from V_2 - V_3 the shortest distance is obtained. Consider a weighted connected graph below:

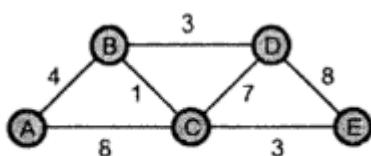
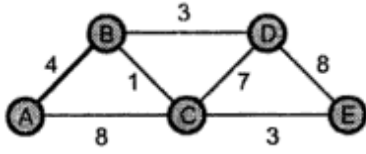
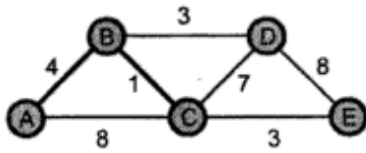
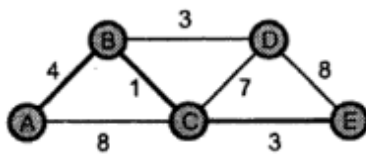


Fig. 10.11

Design and Analysis of Algorithm

Now, we will consider each vertex as a source and will find the shortest distance from this vertex to every other remaining vertex. Let us start with vertex A

Source vertex	Distance with other vertices	Path shown in graph
A	A-B, path = 4 A-C, path = 8 A-D, path = ∞ A-E, path = ∞	
B	B-C, path = 4 + 1 B-D, path = 4 + 3 B-E, path = ∞	
C	C-D, path = 5 + 7 = 12 C-E, path = 5 + 3 = 8	
D	D-E, path = 7 + 8 = 15	

But we have one shortest distance obtained from A to E and that is A-B-V-E with path length =4+1+3=8. Similarly other shortest paths can be obtained by choosing appropriate source and destination.

Path[v2] $\leftarrow$ v1

}}}} // V1 is next selected destination vertex with shortest distance .All such vertices are accumulated in array path []

Chapter 4: Dynamic Programming

4.1 Introduction

The Dynamic Programming (DP) is the most powerful design technique for solving optimization problems. It was invented by mathematician named Richard Bellman inn 1950s. The DP in closely related to divide and conquer techniques, where the problem is divided into smaller sub-problems and each sub-problem is solved recursively. The DP differs from divide and conquer in a way that instead of solving sub-problems recursively, it solves each of the sub-problems only once and stores the solution to the sub-problems in a table. The solution to the main problem is obtained by the solutions of these sub- problems.

Design and Analysis of Algorithm

The steps of Dynamic Programming technique are:

- **Dividing the problem into sub-problems:** The main problem is divided into smaller sub-problems. The solution of the main problem is expressed in terms of the solution for the smaller sub-problems.
- **Storing the sub solutions in a table:** The solution for each sub-problem is stored in a table so that it can be referred many times whenever required.

Design and Analysis of Algorithm

- **Bottom-up computation:** The DP technique starts with the smallest problem instance and develops the solution to sub instances of longer size and finally obtains the solution of the original problem instance.

The strategy can be used when the process of obtaining a solution of a problem can be viewed as a sequence of decisions. The problems of this type can be solved by taking an optimal sequence of decisions. An optimal sequence of decisions is found by taking one decision at a time and never making an erroneous decision. In Dynamic Programming, an optimal sequence of decisions is arrived at by using the principle of optimality. *The principle of optimality states that whatever be the initial state and decision, the remaining decisions must constitute an optimal decision sequence with regard to the state resulting from the first decision.*

A fundamental difference between the greedy strategy and dynamic programming is that in the greedy strategy only one decision sequence is generated, whereas in the dynamic programming, a number of them may be generated. Dynamic programming technique guarantees the optimal solution for a problem whereas greedy method never gives such guarantee.

Matrix chain Multiplication

Let, we have three matrices A_1 , A_2 and A_3 , with order (10×100) , (100×5) and (5×50) respectively.

Then the three matrices can be multiplied in two ways.

- (i) First, multiplying A_2 and A_3 , then multiplying A_1 with the resultant matrix i.e. $A_1(A_2 A_3)$.
- (ii) First, multiplying A_1 and A_2 , and then multiplying the resultant matrix with A_3 i.e. $(A_1 A_2) A_3$.

DYNAMIC PROGRAMMING & TRAVERSAL TECHNIQUES

General Method:

- Dynamic programming is typically applied to optimizing problem. Here the programming stands for planning.
- Dynamic programming is technique for solving problems with overlapping sub problems.
- In this method, each sub problem is solved only once. The result of each sub problem is recorded in a table from which we can obtain a solution to the original problem.
- For each given problem, we may get any number of solutions we seek for optimal solution (maximum value or minimum value solution). and such an optimal solution becomes the solution of the given problem.

Steps of Dynamic Programming:

It involves 4 steps:

- Characterize the structure of optimal solution. That means develop a mathematical notation that can express any solution and sub solution for the given subproblem.
- Recursively define the value of an optimal solution.
- By using bottom-up technique compute the value of optimal solution. for that you

Design and Analysis of Algorithm

have to develop a recurrence relation that relates a solution to its subproblems using mathematical notation of step 1.

- Compute an optimal solution from computed information.

Difference between divide and conquer and dynamic programming

Divide and conquer	Dynamic programming
The problem is divided into sub problems. these sub problems are solved independently. Finally, all the solutions of sub problems are collected together to get the solution to the given problem.	In this, many decisions sequence is generated and all the overlapping sub instances are considered
In this method, duplications in sub solutions are Neglected	Duplications in solutions is avoided totally
It is less efficient because of rework of solutions	It is more efficient
It uses top-down approach for problem solving	It uses bottom-up approach for problem solving
It splits the input at specific deterministic points usually in the middle	It split its input at every possible split points rather than t a particular point. After trying all split points, it determines which split point is optimal.

Difference between Greedy Algorithm and dynamic programming

Greedy Method	Dynamic Programming
It is used for obtaining optimum solution.	It is also used for obtaining optimum solution.

In greedy method, a set of feasible solutions and the picks up the optimum solution	There is no special set of feasible solution in this method.
The optimum selection is without revising previously generated solutions	It considers all possible sequences in order to obtain the optimum solution.
There is no such guarantee of getting optimum solution	It is guaranteed that the dynamic programming will generate optimal solution using principle of optimality.

Principle of optimality:

The dynamic programming algorithm obtains the solution using principle of optimality.

The principle of optimality states that, “In an optimal sequence of decisions or choices, each subsequence must also be optimal”.

When it's not possible to apply principle of optimality it is almost impossible to obtain the solution using dynamic programming.

Applications of dynamic programming:

- All pairs shortest path problem,
- Multi stage graph
- o/I knapsack problem,
- optimal binary search tree.

4.2 Multi stage Graphs

- A multi stage graph $G=(V,E)$ which is a directed graph.
- In this graph all the vertices are partitioned into the K stages where $K \geq 2$.
- In multi stage problem, we have to find the shortest path from source to sink. The cost of each path is calculated by using the weight given along that edge.
- The cost of path from source (denoted by S) to sink (denoted by T) is the sum of the costs of edges on the path.
- In Multistage graph problem we have to find the path from S to T . there is a set of vertices in each stage. The multi stage graph can be solved using forward and backward approach.

Let is solve multistage graph for both the approaches with the help of some example.

Consider the graph G as shown in fig.

There is a single vertex in stage 1, then 3 vertices in stage 2, then 2 vertices in stage 3 and only one vertex in stage 4 (this is a target stage)

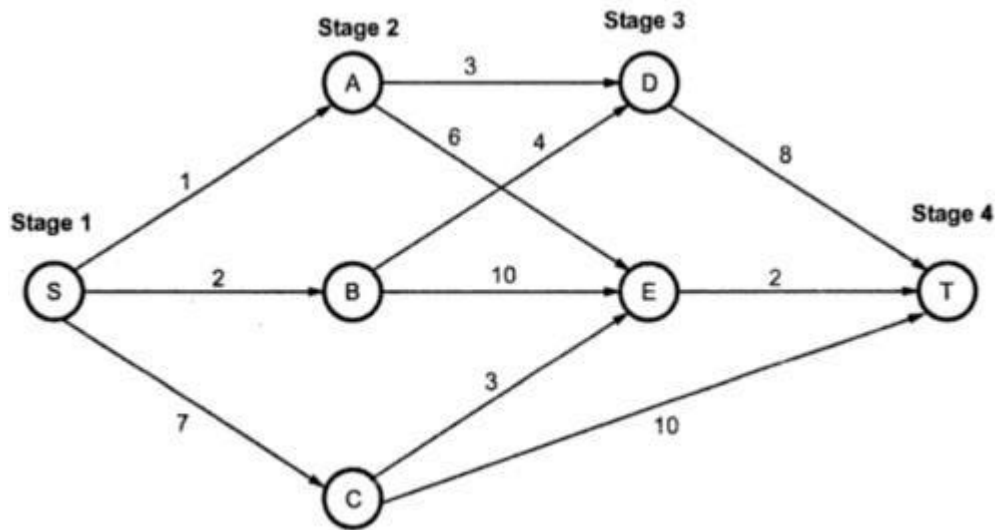


Fig. 4.1 Multistage graph

Backward approach:

$$d(S,T) = \min \{ 1 + d(A,T), 2 + d(B,T), 7 + d(C,T) \} \text{ ----- (1)}$$

We will now compute $d(A,T)$, $d(B,T)$ and $d(C,T)$.

$$d(A,T) = \min \{ 3 + d(D,T), 6 + d(E,T) \} \text{ ----- (2)}$$

$$d(B,T) = \min \{ 4 + d(D,T), 10 + d(E,T) \} \text{ ----- (3)}$$

$$d(C,T) = \min \{ 3 + d(E,T), d(C,T) \} \text{ ----- (4)}$$

now let us compute $d(D,T)$ and $d(E,T)$

$$d(D,T) = 8$$

$$d(E,T) = 2 \quad \text{backward vertex is E}$$

Let us put these values in eq (2), (3), (4)

$$d(A,T) = \min \{ 3 + 8, 6 + 2 \} = 8 \quad \text{A-E-T}$$

$$d(B,T) = \min \{ 4 + 8, 10 + 2 \} = 12 \quad \text{A-D-T}$$

$$d(C,T) = \min \{ 3 + 2, 10 \} = 5 \quad \text{C-E-T}$$

$$\begin{aligned} d(S,T) &= \min \{ 1 + d(A,T), 2 + d(B,T), 7 + d(C,T) \} \\ &= \min \{ 1 + 8, 2 + 12, 7 + 5 \} \\ &= \min \{ 9, 14, 12 \} = 9 \quad \text{S-A-E-T} \end{aligned}$$

The path with minimum cost is S-A-E-T with the cost 9.

This method is called backward reasoning.

The algorithm for this method follows:

Algorithm **Backward_Gr** (G, stages, n, p)

//problem description: This algorithm is for backward approach of multistage graph

//Input: the multistage graph $G = (V, E)$ stages is for total number of stages.

Design and Analysis of Algorithm

```
// n is total number of vertices of G.  
// p is an array for restoring paths.  
//Output:TH epath with minimum cost.
```

Design and Analysis of Algorithm

```
Back_cost[i] ← 0 For( I
    ← 0 to n-2 ) do
{ r ← Get_min(I,n) // r is edge with min cost
  Back_cost[i] ← back_cost[r] + c[r][i] // computing backward cost of each vertex
  D[i] ← r //Assigning minimum distance in d array
}
```

// finding minimum cost path

P[0] ← 0

P[stages-1] ← n-1

For(I ← stages-1 to 1) do

P[i] ← d[p[I+1]];

Analysis: clearly the above algorithm has a time complexity $O(|V|+|E|)$.

Forward approach:

$d(S,A) = 1$

$d(S,B) = 2$

$d(S,C) = 7$

$d(S,D) = \min [1+d(A,D) + d(B,D)]$

$= \min \{ 1+3, 2+4 \}$

$d(S,D) = 4$

$d(S,E) = \min \{ 1 + d(A, E) , 2 + d(B,E) , 7 + d(C,E) \}$

$= \min \{ 1+6, 2+10, 7+3 \}$

$= \min \{ 7, 12, 10 \} = 7$ i.e path S-A-E is chosen.

$d(S,T) = \min \{ d(S,D) + d(D,T) , d(S,E) + d(E,T) , d(S,C) + d(C,T) \}$

$= \min \{ 4+8, 7+2, 7+10 \} = 9$ path S-E,E-T is chosen

The minimum cost = 9 with the path S-A-E-T

This method is called forward reasoning.

The algorithm for forward approach is

Algorithm Forward_Gr(G,stages,n,p)

//problem description: This algorithm is for forward approach of multistage graphs

//Input: The multistage graph $G=(V,E)$

//'stages' is the variable representing number of stages.

//n is the total number of vertices of G.

// p is an array of restoring path.

// output: the path with minimum cost

Cost[i] ← 0

For(i ← n-2 downto 0)\

{ r ← Get_min(I,n) // r is an edge with min cost.

Cost[i] ← c[i][r] + cost[r]

D[i] ← r

}

// finding minimum cost path

Design and Analysis of Algorithm

$P[0] \rightarrow 0$

$P[\text{stages}-1] \rightarrow n-1$

Design and Analysis of Algorithm

For (i=1 to stages-1) do

$P[i] \leftarrow d[p(i+1)]$;

Analysis: the algorithm has $O(|V|+|E|)$ time complexity

Using dynamic programming strategy, the multistage graph problem is solved. This is because in multistage graph problem, we obtain the minimum path at each current stage by considering the path length of each vertex obtained in earlier stage.

Thus, the sequence of decisions is taken by considering overlapped solution. In dynamic programming, we may get any number of solutions for given problem. From all these solutions we seek for optimal solution, finally optimal solution becomes the solution to given problem. Multistage graph problem is solved using this same approach.

4.3 ALL PAIRS SHORTEST PATH PROBLEM:

Problem description:

When a weighted graph, represented by its weight matrix W then objective is to find the distance between every pair of nodes.

We will apply dynamic programming to solve all pairs shortest path.

Step 1: we will decompose the given problem into subproblems

Let, $A^k(i,j)$ be the length of the shortest path from node i to node j such that the label for every intermediate node will be $\leq k$.

We will compute A^k for $k=1 \dots n$ for n nodes.

Step 2:

For solving all pair shortest path, the principle of optimality is used. That means any sub path of shortest path is a shortest path between the end nodes. Divide the path from i node to j node for every intermediate node, say K .

Then there are two arises,

Path going from i to j via k ,

Path which is not going via k , select only shortest path from two cases.

Step 3:

The shortest path can be computed using bottom-up computation method.

Following the recursive method,

Initially: $A^0 = W[i,j]$;

Next computation:

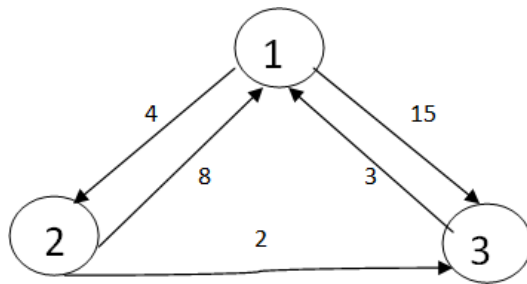
$A^k(i,j) = \min(A^{k-1}(i,j), A^{k-1}(i,k), A^{k-1}(k,j))$ where $1 \leq k \leq n$

The all pair shortest path algorithm is proposed by R.Floyd . hence this algorithm is sometimes called as **Floyd's algorithm**.

This algorithm allows negative weights but does not allow negative cycle.

Example:

Compute all pair shortest path for the following graph



Solution:

$$\begin{aligned}
 A^0 &= \begin{matrix} & \begin{matrix} 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 4 & 15 \\ 8 & 0 & 2 \\ 3 & \infty & 0 \end{bmatrix} \end{matrix} & A^1 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 4 & 15 \\ 8 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix} \\
 A^2 &= \begin{matrix} & \begin{matrix} 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 4 & 6 \\ 8 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix} & A^3 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 4 & 6 \\ 5 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}
 \end{aligned}$$

So, A^3 gives shortest distances between any pair of vertices.

Algorithm:

Algorithm all_pair(W,A)

```

{
For i=1 to n do
For j=1 to n do
A[i,j]=W[i,j]; //copy the weights as it is in matrix A
For k=1 to n do
{for i=1 to n do
{for j=1 to n do
{
A[i,j]=min(A[i,j],A[i,k]+A[k,j]);
}}}}
}

```

Analysis:

The basic operation to find the minimum distance between any pair of vertices lies within three nested loops so, $T(n)=O(n^3)$

4.4 0/1 KNAPSACK PROBLEM

Problem description:

If we are given n objects and a knapsack or a bag in which the object i that has weight w_i is to be placed. The knapsack has a capacity W . Then the profit that can be earned is $P_i X_i$. The objective is to obtain filling of knapsack with maximum profit earned.

Maximized $\sum_{i=1}^n p_i x_i$ subject to constraint $\sum_{i=1}^n w_i x_i \leq W$

Where $1 \leq i \leq n$ and n is total number of objects. And $X_i = 0$ or 1 .

Design and Analysis of Algorithm

The greedy method does not work for this problem.

To solve this problem using dynamic programming method we will perform following steps

Step 1:

the notations are used are

Let, $f_i(y_i)$ be the value of optimal solution. Then

S^i is a pair (p, w) where $p=f(y_i)$ and $w=y_i$

Initially $S^0 = \{(0, 0)\}$

We can compute S^{i+1} from S^i .

These computations of S^i are basically the sequence of decisions made for obtaining optimal solutions.

Step 2:

We can generate the sequence of decisions in order to obtain the optimum selection for solving the knapsack problem.

Let X_n be the optimum sequence. Then there are two instances $\{X_n\}$ and $\{X_{n-1}, X_{n-2}, \dots, X_n\}$. so from $\{X_{n-1}, X_{n-2}, \dots, X_n\}$. we will choose the optimum sequence with respect to X_n . The selection of sequence from remaining set should be such that we should be able to fulfill the condition of filling knapsack of capacity W with maximum profit .otherwise $X_n, \dots, X_1$ is not optimal.

This proves that 0/1 knapsack problem is solved using principle of optimality.

Step 3:

The formula that are used while solving 0/1 knapsack problem. Let, $f_i(y_i)$ be the value of optimal solution. Then,

$F_i(y) = \max(f_{i-1}(y), f_{i-1}(y - w_i) + p_i)$

Initially compute,

$S^0 = \{(0, 0)\}$

$S^i = \{(P, W) \mid (P - p_i, W - w_i) \in S^i\}$

S^{i+1} can be computed by merging S^i and S^{i-1} .

Purging rule:

If S^{i+1} contains (P_j, W_j) and (P_k, W_k) , these two pairs such that

$P_j \leq P_k$ and $W_j \geq W_k$, then (P_j, W_j) can be eliminated .This purging rule is also called as dominance rule. In purging rule basically the dominated tuples get purged. In short, remove the pair with less profit and more weight.

Example:

Solve Knapsack instance $M=6$ and $n=3$. let P_i and W_i are as shown below,

I	P_i	W_i
1	1	2
2	2	3
3	5	4

Solution: let us build the sequence of decision S^0, S^1, S^2, S^3
 $S^0 = \{(0, 0)\}$

$S^1 = \{(1, 2)\}$

That means while building S^0_1 we select the next i^{th} pair. For S^0_1 we have selected first (P, W) pair which

is(1,2).

Design and Analysis of Algorithm

Now,

$S^1 = [\text{Merge } S^0 \text{ and } S^0] = [(0,0), (1,2)]$

$S^1 = \{\text{select next}(P,W) \text{ pair and add it with } S^1\} = \{(2,3), (2+0, 3+0), (2+1, 3+1)\} = \{(2,3), (3,5)\}$

i.e repetition of (2,3) is avoided.

$S^2 = \{\text{Merge candidate from } S^1 \text{ and } S^1\} = \{(0,0), (1,2), (2,3), (3,5)\}$

$S^2 = \{\text{select next}(P,W) \text{ pair and add it with } S^2\} = \{(5,4), (6,6), (7,7), (8,9)\}$ Now, $S^3 = \{\text{Merge candidate from } S^2 \text{ and } S^2\} = \{(0,0), (1,2), (2,3), (5,4), (6,6), (7,7), (8,9)\}$

Note the pair (3,5) is purged from S^3 . this is because, let us assume $(P_j, W_j) = (3,5)$ and $(P_k, W_k) = (5,4)$ here $P_j \leq P_k$ and $W_j \geq W_k$ is true hence we will eliminate the pair (P_j, W_j) i.e (3,5) from S^3 .

As $M=6$ we will find tuple denoting weight 6. it is $(6,6) \in S^3$. Hence we will set $X_3=1$. now $(6 - P_3, 6 - W_3) = (6-5, 6-4) = (1,2)$. The pair (1,2) is present in S^2 but it is originated in S^1 .

Hence set $X_1=1, X_2=0, X_3=1$

i.e it is $\{1,0,1\}$. Analysis: the worst-case time complexity is $O(2^n)$

Introduction

Graph is a nonlinear data structure which is used in many applications. Graph is a basically a collection of nodes or vertices that are connected by links called edges.

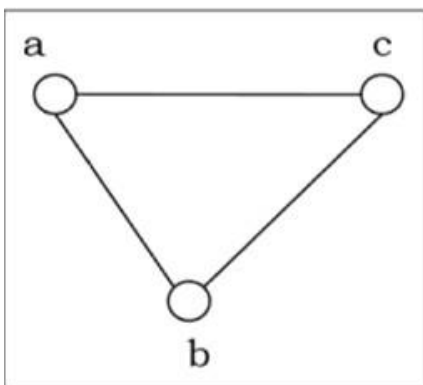
Definition of Graph:

A graph is a collection of two sets V and E where V is a finite non empty set of vertices and E is a finite non empty set of edges.

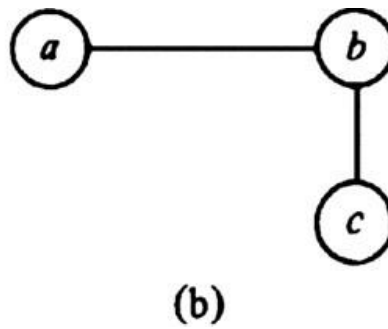
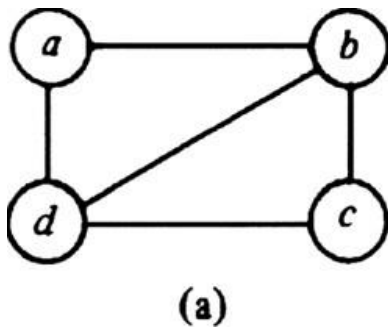
Vertices are nothing but the nodes in the graph and the two adjacent vertices are joined by edges. The graph is thus a set of two sets. Any Graph is denoted by $G=(V,E)$

Properties of Graph:

Complete Graph: If an undirected graph of n vertices consists of $n(n-1)/2$ number of edges then it is called as complete graph.



Subgraph: A subgraph G' of graph G is a graph such that the set of vertices and set of edges G' are proper subset of the set of edges of G .



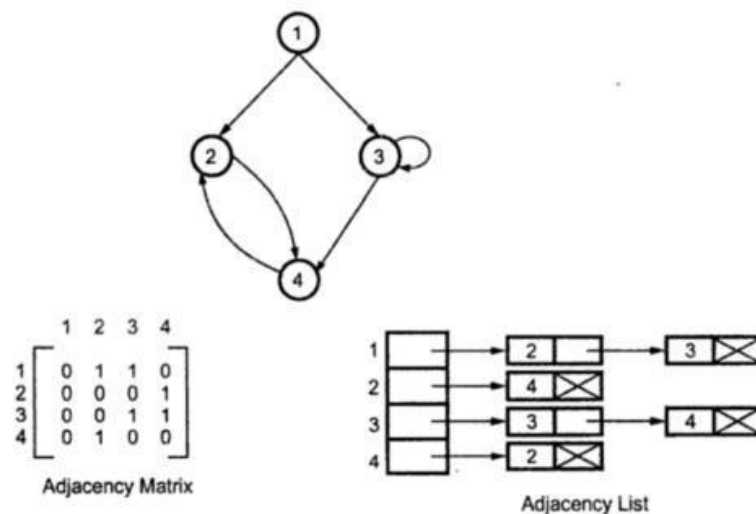
Connected Graph: An undirected graph is said to be connected if for every pair of distinct vertices V_i and V_j in $V(G)$ there is a path from V_i to V_j in G .

4.5 Representation of Graphs:

There are several representations of graphs, but we will discuss the two commonly used representation called Adjacency matrix and adjacency list

4.5.1 Adjacency matrix:

Consider a graph G of n vertices and the matrix M . If there is an edge present between vertices V_i and V_j then $M[i][j]=1$ else $M[i][j]=0$. Note that for an undirected graph if $M[i][j]=1$ then for $M[j][i]$ is also 1.



4.5.2 Adjacency list:

Here the graph is created with linked list is called adjacency list. so all the advantage of linked list can be obtained in this type of graph. We need not have a prior knowledge of maximum number of nodes.

Application of Graph:

- ◆ Graphs are used in the design of communication and transportation networks, VLSI and other sorts of logic circuits
- ◆ Graphs are used for shape description in computer-aided design and Geographic information systems, precedence
- ◆ Graph is used in scheduling systems.

4.6 Graph Traversals

Traversing a graph means searching the remaining nodes or vertices of graph from the given vertex. There are two commonly used techniques for traversing the graph.

1. Breadth first search
2. Depth first search

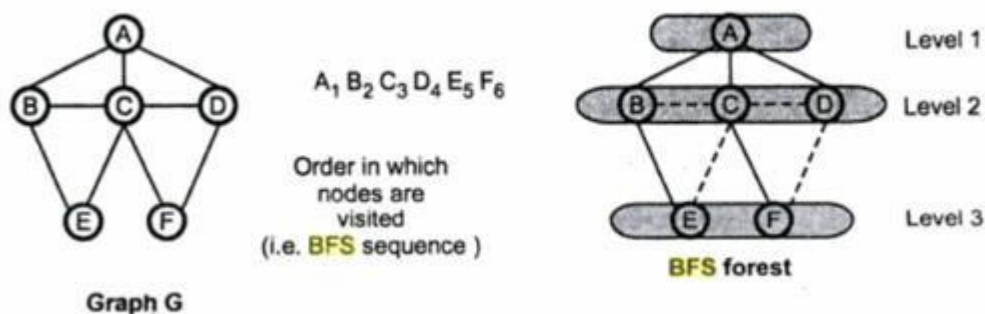
4.6.1 Breadth First search (BFS)

Some terminologies in BFS formally

Breadth first forest:

The breadth first forest is a collection of trees in which the traversal's starting vertex serves as the root of the first tree in such a forest. Whenever a new unvisited vertex is visited then it is attached as a child to the vertex from which it is being reached.

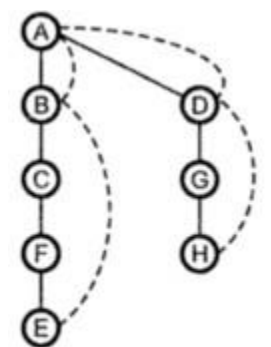
Consider the following graph



- ◆ For finding the connected components in the graph.
- ◆ For checking if any cycle exists in the given graph
- ◆ To obtain shortest path between two vertices.

4.6.2 Depth First Search (DFS)

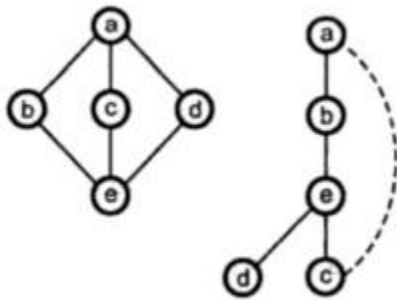
The depth first is a collection of trees in which traversals starting vertex serves as the root of the first tree in such a forest. Whenever a new unvisited vertex is visited then it is attached as a child to the vertex from which it is being reached.



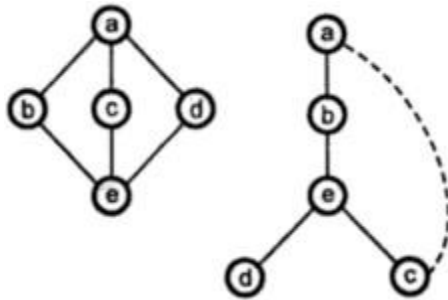
Design and Analysis of Algorithm

DFS is used for checking connectivity of a graph.

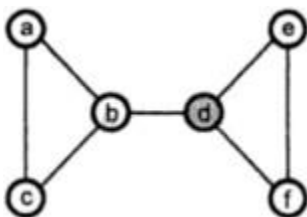
Start traversing the graph using depth first method and after an algorithm halt if all the vertices of graph are visited then the graph is said to be a connected graph



DFS is used for checking acyclicity of graph. If the DFS forest does not have back edge then the graph is said to be acyclic.



This graph contains cycle, because it contains back edge DFS is used to find articulation point



A vertex of connected graph is said to be its articulation point If its removal with all its incident edges breaks the graph into disjoint pieces.

The vertex d is an articulation point

CHAPTER 5: BACKTRACKING

It is a most general method. Backtracking algorithm determines the solution by systematically searching the solution space (i.e. set of all feasible solutions) for the given problem.

- It is a depth first search with some bounding functions. All solutions using backtracking are required to satisfy a complex set of constraints, the constraints may be explicit or implicit.
- Explicit constraints are rules, which restrict each vector element to be chosen from the given set.
- Implicit constraints are rules, which determine which of the tuples in the solution space, actually satisfy the criteria function.

The steps are

- The desired solution is expressible as an tuple $(X_1, x_2, \dots, x_n)$ where X_i is chosen from some finite set S_i ,
- The solution maximizes or minimizes or satisfies a criterion function $C(x_1, x_2, \dots, x_n)$.
- The problem can be categorized into three categories:
 - For instance- for a problem P let C be the set of constraints for P . let D be the set containing all solutions satisfying C then
 - Finding whether there is any feasible solution? - is the desired problem
 - What is the best solution- is the optimization problem
 - Listing of all feasible solution- is the enumeration procedure.

Terminologies used in back tracking:

Backtracking algorithms determine problem solutions by systematically searching for the solutions using tree structure.

- Each node in the tree is called a **problem state**.
- All paths from the root to other nodes define the state space of the problem.
- The **solution states** are those problem states for which the path from root to s defines a tuple in the solution space.
- In some trees, the leaves define the solution space.
- **Answer states**: these are the leaf nodes which correspond to n element in the set of solutions, these are the states which satisfy the implicit constraints.

Design and Analysis of Algorithm

- A node which is been generated and all whose children have not yet been generated is called **live node**.
- The live node whose children are currently being expanded is called E-node.
- A **dead node** is generated node which is not to be expanded further or all of whose children have been generated.

Algorithm:

Alg backtrack()

//this is a recursive backtracking algorithm.

//a[k] is a solution vector.on entering (k-1) remaining next values can be compared.

//T(a1,a2,...,an)be the set of all values for a(i+1) such that(a1,a2,...,ai+1)is a path to problemstate.

//B_(i+1) is a bounding function such that if B_{i+1}(a1,a2,...ai+1) is false.

//for a path (a1,a2,...an) from root node to a problem state then

//path can not be extended to reach an answer node.

{for(each a_k that belongs to T(a1,a2,...a(k-1)))do

{if B_k(a1,a2,...ak)=true) then// feasible sequence

{

If((a1,a2,...ak)is a path to answer node then

Print(a[1],a[2],....a[k]);

If(k<n)then Backtrack(k+1);//find the next set

}}}

Applications of backtracking:

- n-queen problem,
- Graph coloring problem,
- Finding Hamiltonian cycle.

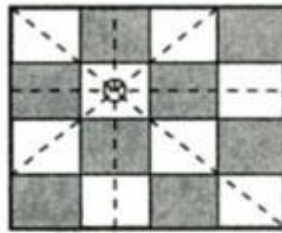
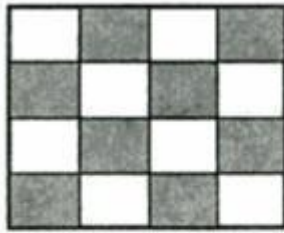
5.1 N-Queen's Problem

This is a problem based on chess games. By this problem, it is stated that arrange the queen on chessboard in such a way that no two queens can attack each other.

The n-queens problem can be stated as follows,

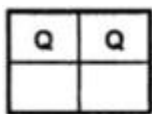
Consider a $n * n$ chessboard on which we have to place n queens so that no two queens attack each other by being in the same row or in the same column or on the same diagonal.

Consider $4 * 4$ board

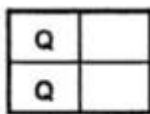


The next queen - if is placed on the paths marked by dotted lines then they can attack each other

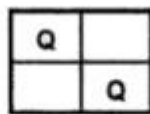
2-Queen's problem is not solvable- because 2 queens can be placed on 2*2 chessboard as



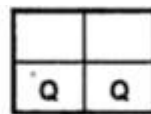
Illegal



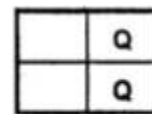
Illegal



Illegal

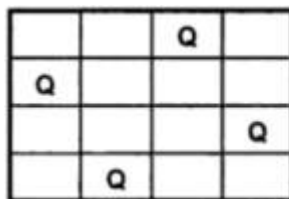


Illegal



Illegal

But 4- queen's problem is solvable

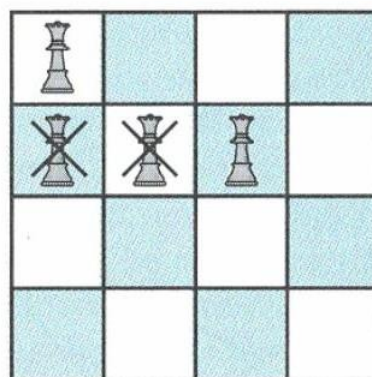
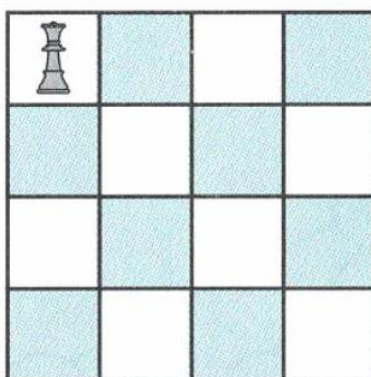


←

Note that no two queens can attack each other.

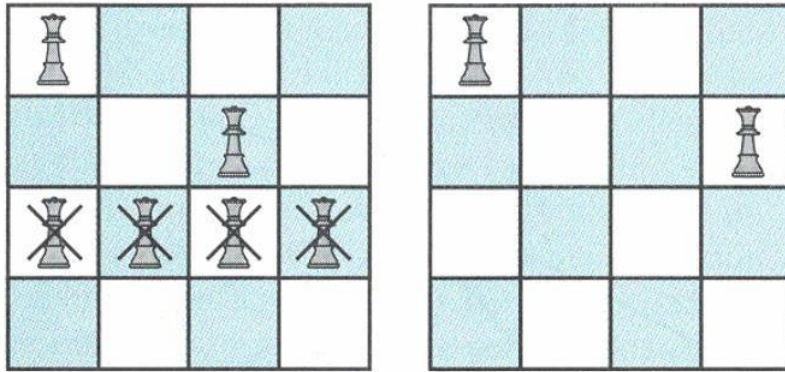
How to solve n-queens problem:

Let us take 4 queens and 4*4 chess board Now we start with empty chessboard Place queen1 in the first possible position of its row.i.e on 1st row and 1st column. Then place queen 2 after trying unsuccessful place-1(1,2),(2,1),(2,2) at (2,3)

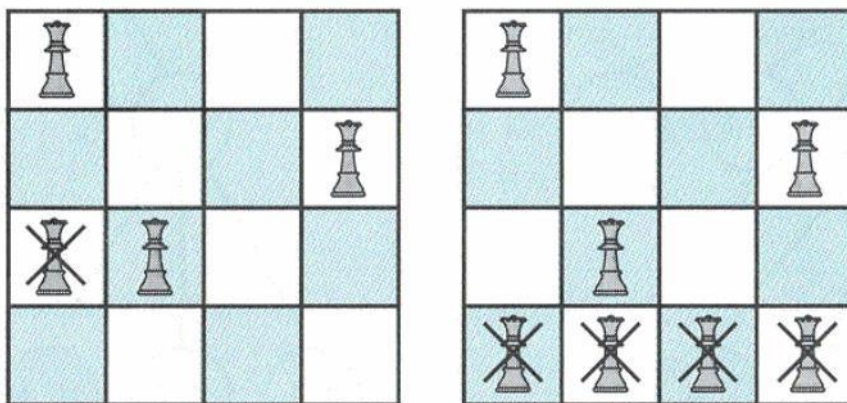


This is the dead end because a 3rd queen cannot be placed in next column as there is no acceptable position for queen 3. hence algorithm backtracks and places the 2nd queen at (2,4) position.

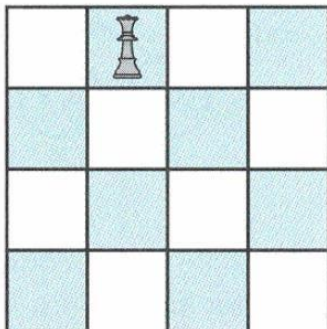
Design and Analysis of Algorithm



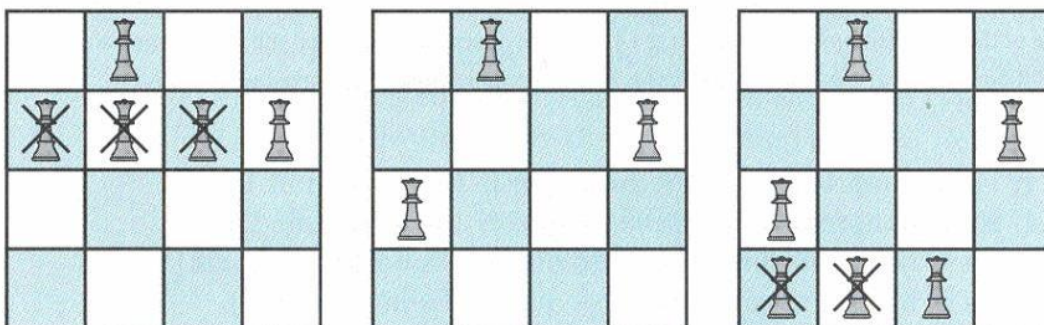
The place 3rd queen at(3,2) but it is again another dead end as next queen (4th queen) cannot be placed at permissible position.



Hence we need to backtrack all the way up to queen 1 and move it to (1,2)



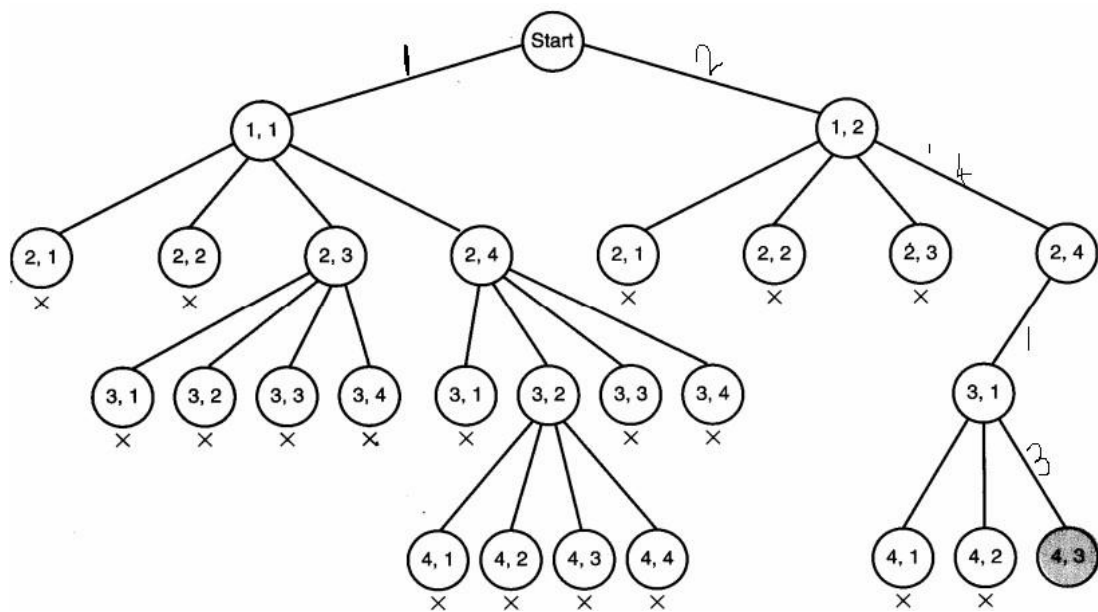
Place queen 1 at (1,2), queen 2 at(2,4), queen 3 at(3,1) and queen 4 at(4,3).



Thus, the solution is obtained.(2,4,1,3) in row wise manner.

Design and Analysis of Algorithm

The state space tree of 4-queens problem,



Now we will consider how to place 8-queens on the chessboard for a feasible sequence.

(6,4,7,1)

Now I will start placing the queens on the chessboard.

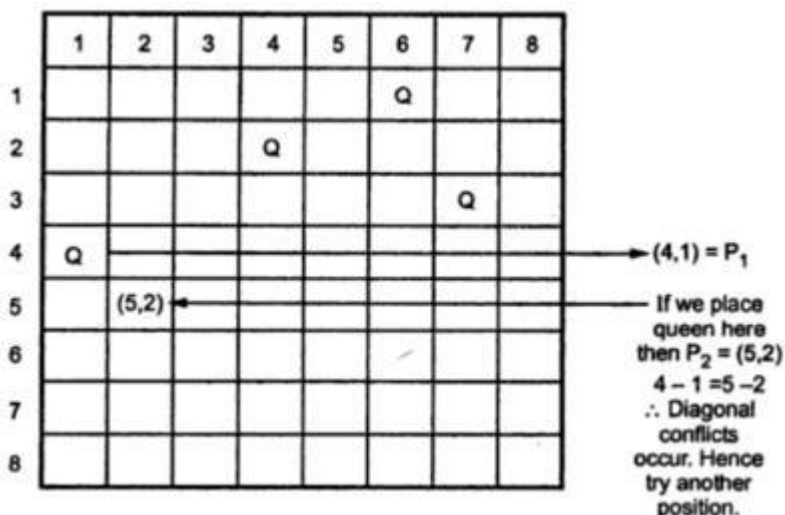
1	2	3	4	5	6	7	8
					Q1		
			Q2				
						Q3	
Q4							

The diagonal conflicts can be checked by following formula:

Let $p1=(i,j)$ and $p2=(k,l)$ are two positions. Then $p1$ and $p2$ are the positions that are on the same diagonal .if

$i+j = K+1$ or $i-j = K-1$.

Design and Analysis of Algorithm



Now, we have to place 5 queens in such way that no two queens can attack each other, so, if Q5 is at (5,2) then Q4 will attack, if Q5 is at (5,3) then no queen will attack it.

					Q1		
			Q2				
						Q3	
Q4							
		Q5					

now we want to place Q6 at location (6,6) then Q1 will attack.

if Q6 is at (6,7) then Q3 will attack, if (6,8) then Q2 will attack., if (6,4) then Q2 will attack. if (6,3) then Q5 will attack, if (6,2) then Q5 will attack, if (6,1) then Q4 will attack, so if Q6 at (6,5) then no one will attack.

					Q1		
			Q2				
						Q3	
Q4							
		Q5					
				Q6			

Design and Analysis of Algorithm

	Q7*						Q7*
							Q8

now we want to place Q7 at location (7,7),(7,6),(7,5),(7,4),(7,3),(7,1) then Q3

,Q1,Q6,Q2,Q5,Q4 will attack.

if it is at (7,8),(7,2) then no queen will attack,now. We want to place Q8 at location (8,8) then Q7 will attack.

If it's (8,1) then Q4 will attack, If its (8,2) then Q3 will attack, If its (8,3) then Q5 will attack, If its (8,4) then Q2 will attack, If its (8,5) then Q6 will attack, If its (8,6) then Q1 will attack, If its (8,7) then Q3 will attack.

So now we need to backtrack and change previously placed queens so, I have placed Q7 at (7,2)

It can be summarized below.

Queen Positions								Action
1	2	3	4	5	6	7	8	
6	4	7	1					Start
6	4	7	1	2				As $4 - 1 = 5 - 2$ conflict occurs.
6	4	7	1	3				$5 - 3 \neq 4 - 1$ or $5 + 3 \neq 4 + 1 \therefore$ feasible
6	4	7	1	3	2			As $5 + 3 = 6 + 2$. It is not feasible.
6	4	7	1	3	5			Feasible
6	4	7	1	3	5	2		Feasible
6	4	7	1	3	5	2	8	List ends and we have got feasible sequence.

					Q1		
			Q2				
						Q3	
Q4							
		Q5					
				Q6			
	Q7						
							Q8

So 8-queens with feasible solutions (6,4,7,1,3,5,2,8).

5.2 Sum of Subsets

Problem statement:

Let $S = \{S_1, \dots, S_n\}$ be a set of n positive integers, then we have to find a subset whose sum is equal to given positive integer d .

It is always convenient to sort the sets elements in ascending order, that is

$$S_1 \leq S_2 \leq \dots \leq S_n$$

Let us write a general algorithm for sum of subset problem.

Algorithm: Let S be asset of elements and d is expected sum of subsets.then,

Step 1: Start with an Empty set.

Step 2: add to the subset, the next element from the list.

Step 3: if the subset is having sum d then stop with that subset as solution.

Step 4: if the subset is not feasible or if we have reached the end of the set then backtrack through the subset until we find the most suitable value.

Step 5: if the subset is feasible then repeat step 2

Step 6: If we have visited all the elements without finding a suitable subset and if no backtracking is possible then stop without solution.

This problem can be well understood with some examples.

Example: Consider a set $S = \{5, 10, 12, 13, 15, 18\}$ and $d = 30$. Solve it for obtaining a sum of subset.

Initially subset={ }	Sum	---
5	5	Then add next element
5,10	15 i.e $15 < 30$	add next element
5,10,12	27 i.e $27 < 30$	add next element
5,10,12,13	40	Sum exceeds $d=30$ hence backtrack ---- Not feasible
5,10,12,15	42	Sum exceeds $d=30$ hence backtrack--- Not feasible
5,10,12,18	45	Sum exceeds $d=30$ hence backtrack. Not feasible
5,10		
5,10,13	28	

5,10,13,15	33	Sum exceeds d=30 hence backtrack ---- Not feasible
5,10		
5,10,15	30	Solution obtained as sum =3=d

The state space tree can be drawn as follows

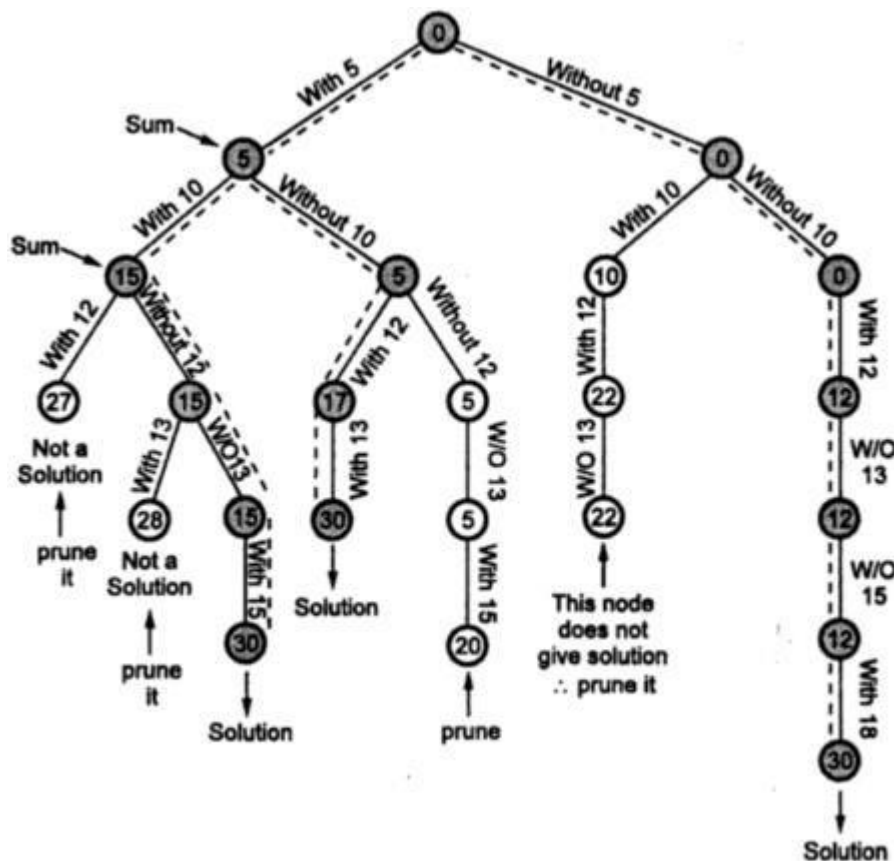


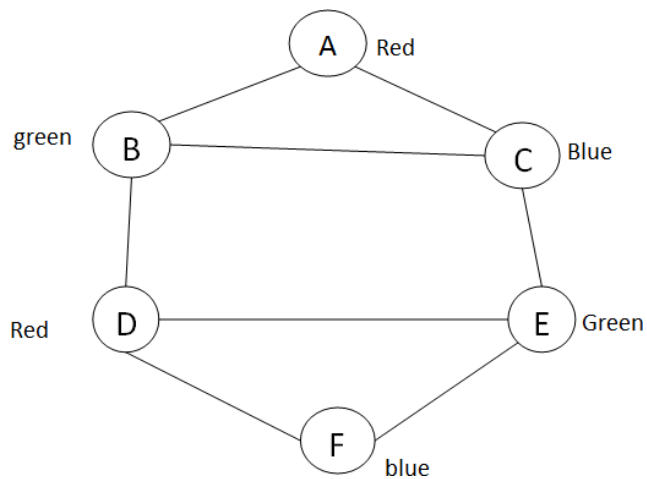
Fig. 5.5 State space tree for sum of subset

5.3 Graph coloring

Graph coloring is a problem of coloring each vertex in graph such a way that no two adjacent vertices have same color. Vertices have same color and yet m-colors are used. This problem is also called as m-coloring problem. If the degree of given graph is d then we can color it with d+1 colors. The least number of colors needed to color the graph is called its chromatic number.

Example:

Consider a graph given below; we require three colors to color the graph. Hence the chromatic number of the given graph is 3. we can use backtracking technique to solve the graph coloring problem as follows:

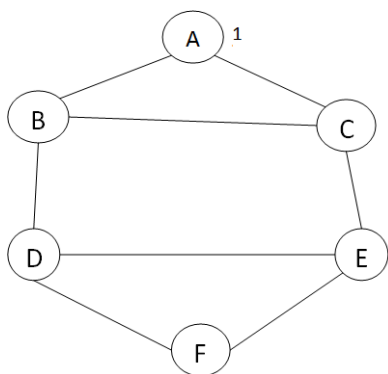


Step 1:

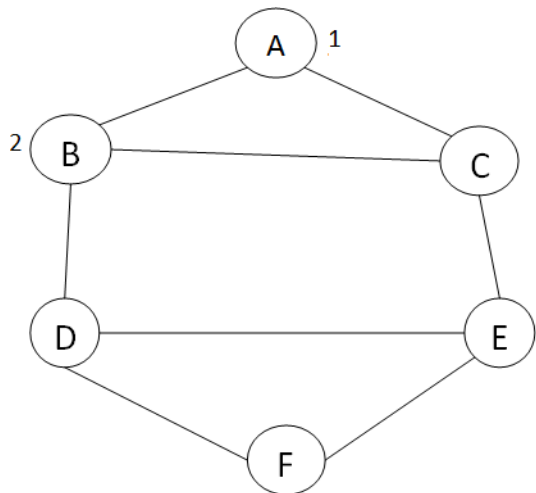
A graph G consists of vertices from A to F. There are three colors used Red, Green, Blue. we will number them out.

That means 1 indicate Red, 2 indicate Green, 3 indicate Blue color.

Step 2:

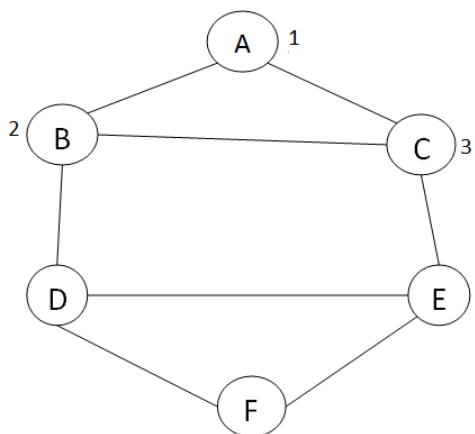


Let us consider A have red. According to the problem, no adjacent vertices have same color. so, B & C should not have red color.



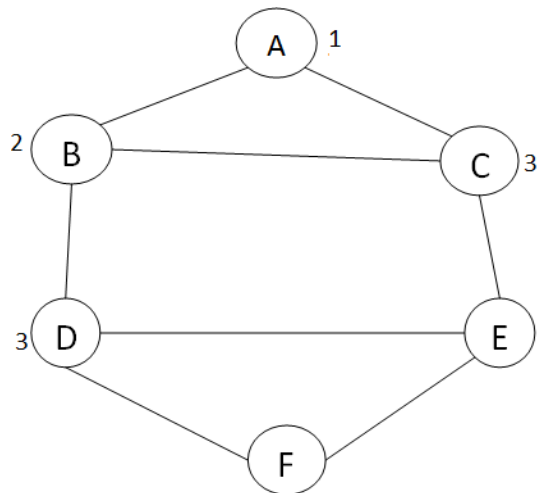
So, I choose B have green. According to the problem, no adjacent vertices have same color. so, A, D, C should not have green color.

Since C should not have green and red color. so, C have blue color, according to the problem, no adjacent vertices have same color. so, B, A & E should not have blue color.

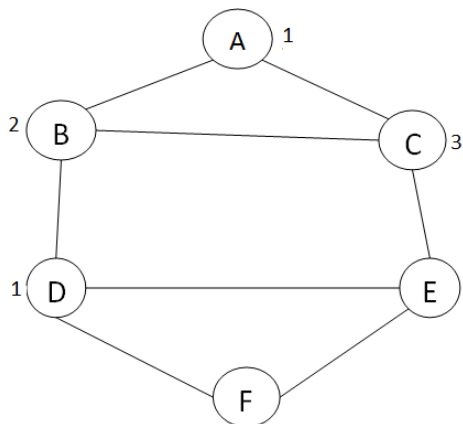


Now D have to choose either blue or red.

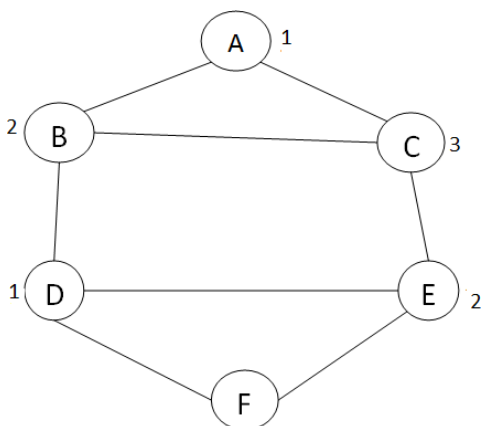
If it chooses blue, then E, F, B should not have blue.



Since E should not have blue color, so it has to choose either red or green. If E choose red, then D, E, F should not have red. but here C, D whose are adjacent vertices of E have same color. So, need to do backtracking and rearrange all places.
if D choose red, then E, F, B should not have red color.

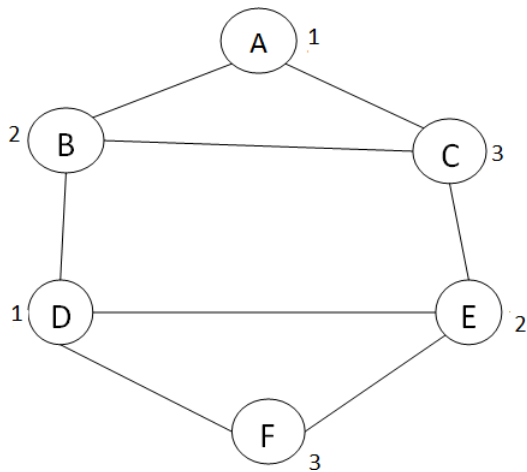


Since E should not have blue or red, so E choose green color's and D should not have green color,

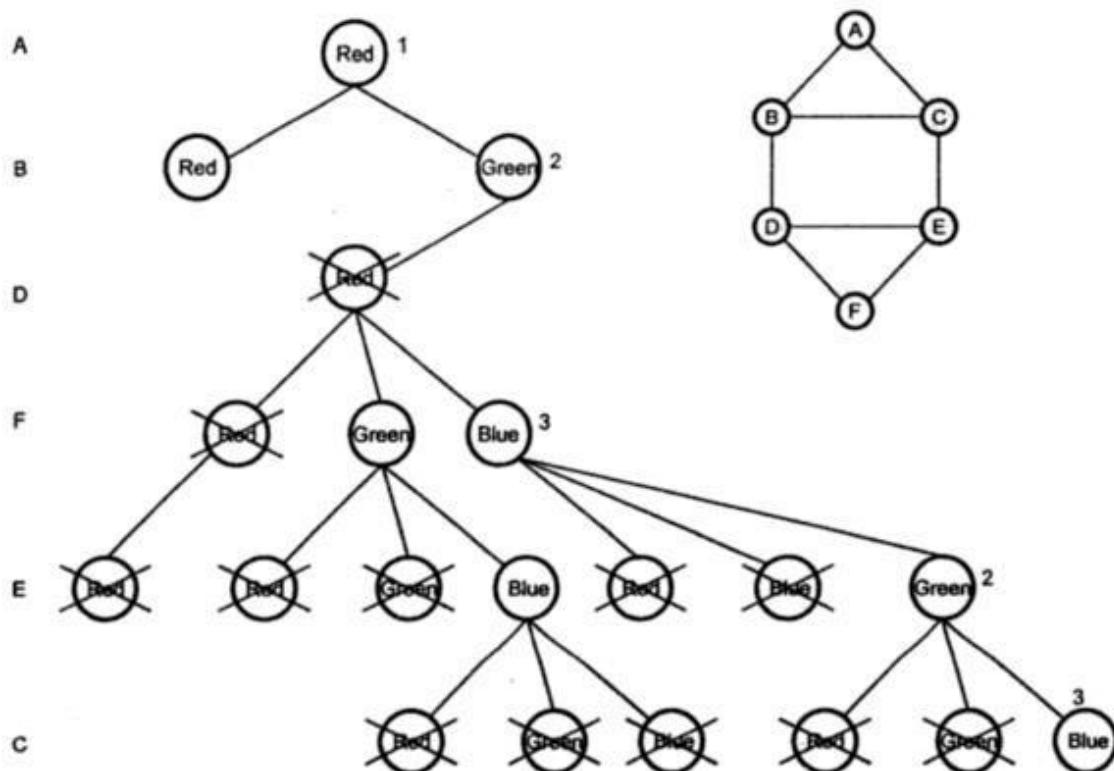


Design and Analysis of Algorithm

Since F should not have red and green so it has to choose blue color. Step 3:



Thus the graph colouring method is solved. the graph is



CHAPTER 6: Introduction to Probabilistic Algorithms

To analyze the cost of algorithms it is important to have a concrete model with a well-defined notion of costs. Sequentially the **Random Access Machine** (RAM) model has served well for many years. The RAM is meant to approximate how real sequential machines work. It consists of a **single processor** with some constant number of registers, an instruction counter and an arbitrarily large memory. The **instructions** include register-to-register instructions (e.g. adding the contents of two registers and putting the result in a third), control-instructions (e.g. jumping), and the ability to read from and write to arbitrary locations in memory. For the purpose of analyzing cost, the RAM model assumes that all instructions take unit time. The total “time” of a computation is then just the number of instructions it performs from the start until a final end instruction. To allow storing a pointer to memory in a register or memory location, but disallow playing games by storing arbitrary large values, we assume that for an input of size n each register and memory location can store $\Theta(\log n)$ bits. The RAM model is by no stretch meant to model runtimes on a real machine with cycle-by-cycle level accuracy. It does not model, for example, that modern-day machines have cache hierarchies and therefore not all memory accesses are equally expensive. Modeling all features of modern-day machines would lead to very complicated models that would be hard to use and hard to gain intuition from. Although a RAM does not precisely model the performance of real machines, it can, and has, effectively served to compare different algorithms, and understand how the performance of the algorithms will scale with size. For these reasons the RAM should really only be used for **asymptotic** (i.e. big-O) analysis. Beyond serving as a cost model for algorithms that is useful for comparisons and asymptotic analysis, the RAM has some other nice features: it is simple, and, importantly, code in most high-level languages can be naturally translated into the model. In the context of parallel algorithms we would like to use a cost model that satisfies a similar set of features. Here we use one, the MP-RAM, that we find convenient and seems to satisfy the features. It is based on the RAM, but allows the dynamic forking of new processes. It measures costs in terms of two quantities: the work, which is the total number of instructions across all processes, and the depth, which is the longest chain of sequential dependences. It may not be obvious how to map these dynamic processes onto a physical machine which will only have a fixed number of processors. To convince ourselves that it is possible, later we show how to design schedulers that map the processes onto processors, and prove bounds that relate costs. In particular 2 we show various forms of the following work-depth, processor-time relationship: $\max \{W/P, D\} \leq T \leq W/P + D$ where W is the work, D the depth, P the processors, and T the time. MP-RAM The Multi-Process Random-Access Machine (MP-RAM) consists of a set of processes that share an

Design and Analysis of Algorithm

unbounded memory. Each process runs the instructions of a RAM—it works on a program stored in memory, has its own program counter, a constant number of its own registers, and runs standard RAM instructions. The MP-RAM extends the RAM with a fork instruction that takes a positive integer k and forks k new child processes. Each child process receives a unique integer in the range $[1, \dots, k]$ in its first register and otherwise has the identical state as the parent (forking process), which has that register set to 0. All children start by running the next instruction, and the parent suspends until all the children terminate (execute an end instruction). The first instruction of the parent after all children terminate is called the join instruction. A computation starts with a single root process and finishes when that root process ends. This model supports nested parallelism—the ability to fork processes in a nested fashion. If the root process never does a fork, it is a standard sequential program. A computation in the MP-RAM defines a partial order on the instructions. In particular (1) every instruction depends on its previous instruction in the same thread (if any), (2) every first instruction in a process depends on the fork instruction of the parent that generated it, and (3) every join instruction depends on the end instruction of all child processes of the corresponding fork generated. These dependences define the partial order. The work of a computation is the total number of instructions, and the depth is the longest sequences of dependent instructions. As usual, the partial order can be viewed as a DAG. For a fork of a set of child processes and corresponding join the depth of the sub computation is the maximum of the depth of the child processes, and the work is the sum. This property is useful for analyzing algorithms, and specifically for writing recurrences for determining work and depth. We assume that the results of memory operations are consistent with some total order (linearization) on the instructions that preserves the partial order—i.e., a read will return the value of the previous write to the same location in the total order. The choice of total order can affect the results of a program since processes can communicate through the shared memory. In general, therefore computations can be nondeterministic. Two instructions are said to be concurrent if they are *unordered*, and *ordered* otherwise. Two instructions conflict if one writes to a memory location that the other reads or writes the same location. We say two instructions race if they are concurrent and conflict. If there are no races in a computation, then all linearized orders will return the same result. This is because all pairs of conflicting instructions are ordered by the partial order (otherwise it would be a race) and hence must appear in the same relative order in all linearization. A particular linearized order is to iterate sequentially from the first to last child in each fork. We call this the ***sequential ordering***.

A randomized algorithm is one that uses **random numbers** or bits during the runtime of the algorithm. Such algorithms, when properly designed, can ensure a correct solution on every input with high probability. For many problems, randomized algorithms have been designed that are simpler or more efficient than the best deterministic algorithms known for the problems. In this chapter, we define a natural randomized parallel complexity class, RNC, and give a survey of randomized algorithms for problems in this class.

Design and Analysis of Algorithm

In recent years, there has been an explosive growth in the availability and diversity of parallel processors for computation. For the purpose of parallel algorithm design, it is convenient to work with an abstract, simplified machine model, known as the **Parallel Random Access Machine** (PRAM). The PRAM incorporates the basic elements of a parallel machine and has the property that an algorithm designed for it will perform without significant degradation on most commonly available parallel machines, including shared-memory multiprocessors and fixed interconnection networks.

The Randomized Parallel Complexity Class RNC

In the design and analysis of highly parallel algorithms for various problems, it has been observed that some problems have simple highly parallel algorithms using a relatively small number of processors whereas others have resisted all attempts so far to obtain any algorithm with a significant amount of parallelism. In this context, researchers have identified a natural parallel complexity class NC. The class NC consists of those problems that have PRAM algorithms that run in polylog time, i.e., time polynomial in the logarithm of the input size, with a polynomial number of processors. This class is robust in the sense that the collection of problems it contains does not vary with the machine model used, whether it is a shared-memory machine or a low-diameter interconnection network. It is also the smallest nontrivial class with this property. Although several important problems have been shown to be in NC, many others have not. The latter problems are of two types (for our purposes): one type consists of those problems that are either provably not in NC by virtue of the fact that they are probably not in P (polynomial time), or are highly unlikely to be in NC by virtue of the fact that they are complete problems for a larger class (such as P); for these problems, we currently know of no technique, randomized or otherwise, to come up with algorithms that run in poly log time with a polynomial number of processors. The other type of problems are those for which no one has come up with either an algorithm to place the problem in NC or a completeness result to make it highly unlikely to be placed in NC. This latter type includes several important problems such as finding a maximum matching or a depth-first search tree (an important type of spanning tree) in a graph. For these problems, randomization has proved to be a valuable tool in coming up with fast parallel algorithms. For more on parallel complexity classes, see Cook (1981, 1985) and Karp and Ramachandran (1990).

The class RNC is the class of problems that can be solved by a Monte Carlo randomized algorithm that runs in polylog time with a polynomial number of processors. Thus, RNC is the randomized counterpart of NC.

REFERENCES

1. Manna, Z. (1974) Mathematical Theory of Computation McGraw-Hill.
2. Baase, S. (1988) Computer Algorithms: Introduction to Design and Analysis, 2nd ed.
3. T. H. Cormen, C. E. Leiserson, R. L. Rivest. Introduction to Algorithms The MIT Press, Cambridge,Massachusetts, 3rd edition
4. <https://nap.nationalacademies.org/read/2026/chapter/11#152>
5. https://www.tutorialspoint.com/parallel_algorithm/parallel_algorithm_models.htm

Design and Analysis of Algorithm

Summary Questions

1. A binary tree have_____ number of children?
A. 0 or 1
B. 2
C. 0 or 1 or 2
D. None
 2. What should be considered when designing an algorithm?
A. If the hardware is used correctly
B. If this software is used correctly
C. If there is more than one way to solve the problem
D. All of the above mentioned
- Explanation:** when designing an algorithm we must check if there is more than one way to solve the problem
3. Which of the following is the disadvantage of implementing tree using normal array?
A. Difficult in knowing children nodes of node
B. Have to know the maximum number of nodes before creation
C. Difficulty in finding the parent of a node
D. Difficulty in finding the child nodes
 4. Which of the following is the application of circular linked list?
A. Compiler design
B. Iterative functions
C. Recursive functions
D. CPU resource allocation
 5. Which of the following is not true about circular linked list?
A. Time complexity of deleting the last node is $O(n)$
B. Time complexity of searching an element is $O(n)$
C. Time complexity of inserting a new node at the head of the list is $O(1)$
D. It is possible to traverse the whole list starting from any stating from any point
 6. Which of the following operations take $O(1)$ time complexity
A. Adding two numbers
B. Displaying some integer on console
C. Assigning some value to a variable\
D. All
E. None
 7. How many passes are required to sort a file of size n by bubble sort method?
A. $n-1$
B. $n-2$
C. n^2
D. $n/2$
 8. Which of the following sorting algorithm is slowest?
A. Bubble sort
B. Heap sort
C. Shell sort
D. Quick sort
 9. Which of the following sorting algorithm is efficient of the list is already sorted?
A. Bubble sort
B. Heap sort
C. Insertion sort
D. None

Design and Analysis of Algorithm

10. Which of the following is not true about comparison-based sorting algorithms?

- A. Any comparison based sorting algorithm can be made stable by using position as a criteria
- B. Counting sort is not a comparison based sorting algorithm
- C. Heap sort is not a comparison based sorting algorithm
- D. None

11. What is the time complexity of the function below?

```
int fun(int n)
{
    int count = 0;
    for (int i = n; i > 0; i /= 2)
        for (int j = 0; j < i; j++)
            count += 1;
    return count;
}
```

- A. $O(n)$
- B. $O(n^2)$
- C. $O(\log n)$
- D. $O(n \log n)$

Explanation: For an input integer n , the innermost statement of $\text{fun}()$ is executed following times. $n + n/2 + n/4 + \dots 1$

So time complexity $T(n)$ can be written as $T(n) = O(n + n/2 + n/4 + \dots 1) = O(n)$

The value of count is also $n + n/2 + n/4 + \dots + 1$

12. What is the time complexity of the function below?

```
int fun(int n)
{
    int count = 0;
    for (int i = 0; i < n; i++)
        for (int j = i; j > 0; j--)
            count = count + 1;
    return count;
}
```

- A. $\Theta(n)$
- B. $\Theta(n^2)$
- C. $\Theta(n^3)$
- D. None

The time complexity can be calculated by counting the number of times the expression "count = count + 1;" is executed. The expression is executed $0 + 1 + 2 + 3 + 4 + \dots + (n-1)$ times.

Time complexity = $\Theta(0 + 1 + 2 + 3 + \dots + n-1) = \Theta(n(n-1)/2) = \Theta(n^2)$

13. Which of the following is the correct relation for tower of Hanoi?

- A. $T(n) = 2T(n-1) + 1$
- B. $T(n) = 2T(n-2) + 2$
- C. $T(n) = 2T(n/2) + 1$
- D. $T(n) = 2T(n-n) + n$

Explanation: Let the three pegs be A, B and C. The goal is to move n pegs from A to C.

Design and Analysis of Algorithm

To move n discs from peg A to peg C:

move $n-1$ discs from A to B. This leaves disc n alone on peg A

move disc n from A to C

move $n-1$ discs from B to C so they sit on disc n

The recurrence function $T(n)$ for the time complexity of the above recursive solution can be written as follows.

$$T(n) = 2T(n-1) + 1$$

14. Let $F(n)$ and $G(n)$ denote respectively, the worst case and average case running time of an algorithm executed on an input of size n . which of the following is always true?

A. $G(n) = \Omega(F(n))$

C. $G(n) = o(F(n))$

B. $G(n) = \Theta(F(n))$

D. $G(n) = O(F(n))$

The worst case time complexity is always greater than or same as the average case time complexity.

15. Which of the given options provides the increasing order of asymptotic complexity of functions $A1$, $A2$, $A3$, and $A4$?

$$A1(n) = 2^n$$

$$A2(n) = n(6/4)$$

$$A3(n) = n \cdot \log(n)$$

$$A4(n) = n^{\log(n)}$$

A. $A3-A2-A1-A4$

C. $A2-A3-A4-A1$

B. $A3-A2-A4-A1$

D. $A2-A3-A1-A4$

16. What does it mean when we say that an algorithm X is asymptotically more efficient than Y ?

A. X will be better choice for all inputs except possibly for small input

B. Y will be better choice for all inputs except possibly for small input

C. X will be better choice for all inputs

D. Y will be better choice for all inputs

Explanation: This means that for large inputs, X will always be a better choice than Y in terms of efficiency. However, for small inputs, it is possible that Y may still be faster than X . Therefore, we cannot say that X will always be a better choice for all inputs, but we can say that X will be a better choice for all inputs except possibly small inputs.

17. Which of the following is correct about the time complexity for all pairs shortest path problem?

A. $O(n^2)$

C. $O(2^n)$

B. $O(n^3)$

D. $O(n)$

Design and Analysis of Algorithm

Explanation: Because of three nested loops.

18. Which of the following sorting algorithm has the smallest worst case time complexity?

- A. Merge sort
- B. Quick sort
- C. Selection sort
- D. Bubble sort

19. Two matrices M1 and M2 are to be stored in arrays A and B respectively. Each array can be stored either in row-major or column-major order in contiguous memory locations. The time complexity of an algorithm to compute $M1 \times M2$ will be best

- A. If both are in row major order
- B. If both are in column major order
- C. Independent of storage scheme
- D. None

Explanation: This is a trick question. Note that the questions ask about time complexity, not time taken by the program. For time complexity, it doesn't matter how we store array elements, we always need to access same number of elements of M1 and M2 to multiply the matrices. It is always constant or $O(1)$ time to do element access in arrays, the constants may differ for different schemes, but not the time complexity.

20. Randomized quicksort is an extension of quicksort where the pivot is chosen randomly. What is the worst case complexity of sorting n numbers using randomized quicksort?

- A. $O(n^2)$
- B. $O(n)$
- C. $O(n!)$
- D. None

Explanation: If all elements of the given array are same then that is the worst case for the randomized quicksort. And time complexity of worst case of quicksort is $O(n^2)$ that is proven already.

21. Let $T(n)$ be a function defined by the recurrence $T(n) = 2T(n/2) + \sqrt{n}$ for $n \geq 2$ and $T(1) = 1$

Which of the following statements is TRUE?

- A. $T(n) = \theta(n)$
- B. $T(n) = \theta(n \log n)$
- C. $T(n) = \theta(\sqrt{n})$
- D. $T(n) = \theta(\log n)$

$n^{(\log_b a)} = n$ which is $= n^{(1.5)} = O(\sqrt{n})$ then by applying case 1 of master method we get $T(n) = \Theta(n)$

22. The worst case running times of Insertion sort, Merge sort and Quick sort, respectively, are:

- A. $\Theta(n \log n)$, $\Theta(n \log n)$ and $\Theta(n^2)$
- B. $\Theta(n^2)$, $\Theta(n \log n)$ and $\Theta(n^2)$
- C. $\Theta(n^2)$, $\Theta(\log n)$ and $\Theta(n^2)$
- D. $\Theta(n \log n)$, $\Theta(n \log n)$ and $\Theta(n^2)$

Explanation: Insertion sort takes $\Theta(n^2)$ in worst case as we need to run two loops. The outer loop is needed to one by one pick an element to be inserted at right position. Inner loop is used for two things, to find position of the element to be inserted and moving all sorted greater elements one position ahead. Therefore the worst case recursive formula is $T(n) = T(n-1) + \Theta(n)$.

Design and Analysis of Algorithm

Merge sort takes $\Theta(n \log n)$ time in all cases. We always divide array in two halves, sort the two halves and merge them. The recursive formula is $T(n) = 2T(n/2) + \Theta(n)$.

Quick sort takes $\Theta(n^2)$ in worst case. In Quick Sort, we take an element as pivot and partition the array around it. In worst case, the picked element is always a corner element and recursive formula becomes $T(n) = T(n-1) + \Theta(n)$. An example scenario when worst case happens is, arrays is sorted and our code always picks a corner element as pivot.

23. Which of the following complexity is true if the input is already sorted?

- A. Bubble sort runs in $\Theta(n^2)$ time
- B. Merge sort runs in $\Theta(n)$ time
- C. Insertion sort runs in $\Theta(n)$ time
- D. All of the above mentioned

Explanation:

Input is already in ascending order, which means it is the worst-case situation.

- ✓ Quicksort runs in $\Theta(n^2)$ time In quick sort worst case if the first or last element is selected at the pivot element.
For a Quicksort, in the worst-case recurrence, the relation will become $T(n) = T(n-1) + T(1) + n$, Which gives $T(n) = \Theta(n^2)$ So, it is correct.
- ✓ Bubble sort runs in $\Theta(n^2)$ time
If an array is already sorted in ascending order, then at that time there will be no swap after of the inner for loop of bubble sort. In this way, bubble sort will take $\Theta(n)$ time complexity.
- ✓ Merge sort runs in $\Theta(n)$ time
Merge sort uses the divide and conquer policy to sort the elements. As elements are already sorted in ascending order.
Recurrence relation for merge sort: $T(n) = 2T(n/2) + n$, This will give $\Theta(n \log n)$ time complexity.
- ✓ Insertion sort runs in $\Theta(n)$ time
When a new element that is greater than all the elements of the array is added, then there will be no swap but only a single comparison. In $n-1$ swaps, only 0 swaps and $n-1$ comparisons are there. The total time complexity in this case will be $\Theta(n)$. So, it is correct.

24. Correctly match the following

- | | |
|----------------------------------|-------------------------|
| (A) All pairs shortest path | (1) Greedy |
| (B) Quick Sort | (2) Depth-First search |
| (C) Minimum weight spanning tree | (3) Dynamic Programming |
| (D) Connected Components | (4) Divide and Conquer |

Codes:

Design and Analysis of Algorithm

	A	B	C	D
a	2	4	1	3
b	3	4	1	2
c	3	4	2	1
d	4	1	2	3

A. a

C. c

B. b

D. d

25. If b is the branching factor and m is the maximum depth of the search tree, what is the space complexity of greedy search?

A. $O(b/m)$

C. $O(b*m)$

B. $O(b+m)$

D. $O(b^m)$

Explanation:

1. In binary tree branching factor is 2 and space complexity for height n is $O(2^n)$.
2. In ternary tree branching factor is 3 and space complexity for height n is $O(3^n)$.
3. If branching factor is b and height is m for search tree then space complexity of greedy search is $O(b^m)$.

So, option (D) is correct.

26. Which of the following is the best method to go for the game playing problem?

A. Heuristic Search

C. Random search

B. Stratified search

D. Optimal search

Explanation:

We use a Heuristic approach, as it will find out brute force computation, looking at hundreds of thousands of positions. e.g Chess competition between Human and AI-based Computers.

27. The time complexity of the following function is (assume $n > 0$)

```
int recursion(int n) {  
    if(n == 1)  
        return (1);  
    else  
        return (recursion (n-1) + recursion(n-1));  
}
```

A. $O(n)$

C. $O(\log n)$

B. $O(2^n)$

D. $O(n^2)$

Design and Analysis of Algorithm

Explanation: Recurrence relation for the code = $T(n) = 2T(n-1) + k$. We can solve the recurrence using substitution method:

$$T(n) = 2T(n-1) + k$$

$$= 2(2T(n-2) + k) + k$$

$$= 2(2(2T(n-3) + k) + k) + k \dots$$

$$= 2^x T(n-x) + 2(1 + 2 + \dots + 2^{x-1})k$$

When base condition is met, i.e. $n=1$, $x=n-1$

$$= 2^{n-1} T(1) + k(2^n)$$

$$= O(2^n)$$

28. Floyd-Warshall algorithm utilizes _____ to solve the all-pairs shortest paths problem on a directed graph in _____ time.

A. Dynamic programming, $\Theta(V^2)$

C. Greedy algorithm, $\Theta(V^3)$

B. Greedy algorithm, $\Theta(V^2)$

D. Dynamic programming, $\Theta(V^3)$

29. The traveling salesman problem can be solved in;

A. Exponential time using backtracking algorithm

B. Polynomial time using backtracking algorithm

C. Polynomial time using branch and bound algorithm

D. Exponential time using dynamic programming or branch and bound algorithm

30. Dijkstra algorithm, which solves the single-source shortest paths problem using _____?

A. Greedy algorithm

C. Backtracking

B. Dynamic programming

D. Divide and conquer method

31. Match the following:

List - I

List - II

(a) The 8-Queen's problem

(i) Dynamic programming

(b) Single-Source shortest paths

(ii) Divide and conquer

(c) STRASSEN's Matrix multiplication

(iii) Greedy approach

(d) Optimal binary search trees

(iv) Backtracking

(a)

(b)

(c)

(d)

(1)

(iv)

(i)

(iii)

(ii)

Design and Analysis of Algorithm

(2)	(iv)	(iii)	(i)	(ii)
(3)	(iii)	(iv)	(i)	(ii)
(4)	(iv)	(iii)	(ii)	(i)

A. (1)

B. (2)

C. (3)

D. (4)

Explanation:

- The 8-Queen's problem is a Backtracking algorithm.
- Single-Source shortest paths is a Greedy approach
- STRASSEN's Matrix multiplication is a Divide and conquer
- An optimal binary search tree is a Dynamic programming.

